

Phenotype Strands After SIMB

Expression, Morphology, and Metabolite Production: What Was Measured and What to Run Next

Michael Volk

September 8, 2026

Abstract

Six phenotype strands were run against one shared cell embedding: knockout expression, morphology, the two jointly, betaxanthin production, free amino-acid pools, and beta-carotene production. Two were presented at SIMB 2026. Here is what each measured, against what ceiling, under what scoring rule, read from all 28 W&B projects and 2,187 runs rather than from a selection.

Every strand sits further from its ceiling than the manuscript’s placeholder text assumes, and the reason differs by strand. **Expression** is under-trained rather than saturated: no run has peaked at any budget up to 10,000 epochs, and the score tracks the epoch budget across a factor of 500 in budget. Its training loss and its metric come apart, and the culprit is specifically the *quantile* loss, which bottoms at epoch 463 and rises for the next 9,500 while Pearson climbs; squared error does not diverge, bottoming alongside the Pearson peak. Separately, the model orders genes at $r \approx 0.24$ while sitting level with “predict each gene’s mean” in squared error, because its predictions are 1.8× more spread out than that correlation justifies; one free rescale takes it clearly past the mean predictor without changing any correlation. **Morphology** has never been trained on more than a quarter of its screen, in any of 397 runs across eight projects, and its ceiling is measured here for the first time at 0.611. **Betaxanthin** is the strongest strand, at 47 % of a 0.914 ceiling, and its argument is coverage rather than accuracy: a flux-based method has features for 19 % of the screen and misses 73 % of its high producers. The **free amino-acid** profile of a deletion predicts that deletion’s betaxanthin at out-of-fold $r = 0.298$ while tyrosine, the chromophore precursor, predicts it at 0.064, so a real coupling exists on an axis that is not the precursor, and a plain auxiliary head does not use it. **Beta-carotene** is limited by its measurement, a hand-scored ordinal with one replicate per strain. And the **graph prior**, reported at chance in the previous revision, is not: asked the question that is defined for every phenotype rather than for expression alone, adjacency reaches AUC 0.66 against a 0.50 control, and which graph carries signal differs by phenotype.

The campaign that follows is expression first, single-phenotype and at full budget, because it is the strand where the next experiment is cheapest to interpret and the diagnosis is furthest along. Morphology at full scale is deferred rather than dropped.

How to read this. section 1.1 defines the words that recur. section 1 is the whole argument and every number a decision turns on. sections 2 to 6 are one section per strand, each ending in what it established and what it cannot say. section 7 collects the failures the strands share, which is where the leverage is. section 8 ranks the options, and section 9 sizes the next campaign. **Going straight to the campaign:** section 2.3, table 11 and section 9.

Revision 3 answers 127 review comments on revision 2, comment by comment. The ledgers are `notes-tex/019-simb-multimodal/review/round-2-dispositions.md` (this round) and `notes-tex/019-simb-multimodal/review/round-1-dispositions.md` (the previous one). Claims that revision 2 got wrong are corrected in place and named in the ledger, not argued away.

Working notes: `notes/experiments.019-simb-multimodal/*.md`, `notes/experiments.020-cachera-betaxanthin/*.md`

Generating scripts: the `scripts/` directory of `experiments/019-simb-multimodal/`, `experiments/020-cachera-betaxanthin/`, and `experiments/023-metabolome-betaxanthin-joint/`

Leaderboard: `experiments/019-simb-multimodal/results/round_leaderboards.csv`

Section status: ✓ ready ■ author-reviewed, keep stable ✗ not done, agents may edit freely

Provenance: ▲ not in the library mirror; verify at the linked source ● from a review’s summary table, not the primary paper

Contents

1	Summary ✗	4
1.1	Terms used throughout ✗	4
1.2	Where each strand stands ✗	5
1.3	What was established ✗	6

1.4	The decision ✗	8
2	Expression ✗	8
2.1	The task and its ceiling ✗	8
2.2	The finding of the round: nothing was trained to saturation ✗	9
2.3	The pinball loss turns early; squared error does not ✗	10
2.4	The perturbation operator has no pair term ✗	12
2.5	The largest signal in the strand is imputation, and it is orthogonal ✗	13
2.6	Why the byte count is misleading ✗	16
3	Morphology ✗	17
3.1	The target ✗	17
3.2	The ceiling was never measured until now, and it is high ✗	18
3.3	The strand was never really run, on two counts ✗	18
3.4	Which second label should morphology be paired with ✗	19
3.5	A scalar target is a reasonable next step, and this is which scalar ✗	20
4	Betaxanthin production ✗	21
4.1	The task, its ceiling, and the two names that get confused ✗	21
4.2	Three betaxanthin numbers, two scoring rules, and one honest comparison ✗	21
4.3	The published accuracy comparison is empty for both methods ✗	23
4.4	Both methods have real signal, and it lives in the tail ✗	24
4.5	The two methods find different genes ✗	24
4.6	The real difference is coverage ✗	25
4.7	Where the comparison is soft ✗	27
5	Amino-acid pools, and whether they help production ✗	28
5.1	The target ✗	28
5.2	These runs turn within their budget, and expression does not ✗	28
5.3	The precursor hypothesis is wrong, and the profile hypothesis is right ✗	28
5.4	Is the coupling redundant with what betaxanthin supervision already teaches ✗	29
5.5	Does the metabolome head actually help the betaxanthin head ✗	31
6	Beta-carotene production ✗	32
6.1	Why this target is different from the other pigment ✗	32
6.2	What was measured ✗	33
7	What the strands have in common ✗	34
7.1	What the leaderboard covers, and what it does not ✗	34
7.2	One table, read the same way ✗	35
7.3	Two opposite convergence failures, and neither was diagnosed from a curve ✗	36
7.4	The training objective diverges from the metric on expression alone ✗	36
7.5	Configuration choice moves results more than method choice ✗	37
7.6	Cross-modality overlap is real and small ✗	38
7.7	The graph prior, checked on one phenotype and then on six ✗	38
7.8	The perturbation axis is the binding resource ✗	42
8	What to run next ✗	42
8.1	Do now ✗	42
8.2	Deferred, and why ✗	43
8.3	The two questions worth a GPU campaign ✗	43
8.4	Structural options, further out ✗	43
8.5	What to do about Figure 3 and Figure 6 ✗	44
9	The expression campaign ✗	45
9.1	The budget arithmetic, measured ✗	45
9.2	Set the epoch budget at 4,000, and the reason is measurable ✗	45
9.3	Seeds: the noise floor is budget-dependent, and the short-budget figure was the wrong one ✗	46
9.4	Phase A, the objective, and why it goes first ✗	46
9.5	Phase B, the pair term, under the winning objective ✗	47
9.6	Morphology, deferred, sized for when it starts ✗	48
9.7	The joint question, which is the one Figure 3 is about ✗	48
9.8	What the campaign cannot fix ✗	49
10	The launch, 2026-08-31: what runs next and why ✗	49
10.1	Evidence 1: the replicate spread at 9,900 epochs is 0.0222 ✗	49
10.2	Evidence 2: what that spread costs in replicates ✗	49
10.3	Evidence 3: wall clock, and the packing cost measured ✗	50
10.4	The allocation: 10 GPU slots and four CPU baselines ✗	50
10.5	The model being launched: 1,194,509 parameters ✗	52
10.6	Graph attention, as currently built ✗	52
10.7	Perturbation encodings that supply one-to-many, in this attention style ✗	53

10.8	What the perturbation-response literature says, and what it changes here	✗	54
11	The readouts, 2026-09-08: three rounds, one large effect	✗	56
11.1	The factorial: embedding content is the only factor that moves the score	✗	56
11.2	The mechanism round: per-gene readout +0.03, under the round's resolution	✗	58
11.3	Packed five-day runs die of host memory	✗	59

1 Summary x

1.1 Terms used throughout x

Alphabetical, and every one of these is used later in a sentence a decision turns on. The four that caused the most trouble on the last read are **grid cell** (it is a hyperparameter configuration, never a data split), **regress out**, **reliability** against **ceiling**, and **under-shrinkage**.

Auxiliary head Any head added to a run for its effect on a *different* head’s score. Adding one changes two things at once, the gradient signal and the population of strains the run can use, which is why every comparison here pins the instance set.

Ceiling The highest Pearson correlation *any* predictor could reach against a target that carries measurement noise. It is $\sqrt{\text{reliability}}$, because a prediction can at best recover the reproducible part of the signal and correlation scales with the standard deviation rather than the variance. Estimated per strand from whatever replicate structure that dataset released, so the estimator differs by strand and is stated where each is used.

Configuration, grid cell, trial Three names for the same object, and none of them means a data split. A **configuration** is one setting of the hyperparameters (depth L , learning rate, graph mask on or off, decoder family). A **grid cell** is one configuration in a grid over those settings. A **trial** is one Optuna trial, which evaluates one configuration. **The data split is held fixed across all of them:** the rounds quoted here pin `data_module.split_seed`, so varying `seed` varies weight initialization and nothing else, and two runs of the same grid cell differ only by initialization. So “the grid cells span Spearman 0.013 to 0.158” is a statement about hyperparameters, not about lucky splits.

Head The output module for one phenotype. The encoder and the perturbation operator are shared; a head is the last few layers that turn the shared representation into one phenotype’s values. A run declares which heads are active, so *betaxanthin head* and *morphology head* name output modules, not models. A head is **not** a linear probe: it is trained jointly with the encoder and carries its own nonlinear layers. Where a number in this document *does* come from a linear probe on frozen inputs, it says so and names the estimator (**ridge oracle**, and the amino-acid to betaxanthin ridge in section 5).

Marginal Used in two senses in this document and disambiguated at every use. A **marginal correlation** is the correlation of one variable with the target on its own, with nothing else in the model, which is the sense in fig. 9a. A **marginal** prediction is a per-feature output that carries no joint structure across features. The colloquial sense, “small”, is not used here.

Metabolome head The head that predicts the 19 Muller amino-acid concentrations. It appears in two roles. As the **only** active head it is the amino-acid strand (section 5). Added **alongside** the betaxanthin head it is an **auxiliary head**: it is trained and scored, but the number being reported is still betaxanthin, and the question is whether sharing an encoder with it helps.

nmse Mean squared error divided by the variance of the target about each feature’s mean, so $\text{nmse} = 1$ is exactly “predict each gene’s training mean”. Below 1 beats the mean predictor; above 1 loses to it.

Head, pinball, and point(): three different objects These get conflated and the conflation hides the argument, so they are separated here. **dist:quantile** names the *head*, meaning the output parameterization: it emits $K = 19$ numbers per feature, the quantile knots at $\tau = 0.05, 0.10, \dots, 0.95$. **Pinball** (the check loss) is the *loss function*, $\max(\tau u, (\tau - 1)u)$ with $u = y - \hat{y}$, applied to each knot and then averaged over K and over features; the mean pinball over an even τ grid approximates the empirical CRPS. **DistHead.point()** is the *point estimate the metric reads*, and for a quantile head it is the median knot alone. So the head emits 19 numbers, pinball scores all 19, and **pearson_per_feature** reads 1. Other heads set this differently: **point** emits one number and is scored by MSE, **crps** and **nll** emit (μ, σ) with **point()** = μ , and **laplace_crps** emits (μ, b) whose **point()** is a *median* like the quantile head’s.

Oracle A deliberately cheating estimator, run to bound what is available in the data rather than to be deployed. The **ridge oracle** of section 2.5 is given m *measured* expression values from the held-out strain itself and predicts the remaining genes by ridge regression. No model in the project has that input at scoring time, which is the point: the gap between the oracle and the model is a statement about where the information is.

pearson_per_feature For each output feature (a reporter gene, a CalMorph parameter, one amino acid), the correlation *across strains* between prediction and measurement; then averaged over features. This is the objective everywhere in this document. It goes to zero when a model predicts each feature’s mean, which is exactly the failure worth detecting.

pearson_per_instance The companion metric, correlating *across features within one strain*. It answers a real question, whether the shape of one strain’s profile is right, but it is dominated by the profile shared by every strain: a model that predicts each feature’s training mean for every strain still scores near 0.4 on it. So a high **pearson_per_instance** is not evidence of strain-specific signal, and it is not the objective here for that reason rather than because the question is uninteresting.

Range restriction, and the Thorndike correction When a correlation is computed on a *selected* subpopulation, narrower than the population of interest, it is attenuated by the narrowing alone. Thorndike’s case 2 formula corrects an observed correlation back to the full population given the two standard deviations. It appears once, on the beta-carotene re-screen (section 6), which was run only on selected top hits and therefore spans a fraction of the original scale.

Regress out To remove a nuisance variable’s linear contribution from both sides before measuring their relationship. Concretely, for nuisance f (single-mutant fitness), predictor X (the 19 amino acids) and target y (betaxanthin): fit $y \sim f$ and $X \sim f$ by least squares, keep the residuals $y - \hat{y}(f)$ and $X - \hat{X}(f)$, and fit X -residual to y -residual. Whatever correlation survives is the part that is *not* explained by the two variables both responding to growth rate. It costs statistical power and it can only ever remove the *linear* part of the nuisance.

Reliability The reproducible fraction of a target’s variance, $1 - \sigma_{\text{noise}}^2 / \sigma_{\text{observed}}^2$. It is a variance ratio, and the **ceiling** is its square root. Two names exist because they are on different scales and get compared to different things: reliability sits beside R^2 , the ceiling sits beside a reported Pearson. Morphology’s 0.444 reliability and 0.611 ceiling are the same measurement stated twice.

roll_max The scoring rule: the maximum of a centered five-point rolling mean of a validation curve. An upward-biased order statistic whose bias grows with the number of epochs run, so every score here travels with its epoch count.

Selection inflation The upward bias in reporting the *best* of many trials. The maximum over n noisy trials exceeds the value a configuration chosen in advance would give, by an amount that grows with n and with the noise. On betaxanthin, taking the maximum over 37 trials is worth an estimated 0.064, so the headline 0.4301 corresponds to ≈ 0.366 for a configuration chosen without hindsight. It is not noise and it is not a result; it is a property of how the number was selected.

Strand One phenotype carried across a series of W&B projects. Six of them here.

Under-shrinkage, and the calibration ratio s Write s for the ratio of the prediction standard deviation to the target standard deviation, and r for the correlation. A well-calibrated predictor of a noisy target should be *less* spread out than the target, at $s = r$; shrinking toward the mean is the correct response to uncertainty. **Under-shrinkage** is $s > r$: predictions swing more than the correlation justifies. It is why a model can order genes correctly and still lose to the mean on squared error, and it is fixed by scaling predictions by r/s , which changes no correlation at all (section 2.3).

1.2 Where each strand stands ✗

Read the epoch column with the score column. Expression and beta-carotene peak in the last tenth of their best run, so those numbers are lower bounds set by the compute budget. Morphology and the expression-morphology joint arm peak at epochs 27 and 28 of runs that stopped at 65 and 70, which is not a result of any kind. Betaxanthin and amino acid peak in the first half of a 999-epoch run and then overfit.

Three betaxanthin numbers circulate, and only two of them are comparable. They are 0.4301, 0.4015 and 0.3705, and mixing them is the single most confusing thing in the previous revision.

- **0.4301** is the *Optuna* objective for the best trial of study **betaxanthin_002**. That objective is a raw running maximum over epochs, logged as **val/betaxanthin/pearson_per_feature_max**.
- **0.4015** is **roll_max** on the *same* W&B project, which first averages five neighboring epochs. Smoothing can only lower a maximum, so this is the same runs scored more conservatively, not a different or worse model.
- **0.3705** is **roll_max** on **020_v4**, which pins the 640 Merzbacher test genes out of training and therefore trains on 3,698 strains instead of 4,235.

So the apples-to-apples cost of holding out a competitor’s test set is 0.4015 against 0.3705, both **roll_max**. **0.4301 must never be differenced against a roll_max number**, and revision 2 did exactly that. Every score in table 1 is **roll_max**, which is what makes its rows comparable to each other.

strand	ceiling	best	of ceiling	epochs (peak/last)	runs
Expression (Kemmeren, Sameith)	0.775	0.228	29%	4002/4105	1353
Expression, masked-label objective	0.775	0.238	31%	9697/9900	16
Morphology (CalMorph, Ohya)	0.611	0.082	13%	27/65	183 (8 flat)
Expression and morphology, joint	–	0.062	–	26/70	214
Betaxanthin (Cachera)	0.914	0.401	44%	74/124	142
Betaxanthin with metabolome head	0.914	0.430	47%	176/496	46
Amino-acid pools (Mulleder)	–	0.209	–	354/998	97
Beta-carotene (Ozaydin)	0.544	0.137	25%	9/49	136

Table 1: Every strand, scored the same way. *best* is `roll_max`, the maximum of a centered five-point rolling mean of the validation curve, an upward-biased order statistic whose bias grows with epochs run, which is why the epoch column travels with it. *epochs (peak/last)* gives where the best run peaked and where it stopped: a peak in the last tenth means the run was cut off while still improving. Ceilings are measured by three different estimators because the three measurement types require them; the amino-acid cell is empty because Mulleder has one replicate per strain, and no ceiling is estimable. *flat* counts runs whose validation metric never left zero. **Coverage: complete. All 28 projects and all 2187 runs have had the validation curve needed to score them read** (table 25), so every *best* here is a maximum over the runs that were actually run. **Every number in this column is `roll_max`, and that is why the betaxanthin row reads 0.401 rather than the 0.4301 quoted elsewhere for the same study:** 0.4301 is the Optuna objective `val/betaxanthin/pearson_per_feature_max`, a raw running maximum over epochs, while `roll_max` first averages five neighboring epochs, which can only lower a maximum. The two are different scoring rules over the same runs, not different runs. section 4.2 does the reconciliation.

The two betaxanthin rows in that table are also not comparable to each other: the joint row trains on 3,832 strains against the plain row’s population, because the joint configuration restricts to genotypes carrying both phenotypes. The within-grid paired comparison in table 22 is the valid form of that contrast.

1.3 What was established ✗

1. **The expression strand contains no valid comparison between mechanisms.** All 67 scored arms stopped at or below 300 epochs, against a validation curve that dips at 200 to 300 and then climbs past its early peak. Every per-arm delta on record measures early-training transients.
2. **The headline 0.2382 is the maximum of eight identical-config replicates, whose spread is 0.0222.** The eight runs past 9,000 epochs share one configuration (`quantile` head, `lr` 3×10^{-4} , `L` = 6, hidden 90, mask, `s1_pool`, seed 0) and score 0.1663 to 0.2382, mean 0.1965. The expected maximum of eight such draws is 0.2282, so the config’s central estimate is **0.196 ± 0.022** and the headline is an order statistic. The same eight runs revise “never peaks”: five peaked between epochs 2,199 and 4,109 and never exceeded that value afterward; only the best draws peaked late. Both facts set the launch design (section 10): a head gap under ≈ 0.04 cannot be resolved with one run per arm, so arms get replicates, not just budget.
3. **Two separate defects were being collapsed into one phrase, and only one of them is a loss-versus-metric conflict.** Revision 2 said “the objective is fighting itself”, which invited the reading that squared error and Pearson are at odds. They are not. Stated as the two things they are:
 - (i) **The *pinball* loss is the only quantity that turns early.** It bottoms at epoch 463 and rises for the next 9,500 while Pearson climbs. Squared error does *not* do this: its minimum is at epoch 9,175, alongside the Pearson peak at 9,674, and on the second-longest run the two are one epoch apart. So the loss that is actually being minimized disagrees with the metric, while the loss that is not being minimized agrees with it. The consequence is narrow and it is about checkpointing: best-by-pinball-loss picks a model $\approx 9,200$ epochs too early, best-by-squared error would have picked within 500 epochs of the right one.
 - (ii) **Separately, the model is under-shrunk.** At its peak it orders genes at $r \approx 0.24$ while its `nmse` sits at about 1.00, level with “predict each gene’s mean” on squared error. This is not a conflict between two metrics; it is one number being mis-scaled. Its predictions are $1.80\times$ more spread out than the correlation justifies, and multiplying them by r/s moves `nmse` to 0.939 while changing no correlation at all.

Correction to revision 2, which overstated (ii). It said `nmse` “never returns below 1”, so the model was “no better than the mean”. Read on the raw logged curve rather than the smoothed one, `nmse` at the raw Pearson maximum is 0.9928, just *below* 1, while at the smoothed maximum it is 1.0070, just *above*. The two readings are about 1.8σ apart in epoch-to-epoch noise (table 5), so the sign of “beats the mean” is not resolvable and the defensible word is **parity**. The under-shrinkage ratio, unlike that sign, is nowhere near a boundary.

And on the objection that squared error “looks minimized at the beginning”: raw mse falls steeply, rises to a local maximum near epoch 900, then falls again to a *late global* minimum at epoch 9,184, essentially at the Pearson peak. It does come back, and it goes lower (fig. 1b).

4. **One expression run is a multi-day object, and this sets the campaign.** The best run took **91.4 hours, 3.81 days**, for 9,999 epochs at 32.9 s/epoch, and still peaked at 97% of the way through. One GPU delivers $\approx 2,500$ epochs per day, so the number of arms a campaign can afford is fixed before any of them are chosen.
5. **The perturbation operator provably cannot express a pair term, and masked unmasking does not supply one. This is the most consequential structural finding here**, because the task is to predict how reporter gene i moves when gene p is deleted, which is by construction a statement about the *pair* (p, i) .

The mechanism is specific and it is not a claim that attention in general cannot do this. It is that *this* attention, at *one* perturbed gene, cannot. The perturbation enters through a cross-attention whose keys are the perturbed genes. With a single deletion the key set has one element, a softmax over one element is identically 1 regardless of the query, and the attention has no remaining degree of freedom: redrawing W_Q and W_K at standard deviation 10 changes the output by exactly 0.0, and 16,200 of 32,760 attention parameters are dead. The model reduces to $g(h_i + c_b)$, in which reporter identity i and strain identity b meet once, by addition. **So yes: this operator only becomes expressive when more than one gene is perturbed**, which is why it works on the trigenic task and degenerates here. Fixing it does not require abandoning attention, it requires a term in which i and b multiply rather than add (table 6).

The scGPT-style masked scheme is a different axis and does not repair this. It conditions gene i ’s token on *other genes’ measured expression values* in the same strain, through features that are gated by a reveal mask. At $k = 0$ no gene is revealed, so those input features are zero *as inputs* and the whole conditioning branch contributes nothing. The measurements themselves are of course not zero; what is zero is the branch that reads them, because nothing has been revealed to it. Scoring happens at $k = 0$.

Capacity is not the binding constraint either: a rank-32 reconstruction of the target matrix reaches 0.727 against a best model run of 0.238. A rank-64 pair term already reaches 0.2274 in 2.0 days against the masked objective’s 0.2362 in 3.8.

6. **Morphology has never been trained on its own data.** The Ohya screen holds 4,718 deletions; every morphology run used the 1,440 that also carry expression, giving $n_{\text{train}} = 1,161$. Its ceiling, measured here for the first time, is **0.611** over 278 features, with 201 of them individually above 0.5. For a second-label experiment, fitness shares 4,220 strains with morphology against expression’s 1,440.

One qualification on the “cap” language. Pairing two phenotypes caps both arms at their intersection only because `require_modalities` drops any genotype missing either one. Masking the loss over absent outputs instead would let a strain with morphology and no expression still contribute morphology gradient, and that is the standard way around the cap. It is a training-path change rather than a query change, and it is not what any run here did.

7. **The free amino-acid profile predicts betaxanthin; the precursor does not.** Over the 4,432 deletions the two screens share, the 19-dimensional pool predicts betaxanthin at out-of-fold $r = \mathbf{0.298}$, while tyrosine alone, the chromophore precursor, predicts it at **0.064** and correlates marginally at -0.076 . On the 724 deletions with no Costanzo fitness record, which carry twice the betaxanthin spread, the profile predicts at 0.455.

This is a statement about two *measurements*, with no network in the loop, and it is not a claim that the pool is worth more than the betaxanthin labels themselves: a model trained on betaxanthin reaches 0.40 on the same phenotype, above the 0.298 a ridge on the pool reaches. The open question the number raises, and does not answer, is whether the pool carries anything the betaxanthin labels do not already carry (section 5.4).

8. **Adding a metabolome head to a betaxanthin run has not helped, and the experiment was naive.** Paired within grid cell, joint minus control comes out negative (table 22). Read that as one crude attempt rather than as a verdict on the coupling: a plain auxiliary head is the least structured way to inject a second phenotype, most cells carry $n = 1$ per arm, and the auxiliary head’s own score (0.05 to 0.18) is well below the 0.21 the amino-acid strand reaches when it is the only head, so the run is paying capacity for a head that is itself underperforming. The coupling in the data and the failure of this particular head to exploit it are separate facts and neither cancels the other.
9. **The graph prior is not at chance, and revision 2’s recommendation to free the nine masked heads is withdrawn.** That recommendation rested on one statistic on one phenotype: whether graph proximity predicts which *reporters* respond to a deletion. On that question the answer is still no, on all nine graphs, AUC 0.4961 to 0.5057 against a degree-preserving control.

But it is the narrower of two ways to state the premise the mask encodes. Ask instead whether *deletions of two adjacent genes give similar phenotypes*, which is defined for every strand rather than expression alone, and the

same graphs on the same strand reach AUC **0.6579** on STRING database against a rewired control of 0.4971. The channel carries information and the null was an artifact of the question.

And the answer differs by phenotype, which is the operational finding. Across six strands, STRING database and co-occurrence carry signal on four; physical interaction is the strongest graph for *fitness* and sits at the noise floor for *morphology*; *tf*link carries nothing anywhere; and regulatory interaction is clearly *negative* on betaxanthin. So a graph worth keeping for one phenotype is not automatically worth keeping for another, and removing masks on a single strand’s evidence would have been the wrong move. Separately, *direction* carries real signal that the mask deletes: *tf*link TF → target gives 0.5508 and regulatory interaction 0.5239 against controls near 0.501, and *_build_attention_mask* symmetrizes that away.

10. **Betaxanthin is the strongest case, and its strength is coverage.** yeast-GEM reaches 19% of the screen and misses 73% of its high producers. On genes a flux-based method has no features for, CGT’s top 25 is enriched 4.4× over base rate. Where both methods can be scored they are tied, they find largely different genes (9 shared of 29 each), and their union reaches 49 of 100.
11. **Hyperparameter choice moves results more than method choice, and this is not a statement about data splits.** The split is pinned across every cell quoted here, so “grid cell” means one hyperparameter configuration and nothing else (section 1.1). Betaxanthin cells span Spearman 0.013 to 0.158 against a between-method gap of 0.0014 on AUC; the amino-acid strand’s best run is 0.209 against a median of 0.0570 over all 97 of its runs. Re-running an identical configuration moves the betaxanthin objective by $\sigma \approx 0.030$, so the spread across configurations is several times the run-to-run floor and is real. The operational consequence for the Merzbacher comparison is the one that matters: as long as both methods are scored on the same split, the comparison stands, and the configuration sensitivity is reported beside it rather than being the reason to distrust it.

1.4 The decision ✗

Expression first, and single-phenotype. It is the strand where the diagnosis is furthest along, where the next experiment is cheapest to interpret, and where the most effort is already sunk. Concretely: resume the best checkpoint rather than restarting, and run it until the validation curve turns, since no run has yet observed a maximum at any budget up to 10,000 epochs. Multi-GPU DDP on IGB is the lever that makes this affordable (section 9).

Then the small-target strands, because they are cheap. Betaxanthin and the amino-acid pool train in hours rather than days and carry few output features, so a joint or constraint-coupled arm can be iterated there while an expression run occupies a GPU. The question they answer, whether a second label helps at all, is the same question the expression-morphology pairing asks and is far cheaper to ask.

Morphology at full scale is deferred, not dropped. It is the single largest change available to Figure 3 and it is a config change rather than research. It is also a longer run than expression: nearly four times the strains at a comparable architecture, with the epoch budget unmeasured. Running it before expression is understood would occupy the hardware for the strand we understand least.

Keep the graph masks; add unconstrained heads beside them. The cross-phenotype probe settles this: the graph channel carries real information, unevenly across graphs and across phenotypes. Two changes follow, both cheap and neither a removal. Stop symmetrizing the two directed relations in the mask builder. And add unconstrained heads beside the masked ones, so the model can learn structure the graphs do not carry without giving up the ones that do. The one graph the probe is decisively negative on is *tf*link, at the noise floor on all six strands.

Figure 6 is ready to be argued. on coverage rather than on accuracy, with the amino-acid material entering as a coupling measurement and as the motivation for a constraint-based metabolic layer, not as a joint-training result.

Figure 3 is not ready. A per-gene Pearson of 0.24 against a ceiling of 0.775 is not a headline, and the manuscript’s placeholder values of 0.543 and 0.619 are far from the measured maxima of 0.238 and 0.082. Those placeholders come out of the manuscript regardless of what the next campaign finds.

2 Expression ✗

2.1 The task and its ceiling ✗

The **expression strand** predicts, for a strain carrying one gene deletion, the $\log_2$ ratio of every measured gene’s transcript abundance against wild type. Targets come from the Kemmeren 2014 knockout microarray compendium

(doi:10.1016/j.cell.2014.02.054) and the Sameith 2015 double-deletion compendium (doi:10.1038/sdata.2015.15), fused in the `fig3_core` build. Of the 1,556 records in that build, 1,484 are single deletions and 72 are Sameith doubles, so 95.4% of the training signal carries exactly one perturbed gene. SRC: `experiments/019-simb-multimodal/results/fig3_overlap_census.json`

The **reproducibility ceiling** is the highest correlation any predictor could reach against a target that carries measurement noise. It is estimated here from the 82 deletions measured independently in both compendia, as the mean over reporter genes of $\sqrt{\text{clip}(\hat{\rho}_g, 0, 1)}$ where $\hat{\rho}_g$ is that gene’s test-retest correlation across the paired strains. The estimate is **0.7746**. SRC: `experiments/019-simb-multimodal/results/expression_ceiling_replicate.json`

The best validation score observed on this task is `pearson_per_feature` = **0.2382**, from run `hx8pxdic` in project `torchcell_019_expr_v9`, whose peak sits at epoch 9,697 of the 9,900 it reached. The best in `torchcell_019_expr_v8` is 0.2276, run `b50f93ju`, peaking at epoch 4,002 of 4,105.

That headline is the maximum of eight replicates, and the spread has now been measured. Every expression run past 9,000 epochs on the leaderboard, eight of them, shares one identical configuration: `quantile` head trained by pinball loss, $\text{lr } 3 \times 10^{-4}$, dropout 0.1, $L = 6$, hidden 90, mask prior, `s1_pool`, seed 0. They are replicates in everything but name, and they span 0.1663 to 0.2382: mean **0.1965**, standard deviation **0.0222**. Blom’s expected maximum of eight normal draws at that mean and spread is 0.2282, and the observed 0.2382 sits at $z = 1.89$ above the replicate mean. **So 0.2382 is the best of eight draws, an upward-biased order statistic, and the config’s central estimate is 0.196 ± 0.022 .** As a fraction of the 0.775 ceiling that is 25% on average and 31% for the best draw. Every use of 0.238 in this document should be read with that qualification, and the launch design in section 10 is built around the 0.0222 figure. SRC: `experiments/019-simb-multimodal/results/launch_plan_evidence.json`

The same eight runs settle where the peaks sit, and the answer is mixed rather than “never turns”: three peaked in their last 12% (epochs 8,859, 9,691, 9,697) but **five peaked between epochs 2,199 and 4,109 and never exceeded that value in the following six thousand**. The two best scores did peak late, so lateness and score are correlated in this small sample, but “expression never peaks” is falsified as a statement about the config; what survives is that the best draws peak late. SRC: `experiments/019-simb-multimodal/results/round_leaderboards.csv`

The metric is the per-reporter correlation across strains, averaged over reporters, which is the quantity that goes to zero when a model predicts each gene’s mean; the companion per-strain metric stays near 0.4 under exactly that collapse and is not the objective. The scoring rule is `roll_max`, the maximum of a centered five-point rolling mean of the validation curve. It is an upward-biased order statistic whose bias grows with the number of epochs run, so it is comparable only between runs of similar length, which is why the epoch count travels with every number quoted from the leaderboard.

The manuscript’s Results section currently states $r = 0.543$ for expression. That string sits inside a paragraph marked [FILLER - R3.] and is a placeholder, not a measurement.

2.2 The finding of the round: nothing was trained to saturation $\times$

Sixty-seven runs across waves 1 to 4b were scored, and not one of them ran past 300 epochs. Against a validation curve that dips at epochs 200 to 300 and then climbs past its early peak, every one of those runs was stopped inside the dip.

SRC: `experiments/019-simb-multimodal/results/decoder_arms_torchcell_019_expr_v8.csv`

wave	runs	epoch range
wave 1	10	70–140
wave 2	5	76–140
wave 3	36	18–276
wave 4a	8	77–81
wave 4b	8	300
all	67	18–300, none above 300

Table 2: Every scored arm in the expression round stopped at or below 300 epochs.

Four long runs were later allowed to continue. All four died at 12.60 to 12.61 hours of wall clock at 1,620 to 1,625 epochs, which is a scheduler limit rather than convergence, and their validation peaks sit at 93 to 99% of the way through the run. Eval-mode training Pearson reached 0.652 to 0.725 and was still climbing when they were killed.

Longer runs have since been allowed. The picture did not change: the best v8 run peaks at epoch 4,002 of 4,105 and the best v9 run at epoch 9,697 of 9,900. Raising the epoch budget from 1,600 to 10,000 moved the score from 0.204 to 0.238 and moved the peak with it, so the curve is still rising where it was cut off. **No maximum of the validation correlation has ever been observed on this task**, at any budget tried, and the saturation epoch remains unknown.

That claim is about the correlation, and the squared-error curve is a separate question worth stating in the same breath. Validation `mse` falls steeply for the first few hundred epochs, rises to a local maximum near epoch 900, and then falls again for the remaining nine thousand to a *late global* minimum at epoch 9,184, essentially where the correlation peaks. So the impression that squared error is minimized early and never returns is close but not right: it does return, and it goes lower. Both curves are in fig. 1b.

Read across all nine expression rounds, the score is a function of the budget. Every expression project, read the same way and sorted by score, comes out in almost exactly the order of its epoch budget.

project	runs	best	epochs of that run	n_{train}
expr_v9	16	0.2382	9,900	1,244
expr_v8	163	0.2276	4,105	1,244
expr_v7	295	0.1631	152	1,244
expr_v6	158	0.1628	124	1,244
torchcell_019-simb-multimodal_cgt_multitask	496	0.1297	25	1,137
expr_v2	120	0.0996	25	1,161
expr_v3	60	0.0874	20	1,161
expr	26	0.0822	19	1,126
expr_v5	35	0.0719	27	1,161

Table 3: Every expression round, read the same way and sorted by score. The order is almost exactly the order of the epoch budgets, across rounds that were varying capacity, targets, graphs, decoders and objectives. Whatever those rounds were testing, the thing that moved the number was how long the run was allowed to go. Training-set size moved by only ten percent across the same span and does not order the column.

These rounds were not varying the same thing. They varied capacity, target normalization, which compendium was included, graph regularization, decoder family, distributional head and objective. None of that orders the column; the epoch budget does, across a factor of 500 in budget and a factor of 3.3 in score. Training-set size moved by ten percent over the same span and does not order it either.

This is the strongest available evidence that the binding constraint on this strand has been compute rather than mechanism, and it is only visible with every round in one table. It is also a caution about reading it too hard: `roll_max` is an upward-biased order statistic whose bias grows with epochs run, so part of the ordering is the scoring rule rewarding longer runs. That bias is a few thousandths at these lengths and the spread here is 0.07 to 0.24, so it explains a sliver rather than the trend.

Consequence. An arm comparison at 300 epochs is not a small effect measured noisily. Both arms were stopped before either had the chance to separate, so the difference between them reflects early-training transients. The null sink arm reads $+0.0024 \pm 0.0090$ over four seed pairs and post-perturbation mixing reads -0.0023 ± 0.0076 , and neither number is evidence about its mechanism. More seeds do not repair this: replicates buy precision on the wrong estimand.

2.3 The pinball loss turns early; squared error does not ✗

Correction. An earlier version of this section said loss and metric “move in opposite directions”. That is true of the *pinball loss* and false of squared error, and collapsing the two hid the finding. The two longest runs ever trained were pulled and read directly. SRC: `experiments/019-simb-multimodal/scripts/expression\_objective\_diagnosis.py`

Correction to revision 2: the model is at *parity* with the mean predictor, not reliably worse than it. Revision 2 said `nmse` “never returns below 1 after about epoch 400”, so the model was “no better than predicting each gene’s mean”. Reading the raw logged curve rather than the smoothed one, that is wrong, and it is wrong in the direction the last review suspected.

What is robust is parity, not defeat: on v9 the model ends up level with the mean predictor in squared error while ordering genes at $r \approx 0.24$, and on v8 it is 3 to 4% worse. What is *not* robust is the stronger sentence revision 2 carried, and any figure or manuscript line resting on “worse than the mean” should be rewritten to “level with the mean, and better than it once calibrated”.

The calibration statement below is the robust one, and it is what the parity result is really about.

That is arithmetic rather than a mystery, and the arithmetic is worth writing out because everything downstream, including table 27, is one instance of it. Write s for the ratio of prediction standard deviation to target standard deviation and r for the correlation. For predictions that are a scaled copy of a correlated signal,

$$\text{nmse} = 1 + s^2 - 2rs.$$

quantity	v9 M_fine (masked)		v8 V_basis64	
	value	epoch	value	epoch
val loss minimum (pinball)	0.2376	463	0.2610	141
val mse minimum	0.0391	9,175	0.0397	3,922
val Pearson maximum	0.2362	9,674	0.2274	3,921
val nmse minimum	0.9981	213	0.9925	145
val nmse at the Pearson peak	1.0100	–	1.0395	–
run length (epochs)	9,999		4,106	
wall clock	91.4 h (3.81 days)		47.9 h (2.00 days)	
seconds per epoch	32.9		42.0	

Table 4: The pinball loss bottoms in the first few hundred epochs and rises for the rest of the run. Squared error bottoms alongside the Pearson peak, within 500 epochs on v9 and within one epoch on v8. Both runs replicate the pattern, and they use different mechanisms and different epoch budgets. **Why the Pearson values differ slightly from table 1:** this table smooths over 25 logged points to *locate* turning points, the leaderboard over 5 to *score* them, so v9 reads 0.2362 at epoch 9,674 there and 0.2382 at 9,697 on the leaderboard. The leaderboard value is the score; these are locators.

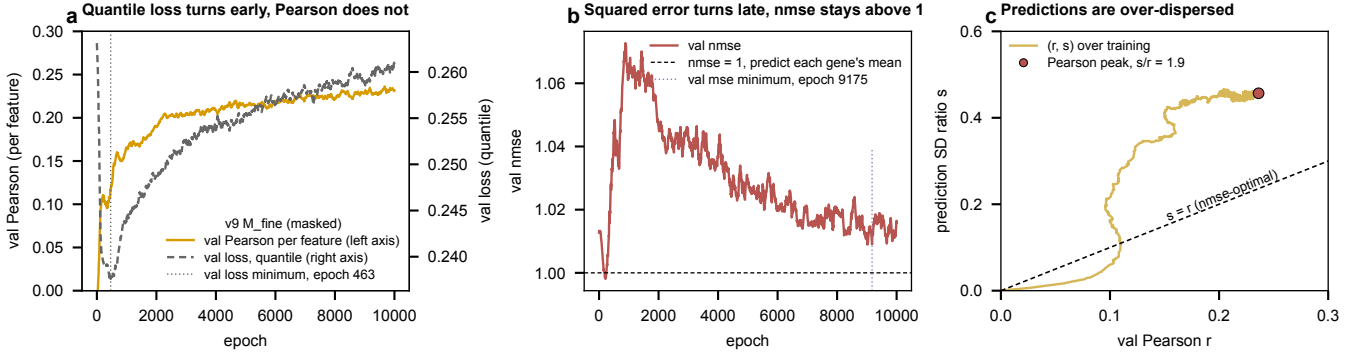


Figure 1: The v9 run, 10,000 epochs. (a) The pinball loss reaches its minimum at epoch 463 and rises thereafter while Pearson climbs. (b) nmse dips just under 1 at epoch 213, rises to 1.073 near epoch 900, then falls back for the rest of the run without ever returning below 1; squared error’s own minimum is at epoch 9,175. (c) The prediction standard-deviation ratio s against the correlation r , with the nmse-optimal line $s = r$.

run	scoring of the peak	peak r	epoch	nmse there
v9 hx8pxdic	raw per-epoch maximum	0.2463	9,177	0.9928
v9 hx8pxdic	5-point smoothed maximum	0.2395	9,682	1.0070
v8 b50f93ju	raw per-epoch maximum	0.2370	3,913	1.0312
v8 b50f93ju	5-point smoothed maximum	0.2304	3,914	1.0390

Table 5: Whether the model beats “predict each gene’s mean” in squared error depends on which epoch you call the peak. On v9 the raw maximum sits at nmse = 0.9928, below 1, and the smoothed maximum at 1.0070, above it. The epoch-to-epoch standard deviation of nmse over the last 5,000 epochs is 0.0077, so the two readings are about 1.8σ apart and the sign of “beats the mean” is inside the logging noise. v8 sits above 1 on both readings.

This is a parabola in s . Differentiating, $2s - 2r = 0$, so it is minimized at $s^* = r$ with value $1 - r^2$. The identity reproduces the logged `nmse` to within 0.016 on both runs, so it describes these models rather than an idealization of them. At its raw Pearson peak v9 sits at $s = 0.4436$ against $r = 0.2463$, so its predictions are $1.80\times$ more spread out than its own correlation justifies; v8 is $2.13\times$ ($s = 0.5055$ against $r = 0.2370$).

That ratio, unlike the sign of `nmse` - 1, is not close to a boundary and does not move with the smoothing choice.

What the post-hoc rescale is, concretely. Take the trained model, change nothing inside it, and multiply every prediction by the scalar $r/s = 0.2463/0.4436 = 0.555$ before scoring. That is the whole operation. It is legitimate because r and s are estimated on the training split and applied to validation, so no validation label enters. Two properties make it free:

- **No correlation changes.** Pearson is invariant to multiplying one variable by a positive constant, so every `pearson_per_feature`, every Spearman, and every ranking is bit-identical afterward.
- **nmse moves to its optimum.** By the parabola above, scaling s to r takes `nmse` to $1 - r^2$, which is 0.939 on v9 and 0.944 on v8, both clearly below 1 rather than at the boundary.

So the model sitting at parity with the mean and the model beating it are the same model with one multiplication applied. **The right way to state the result is: the model is level with the mean predictor as trained, and 6% better than it once calibrated, while its ordering is unchanged throughout.** The uncalibrated `nmse` is a reporting defect rather than a capability limit.

This is not scheduled as an experiment. It changes no correlation, so it cannot improve the strand’s headline number, and it is worth doing at the point a figure is drawn rather than now (section 8).

What this does and does not say about checkpointing. Best-by-loss on the *pinball* loss picks a model $\approx 9,200$ epochs early, which is the defect that motivated best-by-metric checkpointing. Best-by-mse would have picked within 500 epochs of the right one on v9 and within one epoch on v8. So checkpointing is a narrower problem than it looked, and it is not what is holding the strand at 0.24.

Still not measured. Whether training on a different objective, rather than rescaling after the fact, reaches a higher Pearson. section 8 sets that as an arm rather than an assumption.

2.4 The perturbation operator has no pair term ✗

At $|S_b| = 1$ the cross-attention that carries the perturbation runs its softmax over a one-element key set, so the attention weight is identically 1. Re-drawing W_Q and W_K at standard deviation 10 changes the output by exactly 0.0, and 16,200 of 32,760 attention parameters are dead. SRC: `experiments/019-simb-multimodal/results/perturbation_selector_degeneracy.json`

The model therefore reduces to $g(h_i + c_b)$: gene identity i and strain identity b meet once, by addition, and no term depends on the pair (p, i) of deleted gene and reporter gene. The rank of the pair term each candidate mechanism can express is what wave 6 was built to ladder.

mechanism	form	pair rank
<code>V_ref</code> additive	$g(h_i + c_b)$	0
<code>V_sink</code> gated attention	one bounded scalar per head	9
<code>V_basis</code> {16, 32, 64}	$\sum_j b_{ij}(h_i) a_j(z_S)$	r
<code>V_film</code> , <code>V_hadamard</code>	diagonal multiplicative	$d = 90$

Table 6: Pair-rank ladder. The reference arm cannot express any dependence on which reporter is being predicted given which gene was deleted.

Was the ladder actually run. No. table 6 is a *derivation*, not a result table: it states the rank of the pair term each mechanism is algebraically capable of expressing. Only two rungs have been trained, the additive reference and `V_basis64`, and they are not a clean comparison because their epoch budgets differ by $2.4\times$. The rest of the ladder is the Phase B proposal (section 9), and the zero in the top row is a proof about the reference operator rather than a measurement of it.

Capacity is not what binds, and here is what that check was. A pair term of rank r can only help if the target matrix itself has structure at that rank. To check, take the $1,482 \times 6,169$ matrix of $\log_2$ ratios, compute its best rank- r approximation by truncated SVD, and score that approximation with the same `pearson_per_feature`. This is

an upper bound rather than a model: it is fit on the matrix it is scored on, so no held-out claim is being made. The question it answers is narrow and useful, whether r numbers per strain are enough to describe the data at all.

They are. A rank-32 reconstruction reaches 0.7265 and rank-64 reaches 0.7799, both near the 0.775 reproducibility ceiling, against a best model run of 0.238 (replicate mean 0.196). **So a rank-64 pair term has enough expressive room by a wide margin**, and if such an arm comes back null, the null is about the mechanism or the optimization rather than about the rank being too small to matter. SRC: experiments/019-simb-multimodal/results/lowrank_output_ceiling.json

Masked unmasking does not supply the pair term, and this is provable rather than arguable. The scGPT-style scheme conditions gene i 's token on the measured values of other genes in the same strain:

$$h_{b,i}^{\text{obs}} = h_{b,i}^{\text{pert}} + g_{\text{obs}} \cdot \text{MLP}([y_{b,i} m_{b,i}, m_{b,i}]),$$

where $m_{b,i}$ marks gene i as revealed. At $k = 0$ nothing is revealed, so every $m_{b,i}$ is zero and both arguments to the MLP are zero *as inputs*: the measured values $y_{b,i}$ are of course not zero, but they are multiplied by the reveal mask before they enter, so the branch that reads them is switched off rather than the data being absent. The forward pass is then **identical** to the unconditioned model. Scoring happens at $k = 0$. So the added term contributes nothing at the point the number is taken, and it cannot create a dependence on the pair (deleted gene, reporter gene) no matter how well the unmasking is trained.

Which suggests scoring at $k > 0$, and that is a real option with a caveat. If the conditioning branch is worth having, the natural response is to score where it is switched on. That changes the task: at $k > 0$ the model is doing imputation given some measured genes, which section 2.5 shows is a far stronger and largely orthogonal capability, and it is no longer a genotype-to-expression number comparable to anything else in this document. **Worth reporting as a second, clearly-labeled metric** rather than as a replacement, and the gene-budget curve is the right way to report it, since it says what the capability costs to use.

The two are different axes and the round conflated them:

- **Masked conditioning** answers *given some of this strain's genes, what are the rest*. Its channel is other genes' measured values, and table 7 shows that channel is worth a great deal and is measured to be orthogonal to genotype.
- **The pair term** answers *given which gene was deleted, which reporters move*. Its channel is the deletion identity, which is present at $k = 0$.

What can carry a pair term at $k = 0$ is the table 6 ladder: a multiplicative or FiLM conditioner ($h_i \odot (1 + \gamma(z_{S_b}))$, rank $d = 90$), a factored readout ($\sum_j b_{ij}(h_i) a_j(z_{S_b})$, rank r), or masked graph attention applied **after** the perturbation operator so that gene i learns about the deletion through its own neighbors. The last of these is the only one that makes the pair term a function of network distance, and it is the one that has never been built.

Read the two best runs together, carefully. The best masked run reaches 0.2362 at epoch 9,674 of 9,999; the best pair-rank-64 run reaches 0.2274 at epoch 3,921 of 4,106. That is a 0.009 gap at a $2.4\times$ difference in epoch budget, so it is not an arm comparison. What it does establish is that a rank-64 pair term gets within 4% of the masked objective for 40% of the compute, which is the cheapest existing evidence that the pair axis is worth the next round.

2.5 The largest signal in the strand is imputation, and it is orthogonal ✗

What the oracle is, step by step, since it is quoted throughout. It is a Gaussian conditional mean with a ridge loading, fit once and then applied. Nothing in it is learned by gradient descent and no network is involved.

1. Build Y , the $1,482 \times 6,169$ matrix of $\log_2$ ratios, and split it by *strain* into 1,172 train, 155 tune and 155 validation.
2. Form residuals $R = Y - \mu$, where μ is each gene's mean over the training strains only.
3. Estimate the gene-gene covariance Σ on the training strains only.
4. For a held-out strain, choose a set M of m genes and **reveal their measured values**. Predict the remaining $U = 6,169 - m$ genes as $\mathbb{E}[R_U | R_M] = \Sigma_{UM}(\Sigma_{MM} + \lambda I)^{-1} R_M$.
5. The ridge term λI is what makes Σ_{MM} invertible once m approaches the number of training strains, and this is the only sense in which the estimator is a "ridge". λ is picked on the 155 *tune* strains and then applied unchanged, so the bound cannot be inflated by tuning on the strains it is scored on.

6. Score the unrevealed genes with the same `pearson_per_feature` used everywhere else.

At $m = 0$ nothing is revealed, the prediction is the per-gene mean, and the score is exactly 0 by construction. **That is why the oracle is not a rival model: the model is scored at $m = 0$, where the oracle has nothing.** The oracle exists to measure how much information sits in one strain's other genes. SRC: `experiments/019-simb-multimodal/scripts/masked_conditioning_oracle.py`

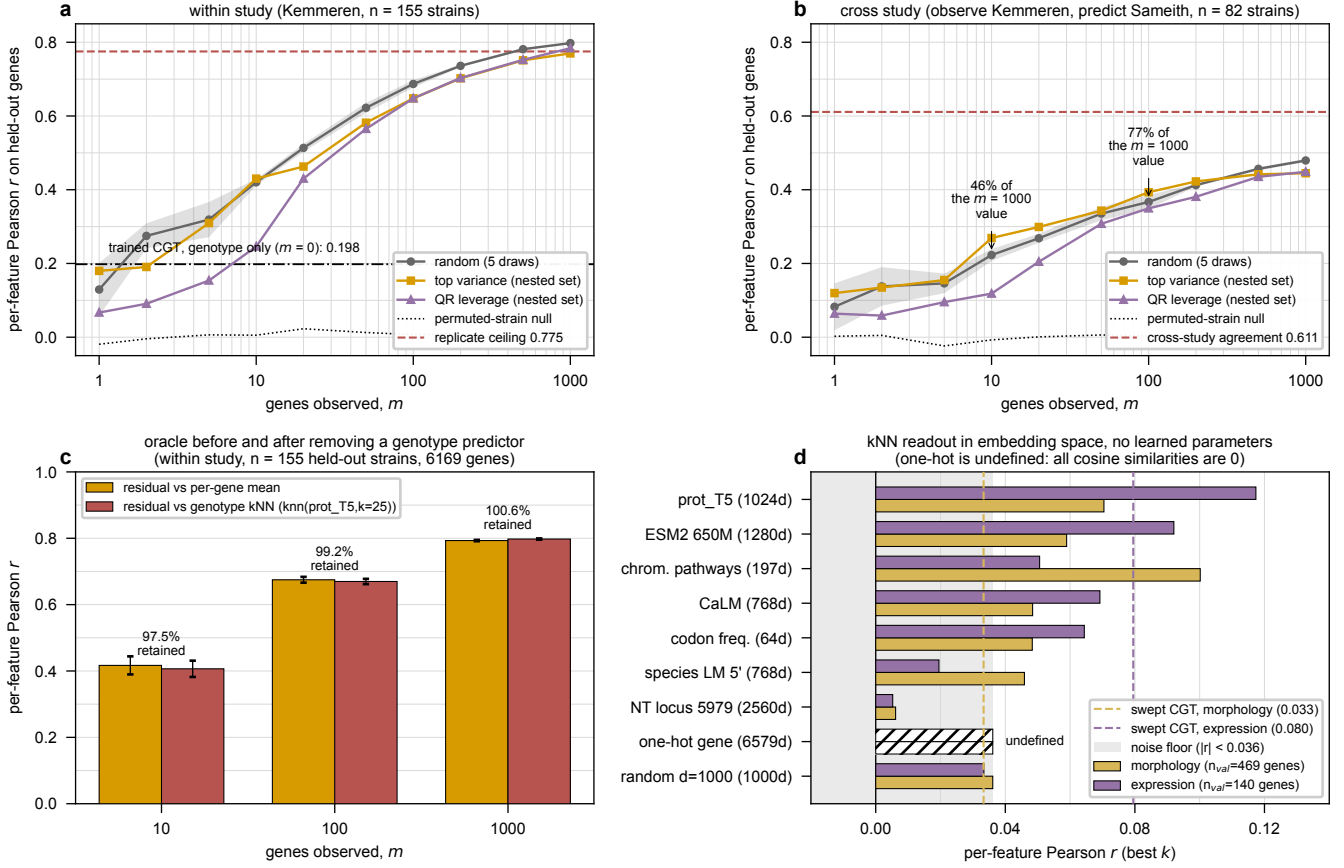


Figure 2: (a) Within-study imputation against the number of revealed genes m , for three ways of choosing which genes to reveal, against the 0.775 replicate ceiling. (b) The same across studies, observing Kemmeren and predicting Sameith, where the relevant ceiling is not 1 but the 0.611 agreement between the two studies' own measurements of the same 82 deletions; the annotations mark that 10 revealed genes reach 46 % and 100 reach 77 % of the $m = 1000$ value. Note that a variance-chosen set beats a random one at the cheap end and loses at $m \geq 500$, while the theoretically tidier QR-leverage rule is below random almost everywhere. (c) The conditioning gain before and after replacing the per-gene mean with a genotype predictor: 97.5 to 100.6 % of it survives, so the two channels are nearly non-overlapping. (d) The kNN embedding probe, a parameter-free readout that predicts a held-out deletion's phenotype as a similarity-weighted average of the nearest *other* deleted genes' phenotypes, against the noise floor set by random embeddings of matched dimension. One-hot is undefined rather than poor: with orthogonal encodings every cosine similarity is zero and there is no neighbor to average. **Provenance caveat:** the two dashed "swept CGT" reference lines in (d) and the genotype-only reference in (a) are constants recorded in the plotting script and the working notes rather than values re-derived from a committed result file, so treat them as approximate orientation marks rather than as scored numbers.

A ridge oracle that observes m randomly chosen genes of a held-out strain and predicts the rest reaches far past anything the model does.

The low end is already strong, which is easy to miss at this resolution. The $m = 10$ column deserves attention on its own: knowing ten measured genes of a held-out strain predicts the remaining 6,159 at $r = 0.4562$ within study and 0.2335 across studies, which is roughly double the trained model's genotype-only score from ten numbers and no training. That is a striking amount of information for a very small measurement.

And $m = 1000$ is not the operating point. Measuring 1,000 transcripts to predict the rest is an expensive capability, and quoting the oracle at $m = 1000$ makes it sound like the requirement. It is not. Filling in the low end densely shows the curve is steep exactly where it is cheap.

And which genes are revealed matters, at the cheap end and across studies. Two non-random selection rules were fit on training residuals only. Choosing the genes with the highest across-strain residual variance beats a

m observed	within-Kemmeren	within-Sameith	Kem $\rightarrow$ Sam	Sam $\rightarrow$ Kem
10	0.4562	0.3649	0.2335	0.2122
100	0.6693	0.6417	0.3815	0.3922
1000	0.7832	0.7779	0.4838	0.4803

Table 7: Masked-conditioning oracle, `pearson_per_feature` on validation strains. The within-study value at $m = 1000$ exceeds the 0.775 reproducibility ceiling, which the cross-study column resolves: same-array conditioning was predicting the target’s own measurement noise. The honest ceiling for the cross-study columns is not 1 either; it is 0.611, the agreement between the two studies’ own measurements of the same 82 deletions.

m revealed	within-study	% of its $m=1000$	Kem $\rightarrow$ Sam	% of its $m=1000$
1	0.1293	16 %	0.0825	17 %
2	0.2747	34 %	0.1377	29 %
5	0.3191	40 %	0.1460	30 %
10	0.4201	53 %	0.2229	46 %
20	0.5135	64 %	0.2684	56 %
50	0.6221	78 %	0.3353	70 %
100	0.6869	86 %	0.3668	77 %
200	0.7360	92 %	0.4124	86 %
500	0.7812	98 %	0.4567	95 %
1000	0.7977	100 %	0.4793	100 %

Table 8: Imputation against measurement budget, genes revealed at random. **One hundred genes reach 77 % of what a thousand genes reach across studies**, and ten genes reach 46 %. The cross-study column is the one to read, since the within-study column is partly predicting the target array’s own noise; against the 0.611 cross-study agreement ceiling, $m = 100$ recovers 60 %. The random arm reproduces both previously published runs on a different split, which is the check that this is the same estimator.

random set on the cross-study arm by +0.046 at $m = 10$ and +0.026 at $m = 100$, which is 6.3 and 3.1 times the random arm’s standard error over five draws. It loses at $m \geq 500$, where the budget is large enough that a random set spans the same structure. A column-pivoted QR leverage rule, the theoretically tidier choice, is *below* random almost everywhere and much worse below $m = 20$.

A variance-chosen set also transfers better: 0.76 of its within-study score survives the study swap at $m = 10$, against 0.49 for a random set. **Hypothesis, untested:** high-variance genes carry more biological amplitude relative to per-array technical amplitude. Nothing measured here tests that mechanism.

Two comparison caveats travel with this. The two chosen rules are deterministic, so each is one set with no across-set spread, and the standard error quoted is the random arm’s alone; this is not a paired test. And **selection by ease of measurement was not run:** no measurement-cost annotation exists in the repo, so any such ranking would have been invented rather than sourced. That is the version worth building, since it is the one that decides whether a fixed cheap panel could stand in for the transcriptome. SRC: `experiments/019-simb-multimodal/scripts/conditioning\_gene\_budget.py`

The gain is not a better genotype-to-expression map, which is what makes it a separate capability rather than a rival result. Replace the per-gene mean with a genuine genotype-only predictor, a k -nearest-neighbor model over `prot_T5_all` embeddings with $k = 25$, and re-run the whole oracle against *its* residuals. If the conditioning gain were mostly the model failing to learn genotype, a good genotype predictor would absorb it and the gain would shrink. It does not shrink: 97.5, 99.2 and 100.6 % of it survives at $m = 10, 100, 1000$.

So the two channels are close to non-overlapping. Knowing which gene was deleted and knowing some of this strain’s other measured genes are almost separate sources of information about the rest, and the strand is scored where only the first is available. SRC: `experiments/019-simb-multimodal/results/conditioning_gain_after_genotype.json`

The kNN probe, and why it belongs beside the oracle. The same question asked on the genotype side alone: predict a held-out deletion’s phenotype as a similarity-weighted average of the phenotypes of the nearest *other* deleted genes in some embedding space, with zero learned parameters. Best arms reach `pearson_per_feature` of 0.117 on expression and 0.070 on morphology with `prot_T5`, against a noise floor of 0.033 and 0.036 set by random embeddings of matched dimension. On morphology the best arm is not a language model at all: `normalized_chrom_pathways`, a graph-derived 197-dimensional feature, reaches 0.100. Every nucleotide-transformer arm sits at or below the floor.

Read against a trained model at 0.196 ± 0.022 on expression (best run 0.238), a parameter-free neighbor average recovering 0.117 says over half the genotype-side signal the model finds is available from embedding proximity alone. **That is a statement about how little the training is adding on this axis**, and it is the second reason, beside the pair-term degeneracy, to suspect the operator rather than the data. SRC: `experiments/019-simb-multimodal/results/knn_`

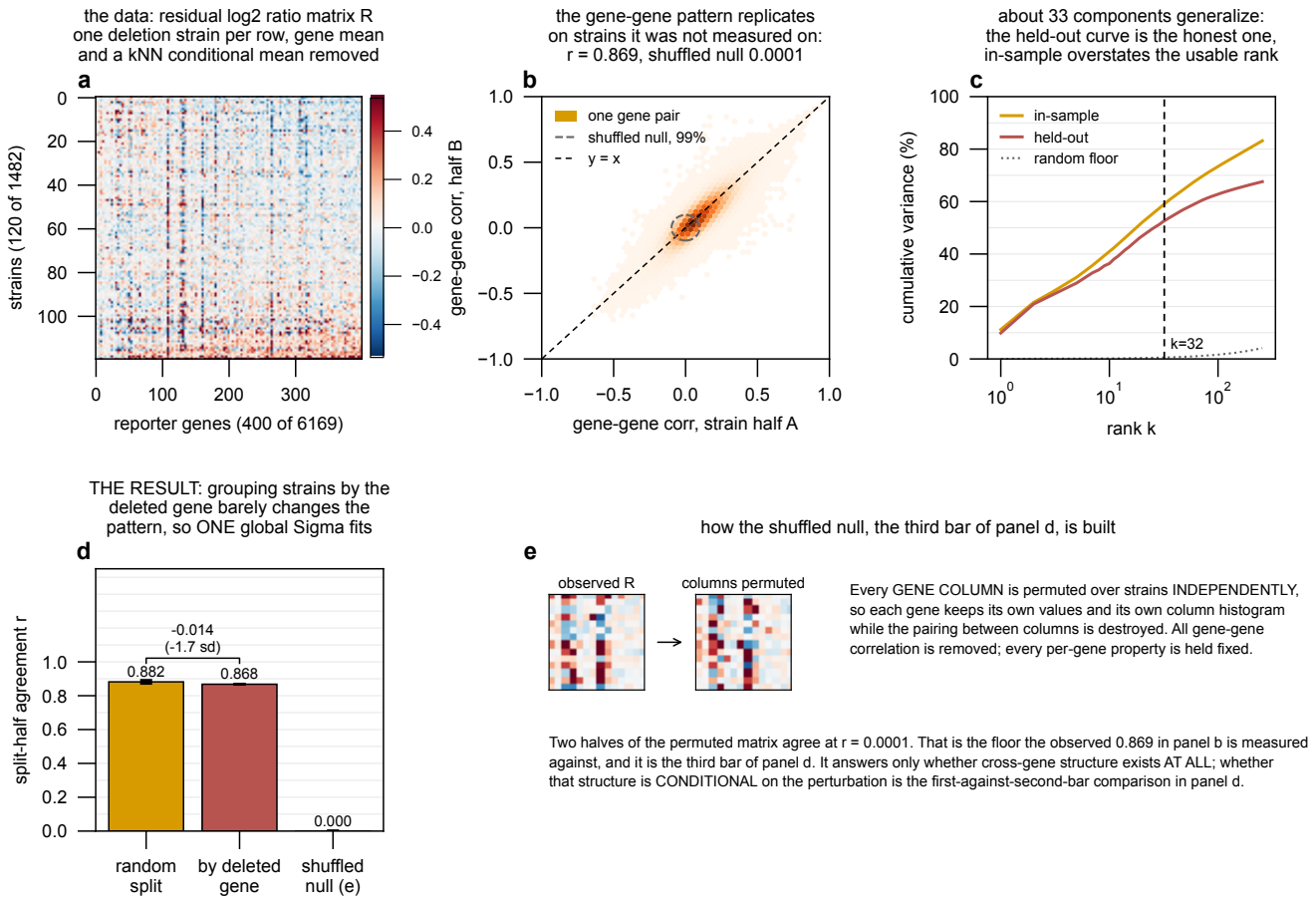


Figure 3: Residual gene-gene structure in the expression compendium, after removing each gene's mean and a kNN conditional mean. (a) the residual matrix itself, a subsample of strains and reporters. (b) the load-bearing check: the gene-gene correlation pattern measured on one half of the strains against the same pattern measured on strains it never saw, agreeing at $r = 0.869$ where the permutation floor is 0.0001. So the structure is real and it generalizes across strains. (c) how much of it survives being fit on one half and scored on the other; the held-out curve is the honest one and it flattens near rank 32, which is why the covariance factor is set there. (d) whether that structure depends on *which* gene was deleted: splitting strains by deleted-gene similarity instead of at random changes the agreement by -0.014 , which is 1.7 standard deviations of the random-split spread, so one global covariance is close to sufficient and a perturbation-conditional one has little left to buy. (e) how the null in (d) is constructed, by permuting each gene column independently so every per-gene property is held fixed and only the cross-gene pairing is destroyed.

What the oracle exploits, and why a per-gene readout cannot. After removing each gene's mean, the residual gene-gene correlation pattern replicates on held-out strains at split-half $r = 0.8687$ against a permutation null of 8.45×10^{-5} , with an effective rank of 32.78 and 59.1 % of variance inside the top 32 components.

That is the whole reason the oracle works. Genes do not move independently: about 33 directions of coordinated variation run through the residuals, so once you have measured a handful of genes in a strain you have located that strain along those directions, and the rest follow. **A per-gene independent readout emits 6,169 separate marginals and discards every bit of it**, which is the structural difference between the model and the oracle and the reason the gap between them is not a matter of training harder.

The conditional test in fig. 3d adds the caveat that this structure is largely *global* rather than perturbation-specific, at 1.7 standard deviations, just inside the threshold the script uses to call it unconditional. So a shared covariance is the right model for it, and making the covariance depend on the deletion is not where the headroom is. SRC: experiments/019-simb-multimodal/results/residual_covariance_diagnostic.json

2.6 Why the byte count is misleading ✗

The expression compendium holds more scalar labels than the trigenic interaction data the model does well on, which is the intuition that predicted this task would be easy. The label count is not the axis that matters.

dataset	records	per record	scalars	perturbation axis
Kuzmin 2018 trigenic interaction	91,111	1	91,111	
Kuzmin 2020 trigenic interaction	301,798	1	301,798	
both, as 010 queries them	392,909	1	392,909	triples; a gene recurs with many partners
Kemmeren 2014 knockout expression	1,484	6,169	9,153,196	1,484 single deletions, each once
Ohya 2005 morphology	4,718	281	1,325,758	4,718 single deletions, each once

Table 9: Scalar labels are not the binding resource, and the trigenic comparison is with **both** Kuzmin papers: `experiments/010-kuzmin-tmi/queries/001_small_build.cql` selects `TmiKuzmin2018Dataset` and `TmiKuzmin2020Dataset`, 392,909 records between them. Expression still carries more scalars, 9.2 million against 0.39 million, but its perturbation axis has 1,484 points and no gene is deleted twice, so nothing in the data isolates the effect of a deletion from the effect of the one array it was measured on. Record counts are raw rows in each dataset’s `preprocess/data.csv`, before the deduplication and aggregation the graph build applies.

In the trigenic data a given gene appears in many records paired with different partners, so the supervision repeatedly constrains that gene’s contribution. In the expression data a gene appears as the perturbation exactly once, and it appears in one array. The 6,169 labels attached to it are 6,169 readouts of a single perturbation event, correlated with each other at the level the residual covariance above measures, so they are worth far less than 6,169 independent constraints on the deletion.

Expression sits at 0.196 ± 0.022 against a 0.775 ceiling, is structurally limited on the pair axis, and is sitting beside a much larger capability it is not being scored on. The quoted 0.238 is the best of eight identical-config draws at 9,900 epochs, $z = 1.89$ above their mean, and five of those eight peaked by epoch 4,109, so neither “the score” nor “never peaks” survives the replicates. The round produced no valid comparison between mechanisms, since all 67 scored arms stopped at or below 300 epochs against a curve that had not turned, and the 0.0222 replicate spread means a single run per arm could not have resolved the gaps in question anyway. Three things do hold. The additive perturbation operator *provably* cannot express a dependence on the pair (deleted gene, reporter gene), and masked unmasking does not supply one where the score is taken. Loss and metric diverge here and on 010, in opposite directions, so no single stopping rule is right for both. And the largest available signal is imputation from other genes of the same strain, which is measured to be nearly orthogonal to the genotype task and is a separate capability rather than a better version of this one.

Cannot say. Whether any of the pair-rank mechanisms help, since only two rungs have been trained and not at matched budgets. Whether a different objective closes the loss-metric gap. Whether the graph prior is the right prior for this task (section 7.7 measures the premise, not the model’s use of it). And where the validation curve turns, which is the number every arm budget depends on.

3 Morphology ✗

3.1 The target ✗

The **morphology strand** predicts the CalMorph phenotype vector of a single-deletion strain. CalMorph’s nominal vocabulary is 501 parameters, which resolve into 281 base parameters and 220 coefficient-of-variation statistics; the served target is the 281 base parameters, of which the model scores 278 after three near-degenerate features are dropped. Source is Ohya 2005 (doi:10.1073/pnas.0509436102), 4,718 deletion mutants plus 122 independent *his3* wild-type replicates.

Raw CalMorph values span about eight orders of magnitude, from 6.7×10^{-5} to 1.49×10^4 in absolute mean, so an unnormalized mean-squared-error loss is dominated by whole-cell size and actin-region size and read ≈ 4.7 million. The training path now applies a per-feature z -score fit on the training split only, which brings the loss to order 1 and makes “predict each feature’s mean” score zero rather than win. Ohya’s own normalization is a per-parameter Box-Cox fit on wild-type data followed by standardization, and it discards 247 of 501 parameters on a Shapiro-Wilk normality test at $p \geq 0.5$. That verdict is a normality test over base and CV parameters together, not a variance floor on the 278 modeled here, so it was not imported.

3.2 The ceiling was never measured until now, and it is high ✗

Ohya measures each of the 4,718 deletion strains once and the *his3* wild type 122 independent times, which is exactly what makes reliability estimable without replicating the mutants. Each strain’s value for feature k is one noisy draw $T_k = S_k + E_k$: the strain’s true value plus one sample’s measurement noise. The wild-type replicates hold genotype fixed, so their spread is E_k alone; the mutant panel holds nothing fixed, so its spread carries S_k and E_k together.

Reliability and ceiling are the same measurement on two scales, which is why both names exist. Reliability is the fraction of observed across-strain *variance* that is real signal,

$$\rho_k = \frac{\text{Var}(S_k)}{\text{Var}(T_k)} = 1 - \frac{\sigma_{\text{WT},k}^2}{\sigma_{\text{mutants},k}^2},$$

estimated with the sample variance over the 122 replicates and over the 4,718 mutants. The **ceiling** is what a *correlation* can reach. A perfect predictor outputs S_k and is still scored against the noisy T_k , so it correlates at

$$\text{corr}(S_k, S_k + E_k) = \frac{\text{Var}(S_k)}{\sqrt{\text{Var}(S_k)}\sqrt{\text{Var}(T_k)}} = \sqrt{\rho_k} = c_k.$$

The square root is the entire difference: correlation is a ratio of standard deviations where reliability is a ratio of variances. Since $\rho_k \leq 1$ the ceiling always exceeds the reliability, so the two must never be quoted interchangeably. Over the 278 modeled features the means are $\rho = 0.444$ and $c = 0.611$, and those are one measurement stated twice. Compare the ceiling to a reported Pearson; compare the reliability to an R^2 .

The **robust CV** used alongside them is a different axis entirely, measuring how far deletions move a feature rather than how well it is measured: $\text{rCV}_k = (Q_{75} - Q_{25})/|\text{med}|$ over the 4,718 mutants, using quartiles rather than a standard deviation because the CalMorph features are bounded ratios and counts with heavy tails.

quantity	value
mutants	4,718
wild-type replicates	122
modeled features	278
mean reliability	0.444
mean ceiling over modeled features	0.611
median ceiling	0.647
features with ceiling > 0.5	201 of 278 (72 %)
best observed (vscej2v, peak epoch 27 of 65)	0.0824
fraction of ceiling realized	13.5 %

Table 10: Morphology has the second-highest ceiling of any strand here and the lowest fraction of it realized. Nearly three quarters of the modeled features are individually measurable to $r > 0.5$.

3.3 The strand was never really run, on two counts ✗

Twenty-three runs exist in `torchcell_019_morph_v5`, spanning 11 to 121 epochs, one of them a collapsed constant predictor. The best peaks at epoch 27 of 65. The joint expression-plus-morphology project is thinner still: 32 runs, none past 87 epochs, best `val/mean/pearson_per_feature` of 0.0505.

Set against section 2, where the same architecture on a related target was still improving at epoch 9,697, a 65-epoch morphology run carries no information about whether morphology is learnable. The morphology result is not a null. It is an absence of measurement, and the two must not be recorded the same way.

And the training set was a quarter of what exists, in every project. This was checked against all eight projects that carry a morphology head rather than against one, because the first version of this claim rested on `torchcell_019_morph_v5` alone and the natural objection is that some other round did train on the full screen. None did.

SRC: `experiments/019-simb-multimodal/results/project_census.json`

SRC: `experiments/019-simb-multimodal/conf/delta_joint_expr_morph_000.yaml`

SRC: `experiments/019-simb-multimodal/results/fig3_overlap_census.json`

group	projects	runs	n_{train} observed
morphology alone (<code>morph_v2, _v3, _v5</code>)	3	183	1,161
morphology with expression (<code>expr_morph .. _v5</code>)	5	214	1,161
total	8	397	1,161, no other value

Table 11: Every morphology run ever launched, across all eight projects that carry a morphology head, trained on the same 1,161 strains. The Ohya screen holds 4,718 deletions and 1,440 of them also carry expression; the configs set `require_modalities: [expression_log2_ratio, calmorph]`, which pins every arm to that intersection, and the 80/20 split takes it to 1,161. For contrast, the same census shows the expression strand’s training set growing across rounds (1,125 to 1,253) and the betaxanthin rounds differing (3,694 to 4,235), so a constant is a finding here rather than an artifact of how the census reads.

The claim does not rest on the table alone. `require_modalities` is set in the config that every morphology arm inherits, and the overlap census independently gives `ohya_with_expression` = 1,440 against `ohya_unique` = 4,718. The W&B column is corroboration of a restriction that is visible in the configuration.

That restriction is correct *for the auxiliary-task question*, since comparing a joint model against a morphology-only model on different instance sets would confound the auxiliary head with a data-quantity difference. It is wrong for the question “how well can morphology be predicted”, and the same configs were used for both. **Morphology has never been trained on its own full dataset.**

3.4 Which second label should morphology be paired with ✗

The overlap census answers this directly, and the answer is not expression.

genotypes carrying	count
morphology (Ohya, total)	4,718
morphology and fitness	4,220
morphology and expression	1,440
morphology, fitness and expression	1,326

Table 12: Fitness shares nearly three times as many strains with morphology as expression does, and covers 89 % of the morphology screen against expression’s 31 %.

Pairing morphology with fitness keeps 4,220 of the 4,718 strains, so the joint arm and the morphology-only arm can be run on a large common instance set without the restriction above costing three quarters of the data. Pairing it with expression caps both arms at 1,440.

The cap is a consequence of `require_modalities`, not a fact about the data. Both caps come from dropping any genotype that is missing either phenotype. Masking the loss over absent outputs instead lets a strain with morphology and no expression still contribute morphology gradient, and a strain with both contribute both. That removes the cap on the *training* side while leaving the controlled comparison available on the shared subset, and it is the documented way around this. No run here did it; `require_modalities` is set in the config every morphology arm inherits.

The question to ask, stated as one sentence. Does adding fitness as a second label improve morphology prediction at all, on any CalMorph feature or on the average across them? That is a yes-or-no question with a correct control (the same architecture, the same strains, morphology head only), and it is worth answering before any more elaborate multi-task claim. Report it per feature as well as averaged, because a second label that helps the 201 well-measured features and does nothing for the rest would be invisible in the mean.

This does not retire the expression-helps-morphology question. That question is what `delta_joint_expr_morph_000.yaml` was built to answer, its control is right, and its three conditions (`expr`, `morph`, `expr_morph`) are the correct decomposition. It has simply never been run past 87 epochs. The intended design is the broader one: train on all of morphology with masked outputs, then ask whether adding expression supervision on the 1,440 strains that carry it improves anything. That is a long and awkward run to tune, which is the practical reason expression is being settled first.

What the earlier “0.05 to 0.14 rise” was. A flatten-Pearson computed over the $[B, 281]$ matrix before per-feature normalization, which is dominated by between-feature scale: large features are large in both prediction and target. After z -scoring, the same smoke run reports ≈ 0 on that metric. The honest metric is the per-feature correlation across strains, averaged over features, which is what table 10 scores.

3.5 A scalar target is a reasonable next step, and this is which scalar $\times$

Ranking single features on reliability alone selects ones that are precisely measured and nearly constant across mutants, which is the wrong end of the trade. The rank key is $s_k = c_k \cdot \text{rCV}_k$, the ceiling multiplied by the robust coefficient of variation, so a feature has to be both measurable and actually moved by deletions.

feature	CalMorph label	SI description	ceiling	robust CV	score
A113	actin_n_ratio	No actin patch found ratio	0.873	2.369	2.068
C124_C	Medium_bud_ratio	Ratio of medium bud on stage C	0.698	0.886	0.618
A110_A1B	Actin_f_ratio	Actin f ratio on stage A1B	0.637	0.939	0.598
C125_A1B	Large_bud_ratio	Ratio of large bud on stage A1B	0.784	0.630	0.494
A105	actin_a_ratio	Actin a ratio	0.655	0.617	0.404
D212	nuclear_B_ratio_to_nuclear_AA1BC_cells	Ratio of B (Nuclear) to A, A1, B and C cells	0.604	0.642	0.387
D201	nuclear_B_ratio	Ratio of B (Nuclear)	0.565	0.641	0.362
A109_A1B	Actin_e_ratio	Actin e ratio on stage A1B	0.476	0.756	0.360
A103_A1B	Relative_distance_of_actin_patch_center_from_neck_in_mother	Relative Distance of actin patch center from neck in mother cell on stage A1B	0.812	0.431	0.350
A103_C	Relative_distance_of_actin_patch_center_from_neck_in_mother	Relative Distance of actin patch center from neck in mother cell on stage C	0.534	0.639	0.342
A110	actin_f_ratio	Actin f ratio	0.700	0.468	0.328
D215	nuclear_B_ratio_to_nuclear_AA1BC_cells	Ratio of B (Nuclear) to A1, B and C cells	0.480	0.578	0.277
A8-2_C	Total_brightness_of_actin_region_in_bud	Actin region brightness in bud on stage C	0.594	0.449	0.267
A8-2_A1B	Total_brightness_of_actin_region_in_bud	Actin region brightness in bud on stage A1B	0.595	0.436	0.259
A121_A	Maximal_distance_between_patches	Maximam actin patch length on stage A	0.492	0.517	0.254
A8-1_A	Actin_region_brightness	Actin region brightness in mother cell on stage A	0.562	0.444	0.250
A120_A	Total_length_of_actin_patch_link	Total length of actin patch link on stage A	0.404	0.613	0.248
D208	nuclear_B_ratio_to_budded_cells	Ratio of B (Nuclear) to budded cells	0.424	0.575	0.244
A108	actin_d_iso_ratio	Actin d (iso) ratio	0.641	0.375	0.241
C122	large_bud_ratio	Ratio of large bud	0.693	0.342	0.237

Table 13: The twenty best scalar morphology targets, ranked by ceiling multiplied by robust CV. The list is dominated by bounded class ratios, actin localization class, bud-size class and nuclear-stage class, which is what a deletion actually moves. Cell size is the opposite trade and cell shape more so: cell size at the unbudded stage **C11-1_A** has a ceiling of 0.928 but a robust CV of 0.097, ranking 101st, and mother-cell roundness **C115_A** has a ceiling of 0.972 with a robust CV of 0.024, ranking 245th of the 275 features with a defined rank key. That last id was described as a whole-cell axis ratio in the previous revision; SI Table 2 calls it roundness of the mother cell, and it is a shape parameter rather than a size one. Descriptions are quoted verbatim from Ohya 2005 SI Table 2, source typography included, and the label column is the same parameter’s name in SI Table 1; the two tables word some parameters differently, and both wordings are the paper’s. In a CalMorph id the leading letter names the stain the parameter is measured from, C the FITC-ConA cell-wall image, A the rhodamine phalloidin actin image and D the DAPI nuclear image, and the suffix names the nuclear stage the cells were binned into, **_A**, **_A1B** or **_C**, with no suffix meaning all stages pooled. The single-letter classes inside a description, actin *a* to *f* and nuclear *A* to *F*, are defined by the drawings in SI Fig. 5B rather than by any text, so those rows rest on that figure.

On “predict cell size first”, which the data does not support. The idea is attractive because cell size is the CalMorph quantity closest to something intuitive, and it is the one place the panel gives a clean answer. Cell size is measured superbly and moved almost not at all.

And on “predict colony size first”. Colony size is fitness, and fitness is already the strongest strand in the project, trained on the Costanzo and Kuzmin single-mutant fitness data. Using it as the morphology warm-up would test the pipeline rather than the phenotype.

Recommendation: run the scalar warm-up on A113 or C124_C, both reliable and genuinely variable, and treat the run as a convergence-budget calibration for the 278-feature target rather than as a result. This is queued behind the expression work (section 8) rather than proposed for now.

feature	CalMorph description	ceiling	robust CV	rank of 275
C11-2_A1B	Area of daughter cell on stage A1B	0.880	0.125	78
C11-1_A	Mother cell size on stage A	0.928	0.097	101
C101_A1B	Cell size on stage A1B	0.928	0.088	118
C103_A	Long axis length of mother on stage A	0.949	0.047	209
C115_A	Roundness of mother cell on stage A	0.972	0.024	245
A113	(the shortlist leader, for contrast)	0.873	2.369	1

Table 14: Cell-size features are the opposite trade from the shortlist: near-perfect ceilings of 0.88 to 0.97, and among the least variable features in the panel. Whole-cell area at stage A has a robust CV of 0.097 against a population median of 0.125, so **173 of the 278 features have more spread than whole-cell area**, and its rank key is 23 times smaller than A113’s. *Correction to revision 2*, which called C115_A a whole-cell axis ratio: the CalMorph SI defines it as roundness of the mother cell, a shape parameter rather than a size one. The one size feature worth noting is bud area (C11-2_A1B), which carries 28 % more spread than mother area, plausibly because a deletion that stalls the cell cycle changes bud size at fixed nuclear stage.

Morphology is the largest unexploited target here, and it has never been measured properly. Its ceiling, estimated for the first time, is 0.611 averaged over 278 features, with 201 of them individually above 0.5, which is the second-highest ceiling of any strand. The best score achieved is 0.0824, at 13.5 % of it. But that score comes from a 1,161-strain training set drawn from a 4,718-strain screen, in a run that stopped at epoch 65, and **every one of the 397 morphology runs across eight projects trained on the same 1,161 strains** because `require_modalities` pins them to the intersection with expression. **So 0.0824 is not a null result about morphology; it is the absence of a measurement**, and the two must not be recorded the same way. For a second label, fitness shares 4,220 strains with morphology against expression’s 1,440.

Cannot say. Anything about whether morphology is learnable from genotype, because no morphology run has passed 121 epochs or seen more than a quarter of the data. Anything about whether expression and morphology share structure, because the joint project’s longest run is 87 epochs.

4 Betaxanthin production ✗

4.1 The task, its ceiling, and the two names that get confused ✗

Cachera is a genome-wide betaxanthin deletion screen (doi:10.1093/nar/gkad656): the four-gene betalain cassette *CYP76AD1*, *DOD*, *ARO4*^{K229L} and *ARO7*^{G141S} was transferred by CRI-SPA into each strain of the yeast knockout collection and per-colony fluorescence read as a production proxy. It supplies 4,735 single deletions and is the ground truth here. **Flux Cone Learning** (FCL) is Merzbacher et al.’s method, which learns from flux samples drawn through a genome-scale metabolic model; `RandomForestClassifier_Resampled` is its best variant. The comparison is CGT against FCL, both scored on the Cachera screen.

Betaxanthin is the only phenotype in this document with a high ceiling. Its records carry a per-strain standard error over a median of 15 colonies, giving a reliability of 0.836 and a Pearson ceiling of **0.914**. SRC: `experiments/019-simb-multimodal/results/pigment_noise_ceiling.json`

4.2 Three betaxanthin numbers, two scoring rules, and one honest comparison ✗

number	where it comes from	scoring rule	n_{train}	split
0.4301	Optuna study <code>betaxanthin_002</code>	raw per-epoch max	4,235	ordinary ratio split
0.4015	the same runs, in W&B	<code>roll_max</code>	4,235	ordinary ratio split
0.3705	<code>torchcell_020_betaxanthin_v4</code>	<code>roll_max</code>	3,698	Merzbacher test genes pinned out

Table 15: The first two rows are the **same runs under two scoring rules**, which is what made the previous revision confusing. The Optuna objective is `val/betaxanthin/pearson_per_feature_max`, a running maximum over epochs; the leaderboard’s `roll_max` first averages five neighboring epochs, and smoothing can only lower a maximum. Only rows two and three are comparable, and their difference, 0.4015 against 0.3705, is what holding out a competitor’s test set costs. Against the 0.914 ceiling the best is 47 % realized on the raw rule and 44 % on the smoothed one.

On whether 0.4301 is a plateau or one lucky trial, since the previous phrasing of this was unreadable. A **trial** here is one Optuna trial, meaning one hyperparameter configuration trained once. It is not a data split: the

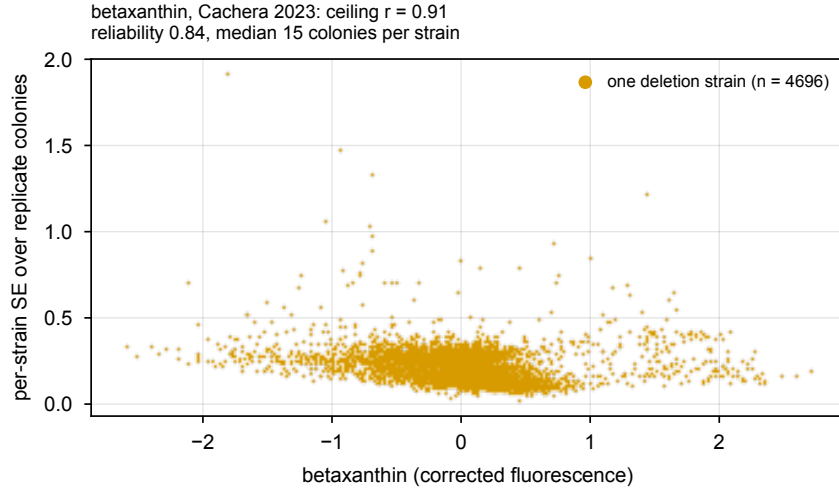


Figure 4: Reliability of the betaxanthin target. Unlike every other phenotype here, Cachera releases a per-strain standard error over a median of 15 colonies, so the noise term is measured directly rather than inferred from replicate structure. That is what puts this ceiling at 0.914 and makes betaxanthin the one strand where a regression target is clearly supportable.

split is pinned across the whole study, so trials differ by hyperparameters and by weight initialization only. Study `betaxanthin_002` ran 37 of them. Two things follow, and they point in opposite directions.

- **The top is flat, so the winner is not a spike.** The best five trials read 0.4301, 0.4234, 0.4216, 0.4211 and 0.4095, a spread of 0.0206. Re-running an *identical* configuration moves the objective by $\sigma = 0.0302$ (table 28), so those five sit inside two thirds of one standard deviation of each other. They are statistically indistinguishable, which means the winner is one of five equivalent configurations rather than a single fortunate run.
- **But taking the maximum of 37 noisy trials is still biased upward.** That is **selection inflation**, estimated at 0.064 here. So a configuration chosen in advance, without seeing the study, should be expected to score around 0.366 rather than 0.430.

Both are true at once: the plateau says the number is not a fluke of one run, and the inflation says it is not what a fresh configuration would give.

The comparison number is lower than the best number on purpose. The head-to-head below runs on `020_v4`, whose split **pins the 640 Merzbacher test genes out of training**. That costs 537 training strains, and it costs score. Using the higher-scoring model instead would mean scoring on genes it had trained on, which is the one thing that would void the comparison. The cell carried forward is `s09_L6_maskon_lr0.0001_yj`, selected by validation at 0.3639 and never by its score on the comparison genes.

The 537 strains are not recoverable, and the fix is a reporting change that needs no retraining. The natural proposal is to train on everything outside the comparison list, carve validation and test from that same remainder, and evaluate twice. Sized against the data, that proposal is numerically the split `020_v4` already ran: it gives 3,682/299/295 plus the 639 comparison genes, against `020_v4`’s actual 3,698/286/931, where the 931 is 639 comparison genes plus 292 of our own. Holding the comparison genes out of training is what costs the 537, under any scheme, and no rearrangement returns them.

arm	train	validation	test
unpinned (<code>torchcell_020_betaxanthin_v3</code>)	4,235	340	340
pinned (<code>torchcell_020_betaxanthin_v4</code>)	3,698	286	931
the proposed re-split, sized	3,682	299	295 + 639

Table 16: Sizing the proposed re-split against what was run. The bookkeeping on the pinned arm closes exactly: train -537 , validation -54 , test $+591 = 537 + 54$, the other 48 pinned genes having already been in the unpinned test set. The realized split ratio is 86/7/7 rather than the nominal 80/10/10, because assignment is per-key stratified; the config comment estimating “roughly 511” lost genes used the nominal ratio and understates the true 537 by 26.

What the proposal does buy, and it is worth taking, is the second evaluation. The existing 931-record test set is already 639 comparison genes plus 292 of our own. Splitting it and reporting both numbers separates “how we do against FCL on their genes” from “how the model does on a clean held-out sample”, and it runs off the existing `020_v4`

checkpoints with no retraining at all. Enlarging our own test set beyond those 292 is the only part that costs anything, at roughly a further 137 training strains for a 429-gene own test set.

One name-resolution defect, now triangulated and resolved. Of the 640 comparison genes, 630 carry a betaxanthin measurement, and the pin file holds 639. The screen reports one gene as *PPA1*, an ambiguous alias naming both *IPP1* (*YBR011C*), which is essential, and *VMA16* (*YHR026W*), which is not. Neither owns *PPA1* as its standard name, so the resolver’s standard-name-before-alias rule cannot break the tie and correctly returns *AMBIGUOUS*. The raw screen does not settle it either: its 27 columns carry a gene name and colony statistics, with no systematic ORF, plate, position or barcode.

Five independent lines were run, and they agree. SRC: `experiments/020-cachera-betaxanthin/scripts/triangulate\_ppa1\_alias.py`

- **The strain is alive, which is decisive on its own.** The *PPA1* row records **15 colonies at 24 h and 15 at 48 h**, with mean areas of 106.5 and 118.5. A haploid *ipp1*Δ is inviable and yields no colonies, so no row for it can exist. This is the strain’s own growth data and does not depend on any claim about which collection was screened. The colony is small, in the 1.6th percentile of screen area, which is what a severe but survivable defect looks like.
- **The screen contains essentially no essential genes**, on 1,140 counterexamples rather than the six previously cited. It covers 4,704 of 5,467 non-essential genes (86 %) and 29 of 1,140 SGD inviable-null genes (2.5 %), and those 29 are autophagy genes and paralogs whose inviability is conditional, all of which grew.
- **The V-ATPase family has a *VMA16*-shaped hole.** Thirteen of the fifteen *VMA* genes are in the screen; the only absentees are *VMA9* and *VMA16*. The file names genes by alias constantly (*VMA1* as *TFP1*, *VMA3* as *CUP5*), so an alias row for *VMA16* is exactly what one expects.
- **Its value sits on top of the V-ATPase cluster.** Betaxanthin −1.566 against *VMA13* −2.19, *VMA3* −1.55, *VMA22* −1.54, *VMA6* −1.44, on a screen median of −0.006. Soft corroboration, carrying nothing alone.
- **Neighbor evidence is neutral here**, unlike the *FEN1* case: neither candidate appears elsewhere in the screen under another name, so the “one row per gene” argument that decides *FEN1* does not apply.

The *PPA1* row is *YHR026W* (*VMA16*), and the split builder is right. Nothing points to *IPP1* except the build itself.

The training build has it wrong, and the head-to-head does not. The defect is real but narrower than feared. In both the loader *LMDB* and the *fig6_pigment_transfer* training build, *YBR011C* carries the betaxanthin value −1.5664 and *YHR026W* carries none. The cause is verified in code rather than inferred: the build predates the layered resolver, and the loader then took `candidates[0]` from the alias map, which returns *YBR011C* for *PPA1*. The same line got *FEN1* right by luck.

The comparison is not corrupted, because `build_merzbacher_split.py` applies the screen-local disambiguation when it reads the raw file, so the comparison’s ground truth for *YHR026W* is the *PPA1* value and *YBR011C* is the gene dropped from 640 to 639. What remains is one false training label out of 4,735: the model was trained on a betaxanthin value attributed to a gene whose deletion strain cannot exist, and scored on the same measurement under its correct ORF.

A plain rebuild does not fix this, it loses the value. The current loader rejects *AMBIGUOUS*, so it would drop the row and leave neither ORF labeled. The disambiguation table lives in the split builder and needs to reach the loader, which is issue #195 territory and lands with the next full graph build.

And the same alias broke Merzbacher’s pipeline too. Their released 640-gene test list contains *both YBR011C* and *YHR026W*, each labeled low, so their code expanded the alias to both candidates rather than disambiguating. It is an independent instance of the same failure, and it is worth knowing that 5 of their 640 test genes are SGD inviable-null in what is otherwise a non-essential list.

4.3 The published accuracy comparison is empty for both methods ✗

FCL reports gene-level accuracy 0.700 against a majority-class rate of 0.673, achieved by calling 607 of 640 genes medium, which is 94.8 %. Our absolute binning sits at 0.681 doing the same thing. Forced to the true class marginal, FCL drops to 0.556 and CGT to 0.549.

The structural point. Every model that finds more high producers has worse overall accuracy, because the only way to call more highs is to stop calling everything medium. Accuracy selects against the capability the task is about, so it should not be the primary metric in any writeup, including ours.

4.4 Both methods have real signal, and it lives in the tail ✗

Of the top k genes each method nominates, the fraction that are truly high producers, against a base rate of 0.156:

model	$k = 10$	$k = 25$	$k = 50$
FCL RF	8 ($5.1\times$, $p = 10^{-5}$)	19 ($4.9\times$, $p = 8 \times 10^{-12}$)	20 ($2.6\times$, $p = 10^{-5}$)
CGT	9 ($5.8\times$, $p = 4 \times 10^{-7}$)	18 ($4.6\times$, $p = 10^{-10}$)	24 ($3.1\times$, $p = 2 \times 10^{-8}$)

Table 17: Precision at k for high producers. Nine of CGT’s top ten are genuine high producers. Both curves approach the base rate by $k = 150$.

Median percentile rank by true class is flat for both methods, at 54/49/56 for FCL and 48/51/52 for CGT across low, medium and high. Read with the table above, that says the signal is real and confined to the tail, not that there is none: neither method separates a typical high producer from a typical low one, and both are far better than chance at the very top of their own ranking.

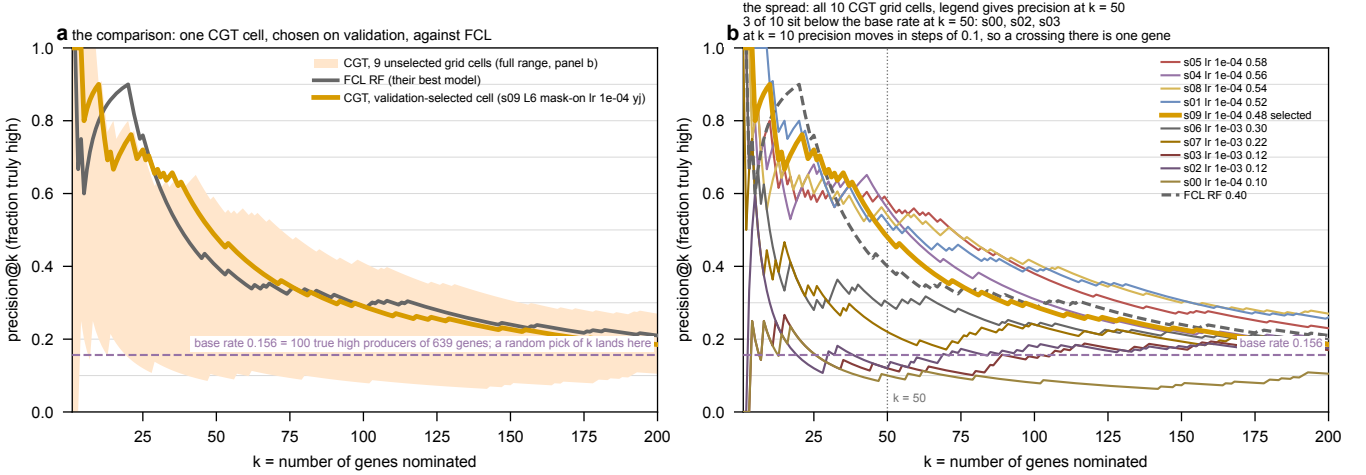


Figure 5: Precision at k for high producers. (a) the comparison: FCL against the CGT cell chosen by validation, with the nine unselected grid cells drawn as a shaded envelope rather than as nine competing lines. The base rate of 0.156 is drawn and labeled, and it is what a random pick of k genes would return. Nine of CGT’s top ten are genuine high producers, and both methods reach the base rate by $k \approx 150$, which is where the tail signal is exhausted. (b) the same ten cells drawn individually, one color each, which is what answers whether some cells sit below the base rate early: **three of the ten do at $k = 50$** . Read crossings at very small k with care, since at $k = 10$ precision moves in steps of 0.1 and a crossing is a single gene. The spread across cells is far larger than the gap between the two methods, which is the hyperparameter sensitivity of section 1.1 rather than a split effect.

4.5 The two methods find different genes ✗

Rank-matched into the same 100 high slots, FCL recovers 29 of the 100 true high producers and CGT recovers 29, and they share 9. The union reaches 49, so running both finds 69% more high producers than running either.

The two methods’ predictions are anti-correlated, and here is what that sentence means. Compare their predicted rankings over the 639 shared genes directly, ignoring the truth. Two methods that are each independently right about the same target should agree somewhat, and the amount is calculable: if one correlates with the truth at $+0.105$ and the other at $+0.039$, then agreeing only through the truth would put their mutual correlation at about $0.105 \times 0.039 = +0.004$, essentially zero. The observed correlation between them is -0.128 , which at $n = 639$ is $z = -3.2$: they disagree substantially more than two independent methods would.

No mechanism is claimed for the negative sign. It could be a genuine anti-relationship between what the two feature spaces encode, or an artifact of how each was fit; nothing here distinguishes those. What *is* claimed is the operational consequence, which does not depend on the sign’s explanation: because the two find largely different genes, their recall adds rather than overlaps, so running both is worth more than picking the better one. That fact is invisible in every aggregate statistic, all of which report the two as tied. The 51 high producers that neither finds is the honest ceiling on this comparison.

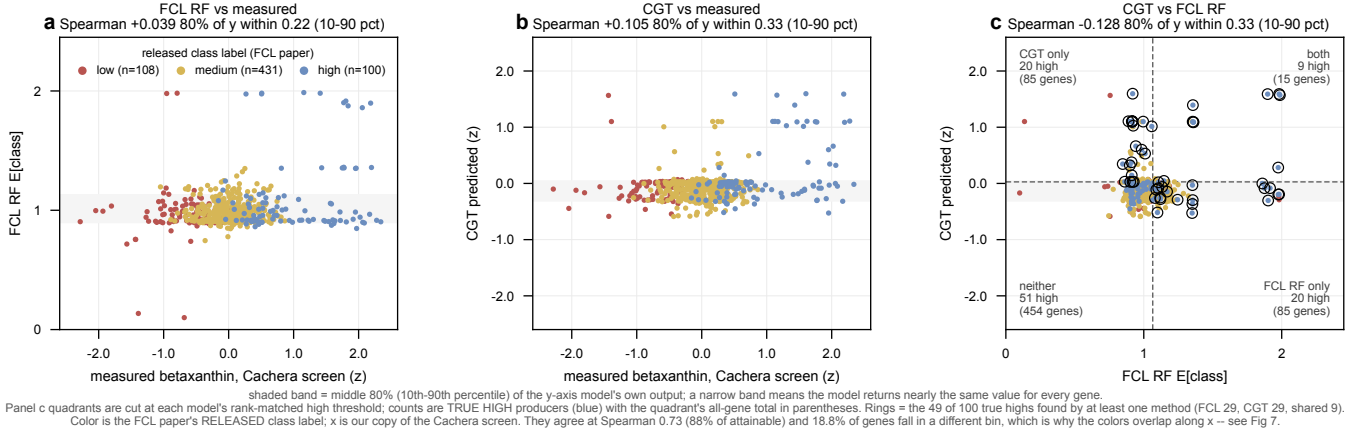


Figure 6: Per-gene spread over the 639 shared genes. Both models emit a nearly constant value: 80 % of FCL’s genes fall inside an expected-class band of 0.22 on a 0 to 2 scale, 80 % of CGT’s inside 0.33 z -units. Panel c shows the disagreement concretely, with each method recovering 29 of the 100 true high producers and sharing 9.

4.6 The real difference is coverage $\times$

yeast-GEM contains 1,161 genes, of which 915 appear in the Cachera screen. That is 19.4 % of its 4,721 deletions. A flux-based method has features for those and for nothing else.

population	in yeast-GEM	outside
every deletion in the screen	915 (19 %)	3,806 (81 %)
top 100 measured producers	32 (32 %)	68 (68 %)
all high producers	60 (27 %)	163 (73 %)

Table 18: Metabolic genes are enriched among high producers, at 26.9 % of the highs against 19.4 % of the screen, a 1.4 $\times$ enrichment that is exactly what the biology predicts. It is not enough to matter: restricting to metabolism buys that concentration and costs three quarters of the hits.

How the class labels are derived, since every metric below depends on it. The Cachera screen releases a continuous production value, and FCL’s task is a three-class one, so a rule is needed to turn measurements into low, medium and high. FCL’s rule: min-max scale production onto $[0, 1]$ over their gene set and cut at 0.40 and 0.65. **We apply their thresholds rather than inventing our own**, which is what makes the two panels comparable. Two consequences follow and both are stated rather than hidden.

- **Applying their rule to our larger copy of the same screen does not reproduce their class counts.** It gives 107/476/56 against their 109/431/100, and 18.8 % of genes land in a different bin than our measured values would give. A model predicting our measurement *perfectly* is therefore marked wrong on about a fifth of genes, so their labels are a headroom bound on any accuracy-style metric, ours included.
- **They never labeled the non-GEM genes at all**, because those genes are outside their method’s reach. We derive labels there with the identical rule, which is the only way that panel exists. The comparison to make is therefore panel against panel rather than number against number, and fig. 8 is drawn that way.

Why every metric here is rank-based or re-binned. We produce a regression and they produce a three-class classifier, so nothing can be compared directly. Two repairs are used throughout, and the choice between them is not arbitrary. **Rank-based metrics** (Spearman, precision at k , AUC) need no thresholds at all and so cannot be affected by the binning disagreement above; they are preferred wherever available. Where a class metric is unavoidable, our continuous predictions are binned **with thresholds fit on the training split only**, never on the test genes, so the binning cannot borrow information from the evaluation. Neither repair fixes the 18.8 % label disagreement, which sits in the ground truth rather than in either model.

On held-out test genes split by yeast-GEM membership, with labels derived the same way on both sides, CGT’s top 25 recovers 17 of 48 true highs inside the model (9.3 $\times$ over base rate, $p = 2 \times 10^{-15}$) and 9 of 21 outside it (4.4 $\times$, $p = 2 \times 10^{-5}$).

FCL’s prediction for the outside set is not poor, it is undefined: flux sampling yields no feature for a deletion the metabolic model does not contain. The FCL test set is 639 of 640 inside yeast-GEM; of our other test genes only 21 of 294 are.

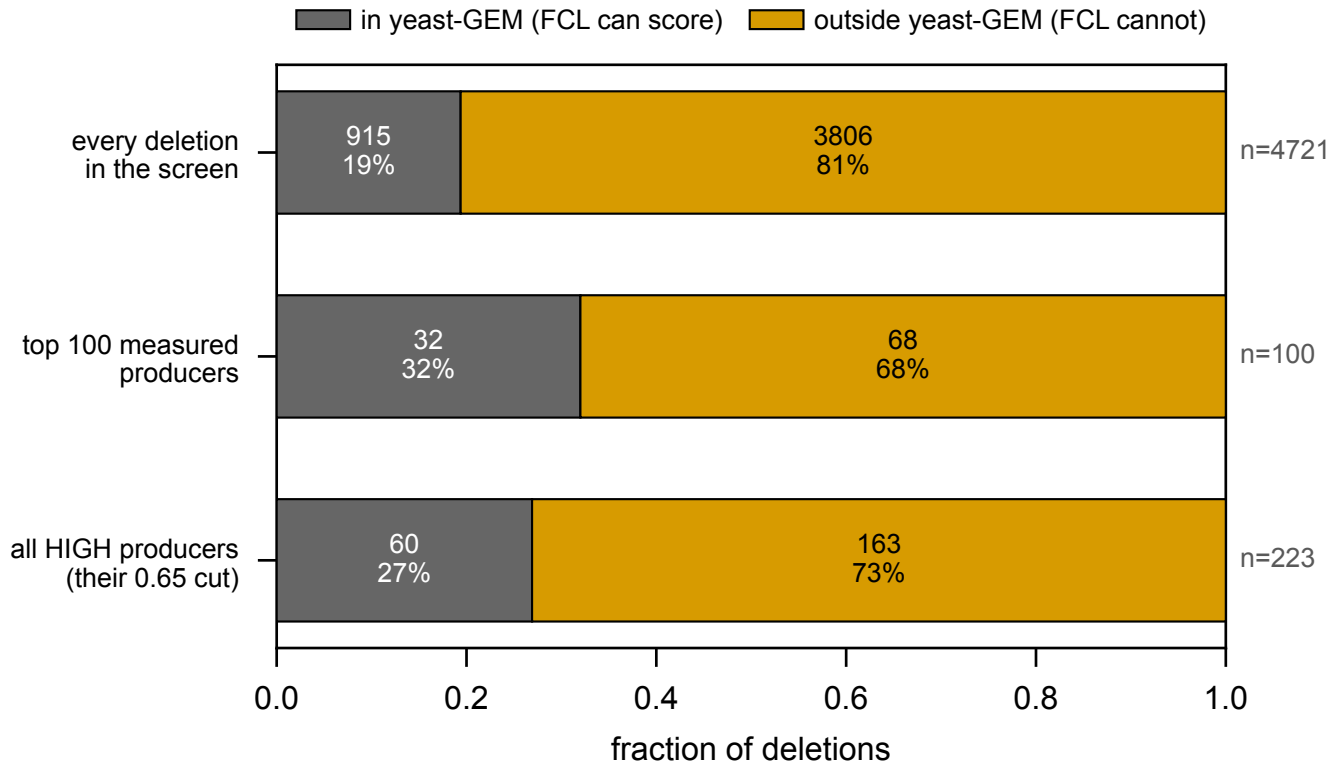


Figure 7: Measured data only, no model and no prediction: how little of the betaxanthin screen a flux-based method can reach.

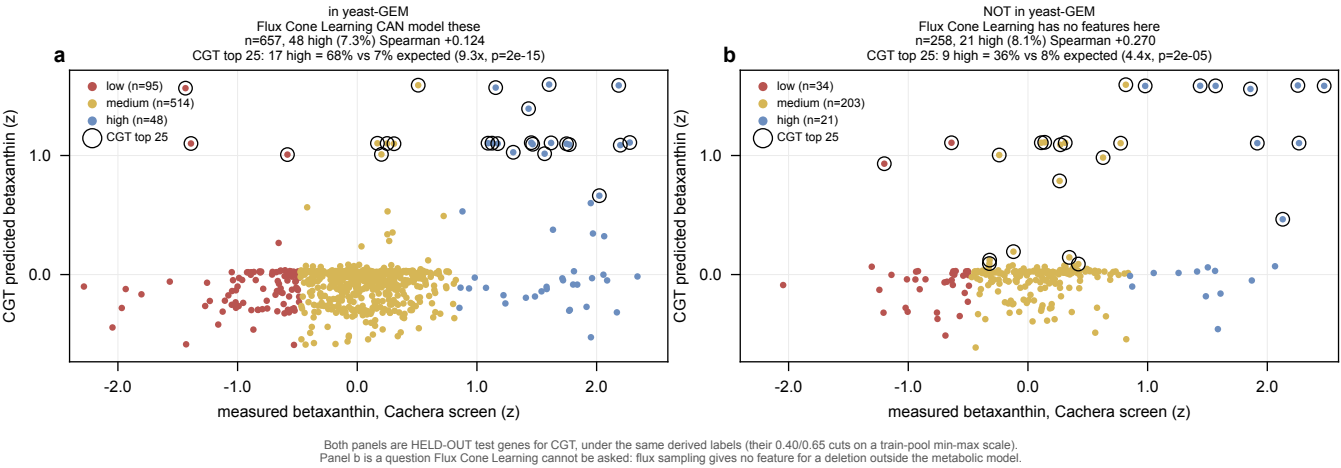


Figure 8: CGT predicts high betaxanthin producers among deletions a flux-based method cannot represent. Held-out test genes split by yeast-GEM membership, identical derived labels on both sides, circled points are CGT’s top 25. Flux Cone Learning has no prediction for the right panel.

One qualification that must travel with this claim. “Inside” and “outside” here mean inside or outside **yeast-GEM**, the genome-scale metabolic model, not inside or outside anyone’s training set. CGT’s rank correlation with measured production is *higher* on the genes yeast-GEM does not contain, +0.270 against +0.124. That is the opposite of what one would expect if metabolic genes were the easy ones, so it is worth testing rather than asserting: the two correlations are measured on different, non-overlapping gene sets, and comparing two correlations requires first mapping each through Fisher’s transform, which turns a correlation into a quantity whose sampling distribution is approximately normal with a known variance. Doing that gives $z = 2.07$, $p = 0.039$: a real difference, but marginal, and on populations that differ in more than their yeast-GEM membership. **The safe reading is that CGT is at least as good outside the metabolic model as inside it**, which is all the coverage argument needs.

4.7 Where the comparison is soft ~~x~~

- **The selection is asymmetric, and it favors FCL. This is now verified rather than suspected, and it is stronger than the previous wording.** Revision 2 said their validation folds “appear to be drawn from the same 640 genes as their test list”. From their Zenodo release: `yeast_production_validation_split.csv` assigns a fold to each of **exactly the same 640 systematic ORFs** that `yeast_production_test_split.csv` names. The intersection is 640 and both set differences are empty, in five folds of 129/128/128/128/127. Their training loop then computes `train_names=set(nontest_names)-set(val_names)`, which is a no-op because those genes were never in the non-test pool, so every fold trained on the full complement and validated on a 128-gene slice of the test set. Model selection across their 16 variants therefore ran on test data.

Their own CHO essentiality release does *not* do this: there the validation genes are exactly the 1,833 flagged as non-test, sharing nothing with the 457 test genes. So the yeast leak is a deviation from their practice inside the same release rather than a naming convention, which is what makes it worth stating as a fact. And the paper alone cannot settle it: its Methods never mention a validation set and the test count is internally inconsistent (659 in the main text, 649 in Table S6, 640 in the release), so only the released files decide it.

Ours, by contrast, is the validation-selected cell with the comparison genes pinned out of training. **So where the two land level, we are level with an opponent selected on the test set.**

- **Validation Pearson does not track test AUC, and this is the hyperparameter sensitivity again rather than a split effect.** Four of ten CGT cells beat FCL’s AUC of 0.570 and the best reaches 0.695, while the validation-selected cell reads 0.557. The runner-up on validation, 0.3528 against 0.3639, has an AUC 0.14 higher. Every one of those cells sees the same split; what differs is the hyperparameters (section 1.1). So the honest statement is that **our own model-selection criterion does not rank cells the way the comparison metric does**, which makes neither “CGT’s AUC is 0.695” nor “CGT’s AUC is 0.557” quotable on its own. Selecting on validation and reporting whatever AUC follows is the defensible protocol, and it is what was done; the spread is reported beside it rather than being selected from.
- **Their labels are a headroom bound**, as detailed above: 18.8% of genes bin differently under their rule than our measured values would give, so a model predicting our measurement perfectly is marked wrong on about a fifth of genes.
- **Regression against classification.** They tried regression and abandoned it, in their words “challenging with the limited number of knockouts at the high and low ends.” Our ceiling of 0.914 makes regression available to us. The comparison is therefore between a regressor and a three-class classifier, which is why every metric above is either rank-based or computed after binning our predictions with thresholds fit on the training split only.
- **The build is stale** with respect to the gene-name resolver (issue #195), which the split builder avoids by working from the raw screen, and which the training data still inherits until a rebuild.

The betaxanthin case is about coverage, and it does not need to beat anyone on accuracy. Where both methods can be scored, CGT and Flux Cone Learning are tied, both with tail-confined signal at roughly $5\times$ enrichment in the top 25, and they find largely different genes, so their recall adds: 29 each of 100 true high producers, sharing 9, reaching 49 together. The asymmetry favors them, since their model was selected on its own test set and ours on validation. **The argument that carries is the other one:** a flux-based method has features for 19% of the screen and misses 73% of its high producers, while CGT predicts high producers in the 81% it cannot represent at $4.4\times$ over base rate, and scores at least as well there as inside the metabolic model.

Cannot say. That CGT beats FCL on any single aggregate. That the rank anti-correlation means anything mechanistically. That the absolute top-100 membership is a deployable call: the rank-matched comparison works by handing *both* methods the true number of high producers and asking which genes each puts in those 100 slots. That is the right way to compare two methods, because neither can win by predicting more or fewer highs than exist. But it is

not available in deployment, where nobody knows how many high producers a screen contains, so the top-100 precision measured here is an upper bound on what the same model would deliver prospectively.

5 Amino-acid pools, and whether they help production ✗

5.1 The target ✗

Mulleder 2016 (doi:10.1016/j.cell.2016.09.007) measured the intracellular concentration of 19 amino acids in 4,678 single-deletion strains. There is one replicate per strain and no released standard error, so the dataset admits **no within-dataset reproducibility ceiling**. Every correlation involving it is attenuated by an unmeasured amount, and that qualification travels with every number below.

The best validation `pearson_per_feature` is **0.209**, run 24vlgope in `torchcell_022_mulleder19_v4`, peaking at epoch 402 of 999. The median across all 97 runs in the strand is 0.0570, so the leaderboard is a long tail under one good configuration rather than a plateau. Earlier rounds read ≤ 0.025 , then 0.171, so the trajectory is upward across rounds.

That median is corrected, and the correction matters for every median in this document. Revision 2 reported 0.0498 “across the 31 runs”. Those 31 were not a sample of the strand: they were the subset whose validation curves had been fetched, and the fetch rule selects the top runs per project by final score and by epoch count (section 7.1). A maximum-finder is the right instrument for a strand maximum and the wrong one for a median, since the runs it skips are systematically the weak ones. Reading all 97 moves the median up to 0.0570. Maxima are unaffected by construction.

5.2 These runs turn within their budget, and expression does not ✗

Twenty-four runs in the preceding round were scored for their convergence shape. The typical run peaks at epoch 48 to 134, continues for another 41 epochs with validation slope turning negative while training slope stays positive, and is classified converged then overfitting. None of them hit the epoch cap. SRC: `experiments/022-mulleder-metabolome/results/training_length_analysis.json`

Said carefully, because “converged” has already misled us once on this project. What is observed is that the validation metric reaches a maximum inside the budget and then declines, which is what makes these runs scorable. That is weaker than convergence, and the distinction is not pedantic: the expression runs were also called converged, on curves that dipped early and flattened, and only at several thousand epochs did it become clear that Pearson was still climbing and squared error would come back down to a late minimum. The same could in principle be true here beyond epoch 999, and no amino-acid run has been extended to find out.

What can be said is the comparison, which does not depend on the word. Amino-acid runs reach a maximum well inside a 999-epoch budget; expression runs have not reached one at 10,000. The two use the same architecture and the same encoder, so whatever makes expression climb for thousands of epochs is a property of the expression target or its loss rather than of the model.

5.3 The precursor hypothesis is wrong, and the profile hypothesis is right ✗

Betalains derive from L-tyrosine (tyrosine to L-DOPA to betalamic acid), and a betaxanthin is betalamic acid condensed with an amino acid. The mechanistic prediction is that a deletion’s free amino-acid pool carries information about how much betaxanthin that same deletion makes. Both screens are keyed by systematic ORF and share 4,432 deletions, so the prediction is directly testable with no model in the loop.

What the prediction actually is, since it is easy to read as a model result and is not one. No neural network appears anywhere in this subsection. The procedure is:

1. Join the two screens on systematic ORF. One row per deletion: 19 measured amino-acid concentrations from Mulleder, and one betaxanthin value from Cachera.
2. Take log of each concentration (they are skewed positive quantities) and z -score each of the 19 columns.
3. Fit **ridge regression** from those 19 numbers to the betaxanthin value, with the penalty chosen by inner cross-validation.

4. Score **out of fold**: five-fold cross-validation, so every prediction is made by a model that never saw that deletion, then correlate the held-out predictions against the measurements. Repeat over three fold shuffles and report the mean.

So $r = 0.298$ means: *knowing a deletion’s 19 amino-acid concentrations lets you predict its betaxanthin at $r = 0.298$ on deletions you did not fit on.* The comparison row “tyrosine alone” is the same procedure with one column instead of nineteen. It is a statement about the two measurements, and it puts a floor under what any model sharing a representation between these phenotypes could hope to exploit.

predictor of betaxanthin	r	SE at this n	SD across shuffles	n
tyrosine alone (the precursor)	0.0641	0.0150	0.0039	4,432
19 amino acids jointly	0.2980	0.0137	0.0029	4,432
<i>on the 3,708 deletions with a Costanzo fitness record</i>				
single-mutant fitness alone	0.1508	0.0161	0.0028	3,708
19 amino acids	0.1763	0.0159	0.0025	3,708
19 amino acids and fitness	0.2001	0.0158	0.0010	3,708
19 amino acids, fitness regressed out of both	0.1326	0.0161	0.0026	3,708
<i>on the 724 with no Costanzo record</i>				
19 amino acids	0.4547	0.0295	0.0219	724

Table 19: Cross-validated ridge, five folds, three shuffles; amino acids enter as log concentration, z -scored. **Two different spreads are reported because they answer different questions and quoting only one is misleading.** The *SE at this n* is the Fisher standard error, $1/\sqrt{n-3}$ mapped back at the observed r , and it is the one to use for “how precisely is this correlation measured”: it is what separates the 0.298 from the 0.064, at 17 standard errors. The *SD across shuffles* is fold-assignment noise on the same deletions, which is why it is an order of magnitude smaller; it says the procedure is stable, not that the estimate is precise. A third spread, the SD across the five folds, runs 0.025 to 0.064 and is an upper reference rather than a standard error, since each fold estimates on $n/5$ deletions.

Tyrosine is not the axis. Its marginal correlation with betaxanthin is $r = -0.076$ ($p = 3.9 \times 10^{-7}$), which is negative, and it ranks sixth of nineteen by absolute correlation. The strongest is methionine at $+0.140$, followed by aspartate $+0.127$ and arginine -0.118 . No amino acid exceeds $|r| = 0.15$. Correcting for the betaxanthin side’s known reliability of 0.836 raises these by a factor of 1.094, which does not change the reading; the Muller side’s correction is unmeasurable and would raise them further by an unknown amount.

The profile is the axis. The 19-dimensional pool predicts betaxanthin at out-of-fold $r = 0.298$, roughly $4.7\times$ the precursor alone.

Revision 2 set that beside the CGT’s own genotype-to-betaxanthin score and said a linear map “recovers most of what the model recovers from genotype”. **That comparison was not valid** and it is withdrawn: the two numbers use different splits and different scoring rules, and one takes measured metabolites as inputs while the other takes only the genotype. The question the comparison was reaching for is answered properly below, by a measurement rather than by placing two numbers next to each other.

5.4 Is the coupling redundant with what betaxanthin supervision already teaches ✗

This is the objection that decides whether the coupling is worth building on, and revision 2 did not address it. Both the amino-acid pool and betaxanthin are readouts of the same deletion. So an r of 0.298 is equally consistent with two very different worlds:

- (i) the pool carries betaxanthin-relevant information that a genotype-to-betaxanthin model does *not* already extract, in which case a shared encoder has something to gain; or
- (ii) both are shadows of one genotype axis the Cachera labels already teach, in which case no amount of architecture can beat betaxanthin-only training, and the 0.298 is real but useless.

The measurement that separates them. Take a trained betaxanthin-only model, read its *held-out* predictions, and regress those out of both sides before asking what the amino-acid pool still predicts. If world (ii) holds, nothing survives. Run over the ten grid cells that dumped test predictions, on the 816 held-out deletions carrying both measurements:

So world (i) holds. There is betaxanthin-relevant information in the free amino-acid pool that the genotype-to-betaxanthin map does not capture, and the joint arm’s failure to use it is therefore a statement about the auxiliary head rather than about the signal.

control model (grid cell)	r of the model	r of 19 AA	19 AA given the model	corr of the two
s08_L6_maskon_lr0.0001	0.3564	0.2720	0.2179	0.2185
s09_L6_maskon_lr0.0001	0.3305	0.2720	0.2262	0.2050
s01_L2_maskon_lr0.0001	0.3074	0.2720	0.2139	0.2263
s05_L2_maskoff_lr0.0001	0.2913	0.2720	0.2259	0.1768
s04_L2_maskoff_lr0.0001	0.2761	0.2720	0.2380	0.1660

Table 20: Against the strongest available control, the amino-acid pool keeps 0.2179 of its 0.2720, which is 80 %, after the betaxanthin model’s own held-out prediction is removed from both sides. At $n = 816$ that is 6.5 Fisher standard errors from zero. The last column is the direct check on the same point: the model’s predictions and the pool’s predictions correlate at only 0.22, where near-total redundancy would need something above 0.75. Five further collapsed cells with $r_{\text{model}} \leq 0.10$ are omitted, since residualizing on a failed model removes nothing by construction.

Three things this does not establish, kept separate because they are separate. It controls for a *linear* function of the model’s prediction, so non-linear redundancy is untouched. The strongest control available is $r = 0.3564$, below the strand’s best score, so redundancy against a better model than any that dumped predictions is not ruled out. And “the coupling is real and incremental” is a different claim from “an auxiliary head is the mechanism that converts it into a gain”; the second is what the next subsection finds against.

Why fitness enters at all, since it is a third dataset and looks like a detour. Both screens are deletion phenotypes and both respond to how well the strain grows. A slow deletion can have a distorted amino-acid pool *and* a distorted pigment readout with no metabolic relationship between the two, which would produce exactly the correlation above for an uninteresting reason. Costanzo single-mutant fitness (KanMX deletions, 30 C) is the standard measurement of that shared nuisance, so it is regressed out of both sides and the fit recomputed on the residuals. It is a control, not a third phenotype in the story.

Single-mutant fitness alone predicts betaxanthin at 0.151, so the shared growth component is real and substantial. Regressing it out of both sides leaves the amino-acid profile at 0.133 against 0.176 without the control on the same genes, so roughly three quarters of its predictive power survives. Combining the two reaches 0.200, above either alone.

Correction: most of that apparent shrinkage was the gene set, not the control. Requiring a Costanzo record shrinks the population from 4,432 to 3,708, and the 724 it drops are not a random quarter. Rather than accept that, every single-deletion fitness source in the repository was pooled. Kuzmin does *not* help, and the reason is structural: Kuzmin 2018 and 2020 screened the same SGA deletion array Costanzo did, agreeing with it at $r = 0.947$ to 0.995 on the overlap, so between them they recover only 56 of the 724. Baryshnikova 2010 recovers 375, O’Duibhir 2014 a further 167, and the Costanzo NatMX query arm, which the incumbent control excluded, another 317. The union controls **4,214 of the 4,432**, leaving 218.

gene set	n	19 AA, no control	fitness alone	19 AA, fitness controlled
Costanzo KanMX only (incumbent)	3,708	0.1763 ± 0.0025	0.1508	0.1326 ± 0.0026
union of all SMF sources	4,214	0.2884 ± 0.0014	0.1332	0.2699 ± 0.0017
all shared, no control possible	4,432	0.2980 ± 0.0029	–	–
still uncontrolled after the union	218	0.3150 ± 0.0312	–	–

Table 21: The fitness control costs far less than it appeared to. On the incumbent gene set the control looks like it removes 0.044 of predictive power ($0.176 \rightarrow 0.133$); on the enlarged set it removes 0.019 ($0.288 \rightarrow 0.270$). Most of the apparent shrinkage was the population that requiring a Costanzo record selects, not the control itself. Donor screens are mapped onto the Costanzo scale by least squares on their overlap and standardized within donor; transfer correlations run 0.951 (NatMX) and 0.891 (Baryshnikova) down to 0.511 (Kuzmin 2020), so the weakest donors carry the most transfer error. The incumbent row reproduces the committed result to 13 decimal places, which is what makes the two rows comparable.

So the conclusion strengthens rather than weakens: **94 % of the amino-acid profile’s predictive power survives controlling for single-mutant fitness** on the population where the control can be applied at all.

And the deletions no fitness screen reaches carry the extremes. The 724 deletions with no Costanzo KanMX record have a betaxanthin standard deviation of 0.914, against 0.464 among the 3,708 that do have one, so they are roughly twice as spread out in production and carry a disproportionate share of the extreme producers. On that same 724 the amino-acid profile predicts at 0.455. Even after the union above, 218 remain uncontrolled, and the profile predicts on those at 0.315.

Why they are missing is itself informative, and unverified here. A gene absent from a KanMX deletion-collection fitness screen is typically one whose deletion grows too poorly to score, or that the collection never carried. Either way

the missing set is enriched for strongly-affected strains, which is the plausible reason it carries the extremes. That mechanism has not been tested.

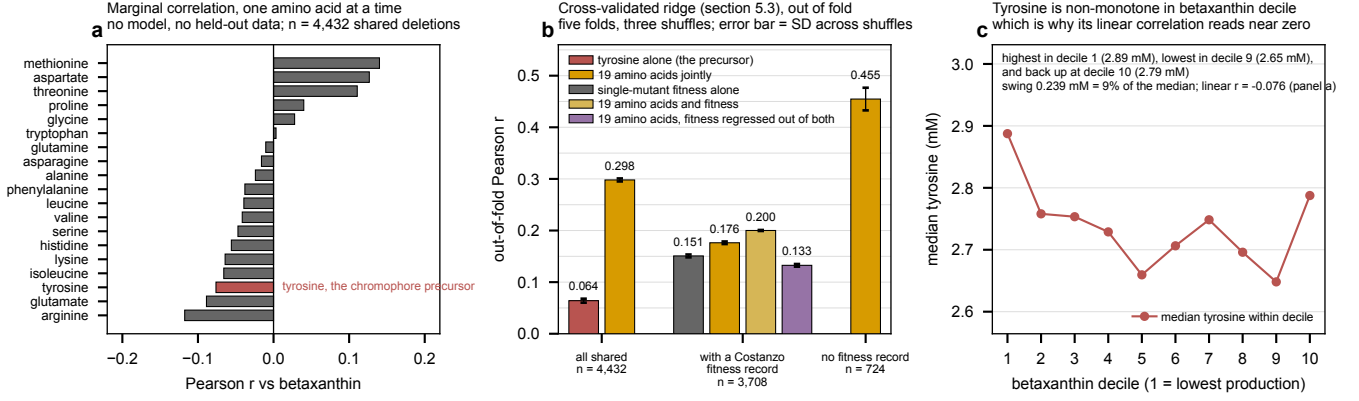


Figure 9: (a) Marginal correlation of each amino acid with betaxanthin over the 4,432 shared deletions; tyrosine highlighted. These reproduce `experiments/019-simb-multimodal/results/pigment_noise_ceiling.json` exactly, which the generating script asserts rather than assumes. (b) Cross-validated ridge fits from table 19. (c) Median tyrosine by betaxanthin decile. The relationship is non-monotone: tyrosine is highest in the lowest-production decile, falls through the middle, and rises again at the top, which is why a linear correlation reads near zero. The axis spans 2.65 to 2.89 mM, so the swing is about 9% of the median.

A mechanism, labeled as unverified. The Cachera background carries $ARO4^{K229L}$ and $ARO7^{G141S}$, feedback-resistant alleles that release the shikimate pathway from the control that shapes the pools Muller measured in the plain deletion collection. A plausible reading is that free tyrosine in an unengineered strain is not the quantity that limits flux in an engineered one. This has not been tested and no experiment here can test it.

5.5 Does the metabolome head actually help the betaxanthin head $\times$

The claim under test is narrow and was stated before the round: in Delta grid cell `s04_L6_maskon_lr0.0001`, the joint arm read +0.0203 over the control on `val/betaxanthin/pearson_per_feature` at a matched budget of 333 epochs. That was one paired difference at one seed with no estimate of the noise floor on it. The same grid read -0.049 and -0.060 in the two $L = 2$ cells, both run to 1,000 epochs, so the sign flips with depth.

grid cell	runs (ctrl/joint)	control	joint	Δ	aux r	epochs (ctrl/joint)
<code>s00_L2_maskon_lr0.0001</code>	1/1	0.3852	0.3434	-0.0419	0.147	998/998
<code>s02_L2_maskoff_lr0.0001</code>	1/1	0.3492	0.3078	-0.0414	0.053	998/998
<code>s04_L6_maskon_lr0.0001</code>	1/1	0.3780	0.3956	+0.0176	0.116	998/998
<code>s06_L6_maskoff_lr0.0001</code>	1/1	0.4261	0.3498	-0.0762	0.061	998/998
<code>s08_L4_maskon_lr0.0001[†]</code>	9/9	0.4059	0.4173	+0.0113	0.119	998/746
<code>s10_L4_maskoff_lr0.0001</code>	1/1	0.3606	0.3628	+0.0022	0.053	998/998
<code>s01_L2_maskon_lr0.001*</code>	2/2	0.0287	0.0413	+0.0125	0.026	998/998
<code>s03_L2_maskoff_lr0.001[†]*</code>	5/1	0.0341	0.0310	-0.0031	0.017	490/998
<code>s05_L6_maskon_lr0.001*</code>	1/1	0.0674	0.0203	-0.0471	0.007	998/998
<code>s07_L6_maskoff_lr0.001*</code>	1/1	0.0428	0.0438	+0.0010	0.011	998/998
<code>s09_L4_maskon_lr0.001*</code>	1/1	0.0424	0.0408	-0.0017	0.009	998/998
<code>s11_L4_maskoff_lr0.001*</code>	1/1	0.0533	0.0478	-0.0055	0.008	998/998
mean, cells where the control learned (5 cells)				-0.0279 $\pm$ 0.0169		2 of 5 positive
mean, all matched-exposure cells (10 cells)				-0.0181 $\pm$ 0.0099		4 of 10 positive

Table 22: Betaxanthin score with and without a 19-amino-acid auxiliary head, paired within grid cell, on `val/betaxanthin/pearson_per_feature`. Δ is joint minus control. *aux* is the auxiliary head’s own score, carried because an auxiliary head at $r \approx 0$ that still moved the primary metric would mean the gain came from regularization rather than from shared metabolic signal, and those are different findings. [†] marks a cell whose two arms ran epoch counts differing by more than 50; since the score is a running maximum, such a difference is part effect and part unequal exposure, and those rows are excluded from both means. * marks a cell whose control arm scored below 0.10, meaning neither arm learned the task; every one of them is an $lr = 10^{-3}$ cell, and a difference between two failed runs measures nothing about the auxiliary head. **Run accounting.** The project holds 46 runs, 46 of which carry both a grid-cell tag and an arm tag, and 46 of those logged a betaxanthin validation curve; the remaining 0 are excluded rather than averaged in as blanks. **Within a cell an arm’s score is the MEAN over its runs, not the maximum**, because a maximum over an unequal number of runs per arm is a selection on the quantity being compared; n per arm is given in the cell column. Unlike every other number in this document, this table is built at full per-run history coverage for its project (table 25), since the default candidate rule ranks runs by final score and would select one arm over the other.

Scored uniformly across the grid, the effect is **negative and small**, and the honest summary of it is three sentences rather than three readings.

The mean over the cells whose control arm learned the task at all is negative, the spread across cells is several times the effect, and every cell carries $n = 1$ per arm except `s08`, so there is no within-cell noise floor to measure the difference against. Depth and the graph mask each move the result by more than the auxiliary head does, which means any single-cell claim about the head is smaller than the variation across the cells it could have been made in. And the auxiliary head is itself weak wherever the primary head is good, scoring well under the 0.209 the amino-acid strand reaches when it is the only head, so the joint arm is paying capacity for a head that is underperforming.

Read this as one crude attempt, not as a verdict. Adding an unstructured second head is the least informative version of the question. Nothing in the architecture says the two phenotypes are connected through a pathway, so the encoder is being asked to discover a stoichiometric relationship from 19 noisy concentrations and one pigment readout. Given table 20, which shows the information is genuinely there and genuinely incremental, the finding to carry forward is that **the route by which a second head’s representation reaches the first head’s readout is the thing that failed**, and that a constraint-based coupling is the version worth trying (section 8).

The designed replication, `experiments/023-metabolome-betaxanthin-joint/conf/igb_bx_aa.yaml`, adds a third setting: an auxiliary head restricted to **tyrosine only**, against a common control, on a gene set pinned by `require_head_targets` so no arm can win by seeing more rows. **It has not been run.** Given table 19 its tyrosine arm is now predicted to be the null one and its 19-amino-acid arm the live one, which is a sharper prediction than the config was written against.

The coupling is real, incremental, and not being used. The free amino-acid profile of a deletion predicts that deletion’s betaxanthin at $r = 0.298$ out of fold, against 0.064 for tyrosine, the chromophore precursor, so the axis is the profile and not the precursor. It is not a growth artifact: pooling every single-deletion fitness screen in the repository controls 4,214 of the 4,432 shared deletions, and 94 % of the profile’s power survives that control. It is not redundant with betaxanthin supervision either: 80 % of it survives residualizing out a trained betaxanthin model’s own held-out predictions, which correlate with the profile’s predictions at only 0.22. And an unstructured auxiliary head does not convert any of that into a gain, scoring negative across the grid. **The bottleneck is the route from a second head’s representation to the first head’s readout, not the availability of the signal.**

Cannot say. How much of the 0.298 is attenuated by Mulleder’s unmeasurable noise, so it is a floor rather than an estimate. Whether the redundancy result survives a *non-linear* control, or a control model stronger than the 0.356 best among those that dumped predictions. And whether any architecture converts the coupling into a gain, since the only mechanism tried is the one least likely to work.

6 Beta-carotene production ✗

6.1 Why this target is different from the other pigment ✗

Ozaydin 2013 (doi:10.1016/j.ymben.2012.07.010) screened the deletion collection carrying a beta-carotene pathway and scored each colony’s color **by eye on an ordinal scale from -5 to $+5$** . The dataset holds 4,474 records, of which 4,344 have a single replicate and 130 have two or three. That is a different kind of measurement from the Cachera fluorescence readout, and it needs a different ceiling: a Pearson ceiling on a subjective ordinal is the wrong object, so rank agreement is used.

Two estimates exist, and they disagree by a factor of seven.

estimate	basis	n	Spearman
replicate max against min	130 replicated rows	130	0.544
independent re-screen	<code>2ndRoundOfTransformations</code> sheet	119	0.075
range-restriction corrected	Thorndike case 2	119	0.105

Table 23: Reliability of the beta-carotene score. Neither estimate is clean. The max-against-min comparison is biased in both directions at once: the two are drawn from one replicate set, which inflates agreement, and they are the widest possible split of that set, which deflates it. The re-screen is a genuine test-retest but was run on selected top hits, so its 1st-screen scores sit almost entirely in $3 \dots 5$ of a $-5 \dots +5$ scale.

Why the 0.075 is not evidence that the score is worthless. The re-screen is the better *design* of the two, a genuine independent test-retest rather than two draws from one replicate set. But it was run only on **selected top**

hits, so its first-round scores sit almost entirely in 3...5 of a -5...+5 scale, with a standard deviation of 0.89 against 1.47 over the full screen (fig. 10b).

A correlation computed on a narrowed slice of a population is attenuated by the narrowing alone, independently of how good the measurement is. The intuition: if every strain re-screened is already known to be a high producer, the only variation left to correlate is the variation *among* high producers, which is small and mostly noise. Thorndike's case 2 formula corrects for exactly this, given the restricted and unrestricted standard deviations, and it takes the 0.075 to 0.105. That is still low, but it is a correction for one restriction, and the sampling is selected on more than range alone.

The reported ceiling is therefore 0.544, from the replicate comparison, with the caveat that neither estimate is clean and they disagree by a factor of five even after correction.

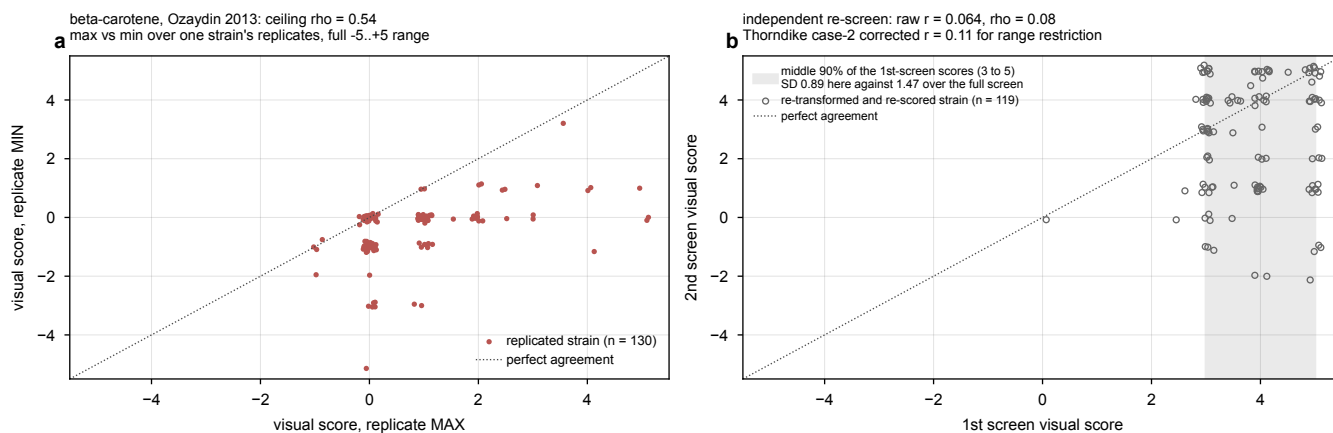


Figure 10: Reliability of the beta-carotene score, which is rank agreement rather than a Pearson because the target is a hand-scored ordinal. (a) The primary estimate, comparing the maximum against the minimum score among the 130 strains with more than one replicate, over unrestricted strains: Spearman 0.544. (b) The independent re-screen, which is a genuine test-retest but was run only on selected top hits; the shaded band marks the middle 90 % of its first-screen scores, which spans a standard deviation of 0.89 against 1.47 over the full screen. That narrowing is the range restriction responsible for its raw 0.075, and it is visible here rather than asserted.

6.2 What was measured ~~x~~

The strand is 136 runs across three projects. The best validation `spearman_per_feature` is **0.1371**, run `5cjpn6zj` in `beta_carotene_v3`, peaking at **epoch 9 of 49**. The median over all 136 is 0.0513. Against the 0.544 ceiling that is 25 % realized.

Correction, and it is a correction to the method rather than to a judgment call. Revision 2 reported the best as 0.118 from run `o0aadmot` peaking at epoch 799 of 999. That run is only the *third* best in the strand. The leaderboard had fetched validation curves for a candidate subset of each project, chosen as the top runs by final logged value and by epoch count, and `5cjpn6zj` peaks at epoch 9, so it ranks low on *both* keys at once and was invisible to the rule. **This is a structural blind spot, not bad luck:** a run that peaks very early and then decays is exactly what the candidate rule cannot see, and it falsifies the previous assumption that incomplete run coverage risked only medians and never maxima (section 7.1).

Beta-carotene is limited by its measurement, not by the model. A hand-scored ordinal with one replicate per strain has a reliability no architecture recovers, and the 0.1371 best `spearman_per_feature` sits at 25 % of a 0.544 ceiling that is itself uncertain by a factor of five. The strand carries real signal and is worth keeping for the coverage argument in section 4, where the claim is about which genes a flux model can represent and does not depend on the absolute score.

Correction to revision 2 on convergence, which the coverage fix reverses. Revision 2 said the best run “peaks late (epoch 791 of 999), so it shares the expression strand’s under-training rather than the metabolite strands’ early overfitting”. With the true best run in hand, the opposite is the case: it peaks at **epoch 9 of 49** and decays. So beta-carotene groups with the *metabolite* strands, which peak early and then overfit, not with expression. That is also what makes it a low-value target for more compute: extra epochs are not what it is short of.

Cannot say. Which of the two ceiling estimates is right, and therefore whether 0.1371 is a quarter of achievable or most of it. That gap is resolvable only with a replicate design the source data does not contain.

Recommendation. Keep beta-carotene as a second production target for the coverage argument in section 4, where the claim is about which genes a flux model can represent and does not depend on the absolute score. Do not spend a sweep on lifting its number.

7 What the strands have in common ✗

7.1 What the leaderboard covers, and what it does not ✗

The strands span **28 W&B projects and 2,187 runs**. Every one of them is enumerated, with its run count, its metric names and its training-set size, by a census that reads summaries only and therefore covers everything.

SRC: experiments/019-simb-multimodal/scripts/project_census.py

strand	projects	runs	n_{train}	values seen
expression	9	1,369	1,125 – 1,253	
expression + morphology joint	5	214	1,161	
morphology	3	183	1,161	
betaxanthin	4	142	3,694 – 4,235	
beta-carotene	3	136	3,846, 3,849	
amino acid	3	97	4,234, 4,235	
betaxanthin + metabolome	1	46	3,832	
total	28	2,187		

Table 24: The whole account. Two columns carry findings on their own. Morphology is the only strand whose training-set size never varies, and the value it never varies from is a quarter of its screen. The betaxanthin range is why its two headline numbers are not comparable (table 15).

Enumerating a run and reading its curve are different things, and revision 2 conflated them. All 28 projects were enumerated, and revision 2 reported that as complete coverage. But finding a run’s *best* value needs its full validation history, one request per run, and that had been fetched for only **306 of the 2,187 runs**. The fetch rule was a per-project candidate set: the top 8 runs by final logged value, plus the top 5 by epoch count, at most 13 per project.

strand	projects	runs	history read	with a usable curve
expression	8	1353	1353 (100%)	1182
expression morphology joint	5	214	214 (100%)	209
morphology	3	183	183 (100%)	183
betaxanthin	4	142	142 (100%)	135
beta carotene	3	136	136 (100%)	136
amino acid	3	97	97 (100%)	97
betaxanthin amino acid joint	1	46	46 (100%)	46
expression masked	1	16	16 (100%)	9
all	28	2187	2187 (100%)	1997

Table 25: Coverage at two granularities, which are two different claims. All 28 projects have been read, meaning every project’s run list and every run’s final summary value is in hand. Reading a run’s best score is a separate request per run.

Every run’s history has now been read. An earlier pass read only a bounded candidate set per project, the eight highest final values and the five longest runs unioned, at most thirteen per project, on the assumption that one request per run would not finish against a rate-limited API. It does finish: the full pass takes about twenty minutes. Until it was run, every strand maximum in this document was a maximum over at most thirteen runs per project rather than over the runs that were run. The *history read* column is the runs whose curves were requested; *with a usable curve* is the subset that also logged the strand’s metric under a name the puller knows, and the gap between the two columns is runs that logged nothing under it. **Why the bounded rule was the wrong instrument.** It ranks by final value and by length, so it is built to find a maximum and is not a random sample: a mean or a median over it is conditioned on a high final value or a long run, and a paired difference computed over it selects on the very quantity being compared. **It can also miss a maximum, and did.** A run that peaks early and then stops short ranks low on final value and low on length, so it is invisible to both arms of the rule at once: reading beta-carotene at full per-run coverage raised that strand’s best from 0.118 to 0.137, held by a run that peaked at epoch 9 of 49.

That rule is biased in three ways, and one of them reached the document. It is a maximum-finder, so any *mean* or *median* over the fetched subset is conditioned on a run having ended high or run long. The amino-acid median was reported as 0.0498 “across the 31 runs”, and those 31 were exactly the candidate subset of 97; at full coverage the

median is 0.0570. It also ranks by the *final* logged value, which this document elsewhere shows can sit thousands of epochs from the peak, so the ranking key is the statistic the document argues against. And it spends slots on runs that never logged the metric at all.

And the rule missed a maximum, which is worse than the bias it was known to have. The assumption written into the puller was that incomplete coverage risks a median but never a maximum, since a maximum can only move upward as more runs land. That assumption is now falsified. Beta-carotene’s true best run peaks at **epoch 9 of 49**, which puts it low on final value *and* low on epoch count, so it was invisible to both arms of the rule at once. The strand’s best was reported as 0.118 and is 0.1371 (section 6). A run that peaks very early and decays is a structural blind spot of this rule, not an unlucky draw.

Where coverage stands: complete. All 2,187 runs across all 28 projects now have their validation curves read, so every score quoted below is a maximum over the runs that were actually run. The pull took about 25 minutes at roughly 1.3 seconds per run, which is worth recording because the previous revision treated full coverage as prohibitively expensive and it was not.

Two things changed and one did not. Beta-carotene’s best rose from 0.118 to 0.1371, as above. Reading everything also surfaced 8 collapsed morphology runs the candidate set never sampled, and moved the medians, which were the statistic most exposed to the old rule. **Every other strand maximum was already correct:** expression 0.238 (the maximum of eight identical-config replicates whose mean is 0.196, section 10), morphology 0.082, betaxanthin 0.401, amino acid 0.209, and the expression-morphology joint arm 0.062 are unchanged at full coverage. So the candidate rule was mostly doing its job, and its one failure is the instructive part rather than the typical case.

One resolution caveat, because it explains small disagreements. History is sampled, and the leaderboard reads every curve in one uniform pass at 500 points. section 2.3’s diagnosis samples the same two runs at 4,000 points and smooths more heavily, because it is locating turning points rather than scoring. The same run therefore reads 0.2382 here and 0.2362 there. The leaderboard value is the one quoted as a score; the diagnosis values are locators. Coarser sampling can only lower a `roll_max`, so these scores are lower bounds in that direction too.

7.2 One table, read the same way ✗

Table 1 is the whole account and sits in section 1; it is not repeated here. What the two panels below add is what a table cannot show, the size of each gap and where inside its own budget each best run peaked.

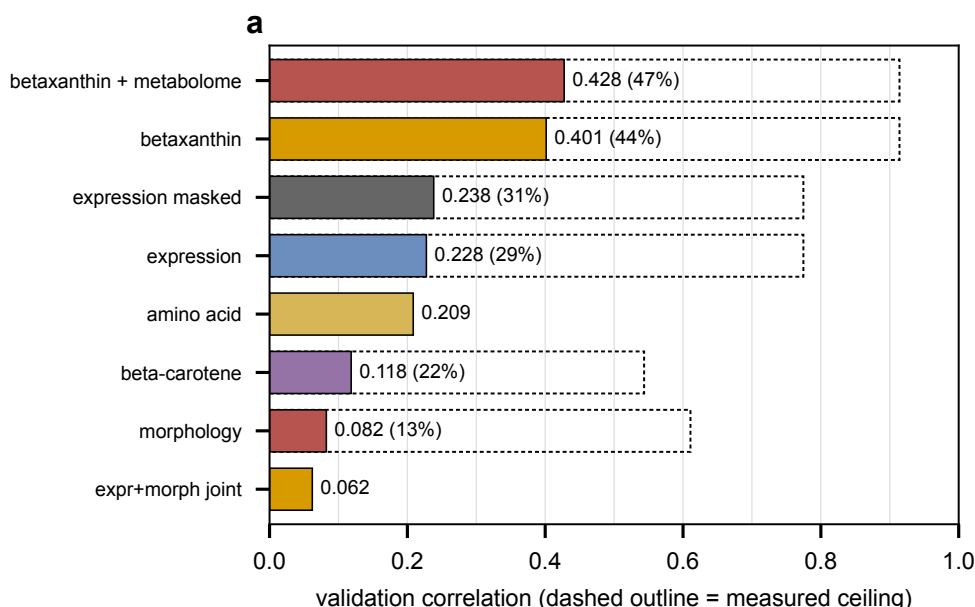


Figure 11: Achieved score against measured ceiling, best run per strand. The dashed outline is the ceiling and the gap is the headroom; amino acid and the expression-morphology joint arm carry no outline because no single ceiling applies to them. A bare correlation is not comparable across these rows, which is the reason the panel exists.

Three things are visible only when the strands sit together. The ceilings differ by more than a factor of two, so an unqualified r is uninterpretable across rows. The epoch budgets differ by more than a factor of a hundred, so the score column is partly a record of how long each strand was allowed to run. And the two strands closest to their ceilings, betaxanthin and amino acid, are the two with a scalar or low-dimensional target.

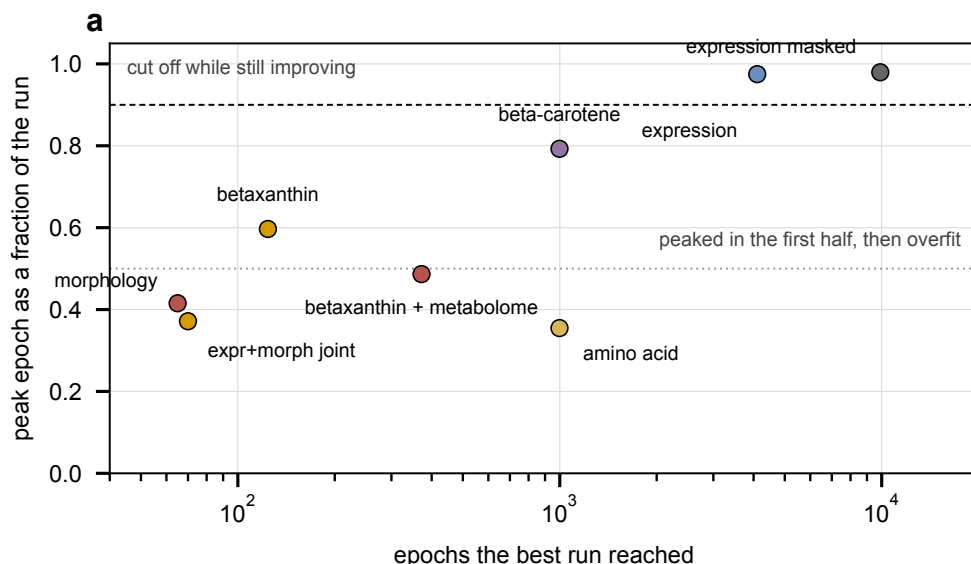


Figure 12: Where the best run’s peak sits inside its own budget. Points near the top were still improving when the run stopped, so their scores are lower bounds set by compute. Points in the lower band peaked early and spent the rest of the budget overfitting.

7.3 Two opposite convergence failures, and neither was diagnosed from a curve ✗

- **Expression’s best runs peak late, but “never peaks” did not survive the replicates.** Raising the budget from 1,600 to 10,000 raised the best score from 0.204 to 0.238 and moved that run’s peak with it, and the two best 9,900-epoch runs peaked in their last 12%. But five of the eight identical-config runs at that budget peaked between epochs 2,199 and 4,109 and never exceeded that value afterward (section 10), so more budget raises the maximum draw rather than every draw.
- **The metabolite and pigment strands peak early and then overfit.** Amino-acid runs peak at epochs 48 to 134 with validation slope turning negative while training slope stays positive. Beta-carotene’s best run peaks at epoch 9 of 49, which is the most extreme case in the project.
- **Morphology was never run long enough to tell which it does.** Longest run, 121 epochs.

Same encoder, same optimizer family, opposite behavior. The difference is in the target: a 6,127-dimensional vector of $\log_2$ ratios against a scalar or a 19-vector. Any single epoch budget applied across strands is wrong for at least two of them, which is what produced the truncated expression waves and would produce overfit metabolite runs if the budget were set from expression.

7.4 The training objective diverges from the metric on expression alone ✗

The full measurement is section 2.3; what belongs here is the cross-strand statement. On expression the *pinball loss* bottoms at epoch 463 and rises for the next 9,500 while Pearson climbs. *Squared error itself does not diverge*: its minimum lands alongside the Pearson peak on both long runs. Separately from the divergence, the model sits at parity with the mean predictor in squared error, at $\text{nmse} \approx 1.00$ to 1.04 depending on which epoch is called the peak (table 5), and goes clearly below 1 once rescaled.

Correction, and it reverses revision 2’s claim about 010. Revision 2 said “the 010 fitness and interaction task shows neither behavior, with both losses falling monotonically and the validation-loss minimum at the last epoch”, citing no run and no script. It was pulled and read, 20 runs of which 16 carry a live gene-interaction metric. Every part of that sentence is wrong.

- **010 trains gene interaction only.** Its config normalizes `gene_interaction` and nothing else, and no run logs a fitness key. The phrase “fitness and interaction” should not appear.
- **Neither loss is monotone wherever monotonicity is testable.** Validation loss is monotone in 11 of 16 runs, but every one of those 11 is capped at 20 or 30 epochs, too short to have turned. Of the 5 runs that passed 40 epochs, *zero* are monotone: they carry 3 to 9 validation-loss increases over 41 to 63 steps.

- **010 shows the loss-metric divergence too, with the opposite sign.** The gene-interaction Pearson peaks *before* the validation-loss minimum in 13 of 16 scored runs, median gap 8 epochs and maximum 39, then falls by as much as 31 % while the loss keeps improving. On `1zs9pcj3` the metric peaks at epoch 24 and the loss bottoms at 63.

run	val-loss min epoch	metric peak epoch	peak	final	relative fall
<code>1zs9pcj3</code>	63	24	0.4520	0.4428	2.0 %
<code>yv4r30bi</code>	61	25	0.4472	0.4312	3.6 %
<code>c7671wgj</code>	48	24	0.4619	0.4572	1.0 %
<code>t34kenj1</code>	29	13	0.3331	0.2414	27.5 %
<code>g5gev0w2</code>	19	11	0.3236	0.2230	31.1 %

Table 26: On 010 the validation loss keeps falling after the metric has peaked, which is the mirror image of expression, where the pinball loss turns early and the metric keeps climbing. Both are loss-metric divergences, so 010 is not the clean counterexample revision 2 used it as. 010 has also never approached its own budget: `trainer.max_epochs` is 600 and no run passed 65.

What survives is the weaker and still useful statement: the divergence takes a different form on each target, so no single stopping rule or checkpoint criterion is correct across them, and best-by-loss checkpointing is wrong on both, early on expression and late on 010. SRC: `experiments/019-simb-multimodal/scripts/check010\_loss\_metric\_curves.py`

So the divergence is a property of one objective on one target, not a general feature of the multitask setup, and the metabolite strands, which peak early and overfit, are governed by something else. Any campaign that sets a stopping rule or a checkpoint criterion from one strand and applies it to another will be wrong for at least one of them.

And the calibration failure is not expression’s alone. Reading `nmse` at each strand’s correlation peak, across every round that logs it:

strand	runs	<code>nmse</code> > 1	best r	<code>nmse</code> at peak	$1 - r^2$
amino acid	31	25 of 31	0.2085	0.9607	0.9565
beta carotene	39	38 of 39	0.1184	1.2881	0.9860
betaxanthin	44	24 of 44	0.3705	0.9316	0.8627
betaxanthin amino acid joint	46	18 of 46	0.4297	0.8161	0.8153
expression	22	18 of 22	0.2276	1.0348	0.9482
expression masked	9	9 of 9	0.2382	1.0111	0.9432

Table 27: Whether the selected model beats “predict each feature’s mean” in squared error. `nmse` at peak is its value at the epoch the correlation peaked, which is the model anyone would ship. Above 1 means the model loses to the mean while still ordering features correctly, which is under-shrinkage rather than incapacity. **The arithmetic behind the last column.** For predictions that are a scaled copy of a correlated signal, $\text{nmse} = 1 + s^2 - 2rs$, where s is the ratio of prediction to target standard deviation. That is a parabola in s with its minimum at $s^* = r$ and value $1 - r^2$ there, so scaling every prediction by r/s moves `nmse` to $1 - r^2$ and, being a positive rescale, leaves every correlation untouched. $1 - r^2$ is therefore the floor this rescale can reach, and the gap between the two middle columns is what under-shrinkage costs. A strand already *below* $1 - r^2$ is not a contradiction: its predictions carry structure beyond one global scaling, so a single scalar is not the right correction there. Only rounds from the v8 era log these metrics; earlier ones are absent, not zero.

Two strands beat the mean predictor and two do not, and the split tracks the correlation rather than the phenotype: betaxanthin at $r = 0.43$ reaches `nmse` 0.81, amino acid at 0.21 reaches 0.96, expression at 0.24 sits at 1.01, and beta-carotene at 0.13 sits at 1.29 in every one of its twelve runs. The lower the achievable correlation, the further predictions drift above the spread that correlation justifies, which is what under-shrinkage looks like when most of the target is noise.

The consequence is operational and free. The rescale is a post-processing step, so every strand above 1 can be taken below it without retraining and without moving any correlation. Reporting a correlation next to an `nmse` above 1 is reporting a model that loses to the mean on a squared-error scoreboard, and that is avoidable before the figure is drawn rather than after a reviewer asks.

7.5 Configuration choice moves results more than method choice ✕

On betaxanthin the ten grid cells span Spearman 0.013 to 0.158 and AUC 0.406 to 0.695, against a between-method gap of 0.0014 on AUC. Four low points are the $\text{lr} = 10^{-3}$ cells, which collapsed to a constant predictor. On the amino-acid strand the best run is 0.209 and the median over all 97 is 0.0570.

Any single-number claim about a method is dominated by which cell is quoted, which is why the cell must be fixed by validation in advance, and why the leaderboard records the median alongside the maximum. It also means an arm difference smaller than the cell spread is not measurable without pinning the cell and replicating seeds inside it.

Is the spread just the dataset split. No, and this one has been measured rather than argued. The betaxanthin Optuna study repeated some hyperparameter configurations exactly, which gives a direct estimate of the floor:

repeated configuration	objective values	range
trials 3, 10, 11	0.4216, 0.4095, 0.3584	0.0632
trials 19, 31	0.4211, 0.3896	0.0315
pooled replicate σ (3 d.o.f.)		0.0302

Table 28: Re-running an identical configuration moves the objective by $\sigma \approx 0.030$. The observed spread across configurations is several times that, so configuration choice is doing real work; but any single difference under ≈ 0.03 is inside this floor.

Three components are separable in what has been run. **Collapsed configurations:** on betaxanthin and on the 023 grid every $\text{lr} = 10^{-3}$ cell fell to a near-constant predictor, and those account for the bottom of the range. **Genuine configuration effects:** depth and the graph mask move the 023 result by more than 0.05 (table 22), well outside the floor. **Run-to-run noise:** the $\sigma = 0.030$ above, which in rounds that pin `data_module.split_seed` is initialization only, since the partition is held fixed precisely so that varying `seed` varies weights and nothing else.

There is a fourth term that is not noise and is often mistaken for a result: **selection inflation**. Taking the maximum over 37 trials inflates it by an estimated 0.064, so the study’s headline 0.4301 corresponds to a floor of ≈ 0.366 for a configuration chosen without hindsight. Its top five trials span 0.0206, which is 0.68σ , so they are indistinguishable from each other.

7.6 Cross-modality overlap is real and small ✗

Which two datasets, and why this was measured at all. The **proteome** here is Messner 2023 (doi:10.1038/s41586-023-06739-5), a knockout-collection proteome measured in synthetic minimal medium at 30 C; the **expression** is the same Kemmeren 2014 knockout compendium the expression strand predicts (doi:10.1016/j.cell.2014.02.054), measured in synthetic complete medium at 30 C. They share 1,832 genes and 1,350 deletion strains. Joint proteome-and-expression training is not on the near roadmap, so this is a gating measurement rather than a result: before spending compute on that pairing, how much does one modality already tell you about the other?

Per-gene correlation across strains has median $r = 0.08$, with 1.7 % of genes above 0.3 and 9.4 % negative. A ridge map recovers held-out R^2 of 0.035 in one direction and 0.025 in the other, above a trivial baseline of -0.004 but far from redundancy. SRC: `experiments/019-simb-multimodal/results/proteome_expression_eda.json`

One qualification that bounds every number above. The two modalities were measured in **different media**, synthetic minimal for the proteome and synthetic complete for the expression. Media differences move both transcript and protein abundance, so part of the observed decoupling is a condition artifact rather than biology, and every correlation here is attenuated by an unmeasured amount. The overlap is therefore certified as *non-redundant* and not certified as cleanly *complementary*, and that distinction bounds how hard a “the proteome helps expression” claim can be pushed until a same-media pair exists.

Provenance note on these figures. This query fails against the served NCSA database with the known cold-read `IOException`, so it was re-run against the local GilaHyper build, and the endpoint is now recorded in the results JSON. Most values reproduce exactly; two moved slightly, fraction negative $8.9 \rightarrow 9.4\%$ and the reverse-direction R^2 $0.024 \rightarrow 0.025$, and the numbers above are the local build’s.

7.7 The graph prior, checked on one phenotype and then on six ✗

The nine interaction graphs do not inject features. They enter as a hard additive attention mask, so composing L layers makes the influence of gene X on gene Y approximately the L -step random-walk reachability of Y from X . The graph channel therefore rests on a falsifiable claim, and **the claim can be written two ways, which turn out to have different answers**.

The reporter form *Deleting X perturbs the reporter genes near X on graph k .* Responders are the top 1 % of reporters by $|\log_2 \text{ratio}|$; the statistic is the probability that a random responder sits closer to the deleted gene than a random non-responder. This needs a readout indexed by gene, so it exists for expression only.

protein: Messner 2023 YKO proteome, synthetic minimal (SM), 30 C
 mRNA: Kemmeren 2014 YKO expression compendium, synthetic complete (SC), 30 C

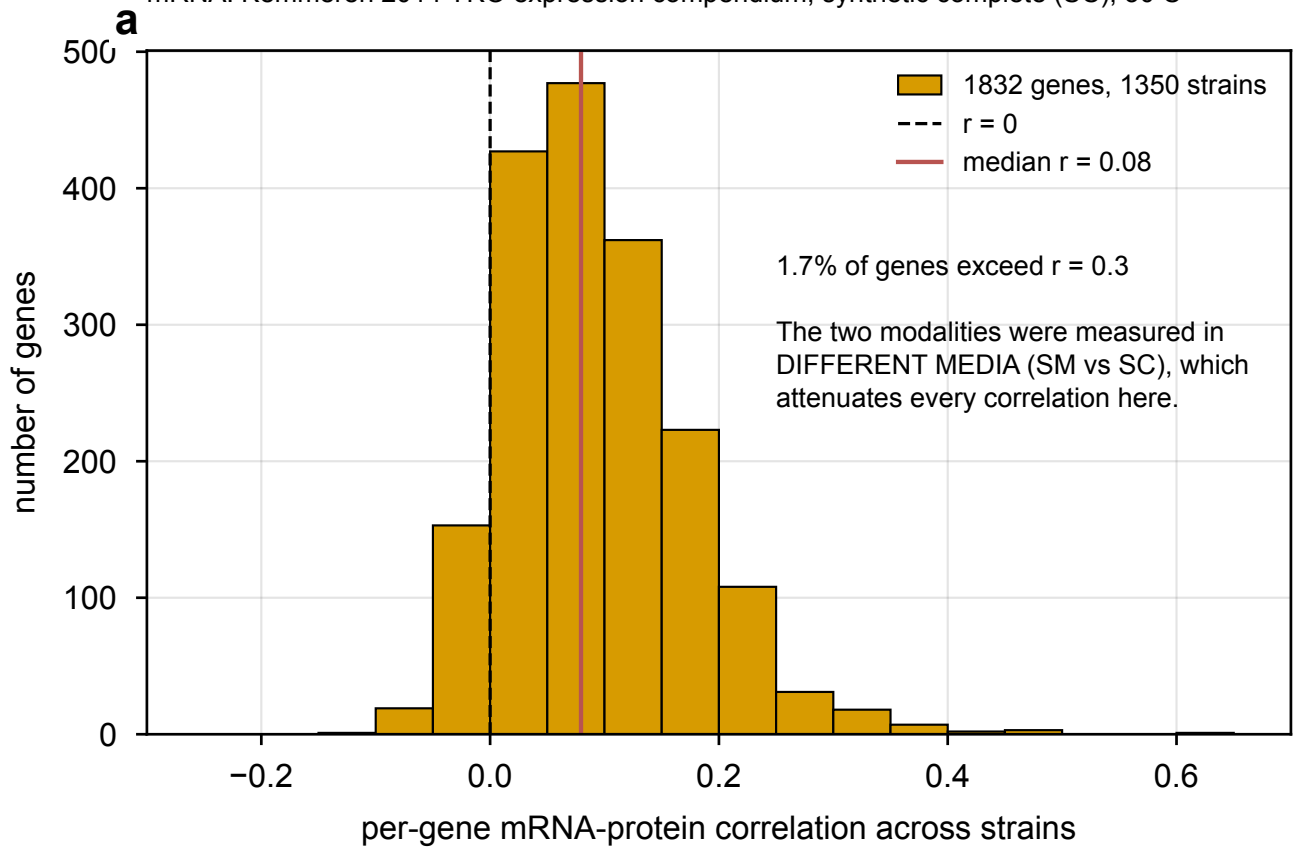


Figure 13: Per-gene correlation across strains between the Messner 2023 knockout proteome and the Kemmeren 2014 knockout expression compendium, over the 1,832 genes and 1,350 deletion strains they share. A narrow lump barely off zero: mRNA and protein levels for the same gene move largely independently across this panel. The few high- r genes are dominated by the strain in which that gene is itself deleted, which drives both its transcript and its protein toward zero and manufactures a correlation from one point.

The pair form *Deletions of two genes adjacent on graph k give more similar phenotypes than deletions of two unrelated genes.* Adjacent pairs are drawn as t -step random-walk endpoints, matching the reachability the composed attention implies; background pairs are drawn uniformly from the same phenotyped gene pool. Similarity is the across-feature correlation for a vector readout and the negative absolute difference for a scalar one. **This form is defined for every strand**, which is what makes the cross-phenotype comparison possible.

Both use the same two controls, a configuration-model redraw holding every node's degree and a matched-density random graph, and 0.5 is “the graph says nothing”. The pair form scores 60,000 pairs per cell, giving a sampling standard error of about 0.0017, so an excess over control beyond ≈ 0.005 is real. SRC: `experiments/019-simb-multimodal/scripts/graph\_prior\_probe.py`

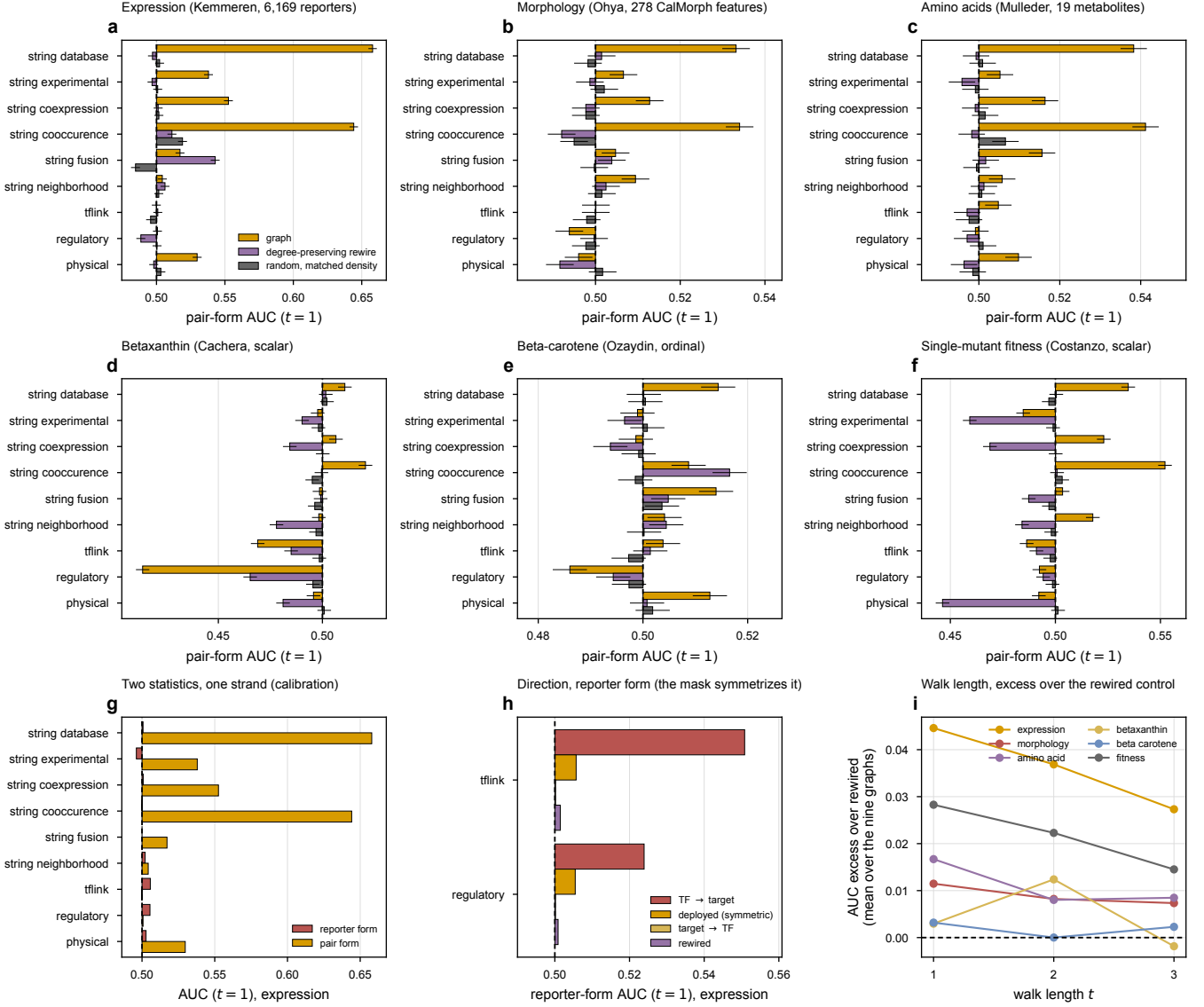


Figure 14: The graph prior across every strand. (a) to (f) one panel per phenotype, showing each of the nine graphs against its degree-preserving and matched-density controls, bars anchored at the 0.5 null so hundredth-scale deviations are visible. (g) the reporter form and the pair form on expression, the one strand where both are defined, which is what calibrates reading the pair form elsewhere. (h) the direction result on the two directed graphs. (i) how the excess over control changes with walk length. **Note the x axes differ between panels a to f**, because expression's effects are an order of magnitude larger than morphology's and a shared scale would flatten five panels to show one; compare each bar against its own controls rather than across panels, and read table 29 for the cross-panel comparison.

On expression the two forms disagree, and the pair form is the one with signal. The reporter form is at chance on all nine graphs: every AUC between 0.4961 and 0.5057, largest excess over the degree-preserving control +0.0046, unchanged when responders are defined at the top 5% instead of the top 1%. That was the whole basis of revision 2's “the graph prior is at chance”.

The pair form, on the same strand and the same graphs, reaches **AUC 0.6579 on STRING database** against a rewired control of 0.4971, an excess of +0.161, and 0.6441 on STRING co-occurrence against 0.5113. **So the graph**

channel does carry information about this phenotype, and revision 2’s headline was an artifact of asking the narrower question. The two forms are not contradictory: knowing that two adjacent deletions produce similar transcriptome-wide responses is a different and weaker statement than knowing which individual reporters move, and it is the one the data supports.

Different graphs matter for different phenotypes, which is the finding that decides what to do with the masks. All six strands were probed in the pair form: expression, morphology, the 19 amino acids, betaxanthin, beta-carotene, and Costanzo single-mutant fitness. The table gives the excess over the degree-preserving control, where anything beyond ± 0.005 is outside sampling noise.

graph	expression	morphology	amino acid	betaxanthin	beta-carotene	fitness
physical interaction	+0.032	+0.004	+0.013	+0.015	+0.012	+0.046
regulatory interaction	+0.012	−0.006	+0.002	−0.052	−0.008	−0.002
tflink	−0.001	0.000	+0.008	−0.016	+0.002	−0.005
STRING neighborhood	−0.002	+0.007	+0.005	+0.020	0.000	+0.034
STRING fusion	−0.026	+0.001	+0.014	−0.001	+0.009	+0.016
STRING co-occurrence	+0.133	+0.042	+0.043	+0.021	−0.008	+0.051
STRING coexpression	+0.051	+0.015	+0.017	+0.022	+0.005	+0.054
STRING experimental	+0.041	+0.008	+0.009	+0.007	+0.002	+0.025
STRING database	+0.161	+0.032	+0.039	+0.009	+0.014	+0.034

Table 29: Pair-form AUC minus its degree-preserving control, per graph and per phenotype, at $t = 1$. Sampling standard error ≈ 0.0017 over 60,000 pairs, so ± 0.005 is the noise floor. **The answer is not uniform, which is what the previous revision assumed it was.** STRING database and co-occurrence carry signal on four strands; **tflink** carries essentially nothing anywhere. Physical interaction is the strongest graph for *fitness* and second-strongest for expression, and sits at the noise floor for *morphology*, so a graph that is worth keeping for one phenotype is not automatically worth keeping for another. Regulatory interaction on betaxanthin is the one clearly negative cell: TF-adjacent deletion pairs give *less* similar betaxanthin than random pairs. Beta-carotene is the one strand where nothing exceeds +0.014, consistent with its coarse ordinal readout.

Coverage is a second problem, separate from signal. Many deleted genes have no edge at all on a given graph, so it makes no prediction for them: STRING neighborhood scores only 29 % of deleted genes, fusion and co-occurrence 46 %, database 75 %. A head constrained by a graph that is silent on most perturbations is unconstrained on those perturbations, which is a different failure from being constrained toward the wrong thing.

A namespace bug worth recording, because it would have produced a clean fake null. The Ohya CalMorph mirror stores systematic ORFs in lowercase. Joined naively against the genome’s uppercase names, morphology would have had *zero* graph overlap and every graph would have scored a tidy 0.5, indistinguishable from an honest null. It is fixed, and the script now raises if fewer than half of a strand’s genes resolve into the genome gene set, so this class of failure cannot pass silently again.

Direction carries signal in the reporter form, and the mask deletes it. Two graphs are directed. On tflink, following edges $\text{TF} \rightarrow \text{target}$ gives **0.5508** against a rewired control of 0.5017; on regulatory interaction, **0.5239** against 0.5009. Both are unambiguous. But `_build_attention_mask` sets `head_mask[i, j]` and `head_mask[j, i]` together, so the mask is symmetric, and averaging the informative direction with the uninformative one returns both to chance: 0.5057 and 0.5055.

This finding lives in the *reporter* form only, and necessarily so: pair-form similarity is a symmetric function of the two genes, so no orientation of the graph can change it. The two statistics are therefore complementary rather than redundant, and the direction result is not weakened by the pair form being unable to see it.

graph	TF $\rightarrow$ target	deployed (symmetric)	target $\rightarrow$ TF	rewired
tflink	0.5508	0.5057	0.5002	0.5017
regulatory_interaction	0.5239	0.5055	0.5002	0.5009

Table 30: The two directed graphs, AUC at $t = 1$. The signal is real and it is directional. Symmetrizing is not a neutral convenience here: it averages a genuine prior with its uninformative reverse. For the seven undirected graphs all three columns coincide, which is what makes this table’s machinery self-checking.

The graph masks stay, and revision 2’s recommendation to free them is withdrawn. The evidence for freeing them was an expression-only null in the reporter form. On the same strand and the same graphs, the pair form reaches AUC 0.6579 against a rewired control of 0.4971, so the channel carries information and the null was an artifact of the narrower question. Across six phenotypes the answer is not uniform: STRING database and co-occurrence carry signal on four strands, physical interaction is strongest for fitness and null for morphology, `tfink` carries nothing anywhere, and regulatory interaction is negative on betaxanthin. **Two changes follow, and neither is a removal.** Stop symmetrizing the two directed relations, which is a mask-builder change worth +0.04 to +0.05 AUC on the graphs it touches. And add unconstrained heads beside the masked ones, so the model can learn structure the graphs do not carry without giving up the ones that do.

What this does not establish. A data-side probe bounds what is available; it does not show the model uses it. That the information exists is necessary and not sufficient, and the kNN probe already showed the perturbed side failing to exploit structure that was present. Nor does a per-phenotype excess tell you which *head* should carry which graph: this measures the premise the mask encodes, not the architecture that consumes it. The one place the probe is genuinely decisive is negative, and `tfink` is that case, at the noise floor on all six strands in the pair form.

7.8 The perturbation axis is the binding resource ✗

Table 9 makes the point for expression: 9.2 million scalar labels across 1,484 perturbations, each gene deleted exactly once. Morphology is the same shape, 1.3 million labels across 4,718 perturbations, each once. The trigenic interaction data is the opposite shape, 91 thousand labels across 91 thousand records in which a given gene recurs with many partners.

The strands that work in this project are the ones where the supervision constrains a gene repeatedly. **Hypothesis, untested:** the reason multi-task training is attractive here is not that the labels are informative about each other, but that a second phenotype gives a second, differently-shaped constraint on the same perturbation.

What “share strains” rather than “share genes” means, since the phrasing was opaque. Two phenotypes **share a strain** when the *same physical deletion mutant* was measured for both, so the two labels constrain one perturbation jointly. They merely **share genes** when both screens cover the same gene but in different strain backgrounds or different experiments, so the two labels constrain the same gene through two separate perturbation events. Morphology and expression share 1,440 strains; morphology and fitness share 4,220. Betaxanthin and the amino-acid pool are the sharing-strains case too, since both were measured on deletions of the same collection.

The prediction is that a second label helps more in the shared-strain case, because the model gets two views of one perturbation rather than two loosely-coupled views of one gene. **It has not been tested**, and the fitness-morphology pairing is the cheapest test of it available.

8 What to run next ✗

Ordered by what each would settle, under one governing decision: **expression is settled first, and morphology at full scale is deferred until it is.** Morphology is the larger change to Figure 3 and it is not research, but it is also close to four times the strains at a comparable architecture with an unmeasured epoch budget, so running it now would occupy the hardware for the strand whose diagnosis is least advanced.

8.1 Do now ✗

1. **Resume the best expression checkpoint and run until the curve turns.** No run has observed a maximum at any budget up to 10,000 epochs, so the saturation epoch, which sets every arm budget afterward, is still unknown. A fresh 10,000-epoch run costs 3.8 days; resuming the existing checkpoint is the cheap form of the same experiment. **Resume rather than restart, and use multi-GPU DDP** to shorten the wall clock, subject to what the IGB `mm1i` partition has free. The known risk is that Pearson is improving slowly enough near the peak that the turn may take a long time to become unambiguous; that is a reason to start it immediately rather than a reason not to.
2. **Stop symmetrizing the two directed relations** in `_build_attention_mask`. `TF → target` reaches 0.5508 on `tfink` and 0.5239 on regulatory interaction against controls near 0.501, and the builder currently averages that against its uninformative reverse, returning both to chance. This is the one graph-side repair with evidence behind it.

On the objection that attention is symmetric: the attention *mask* does not have to be. `head_mask[i,j]` and `head_mask[j,i]` are set together today, and writing them independently is a change to the mask builder rather than to the model. The attention scores themselves are already asymmetric, since $QK^\top$ is not symmetric; only the mask forces symmetry.

3. **Add unconstrained heads beside the masked ones, rather than freeing the masked ones.** The cross-phenotype probe found the graph channel does carry signal, unevenly by graph and by phenotype (table 29), so the masks stay. Keeping them preserves the guarantee that some heads attend along known biology; adding unconstrained heads gives the model somewhere to learn structure the graphs do not carry. Removing any particular masked head is a later decision, and `tflink` is the only current candidate for it.
4. **The small-target joint arms, betaxanthin and the amino-acid pool.** These train in hours rather than days and carry few output features, so the second-label question can be iterated there while an expression run occupies a GPU. Given section 5.4, the sharpest single experiment is to run the joint arm with replication and ask whether its held-out betaxanthin predictions absorb the 0.218 residual that betaxanthin-only training leaves on the table. That is one number, it reuses the existing residualization, and it is decisive where the unreplicated grid is not.

8.2 Deferred, and why ✗

1. **Morphology at full scale.** Drop `require_modalities` for the morphology-only arm and train on all 4,718 Ohya deletions instead of the 1,440 that also carry expression, or better, mask the loss over absent outputs so no strain is discarded. It remains the largest single change available to Figure 3, and it is a config change. It waits on expression.
2. **The scalar morphology warm-up, on A113 or C124_C.** When morphology comes off the shelf this goes first, since it calibrates the epoch budget for the 278-feature target at a fraction of the cost. Not cell size and not colony size: the size features are precisely measured and barely moved by deletions (table 14), and colony size is fitness.
3. **The post-hoc rescale on the best checkpoint.** It changes no correlation, so it cannot improve the strand’s headline number, and the expression work is not finished. Do it when a figure is drawn.

8.3 The two questions worth a GPU campaign ✗

Both are laid out with their epoch budgets, seed counts and slot costs in section 9, which is the operational version of this subsection. In summary:

- **Settle the objective first** (table 32). The pinball loss reaches its minimum at epoch 463 and rises for the next 9,500 while Pearson climbs, and the model ends up only level with the mean predictor in squared error until it is rescaled. A mechanism arm evaluated under an objective that is optimizing something else cannot separate “the mechanism does not help” from “the objective does not reward it”.
- **Then the pair term** (table 33). The reference operator provably cannot express a dependence on the pair (deleted gene, reporter gene), and masked unmasking does not supply one at the point the score is taken. A rank-64 pair term already reaches 0.2274 in 2.0 days against the masked objective’s 0.2362 in 3.8.
- **Pair morphology with fitness for the first full-scale joint test**, since fitness shares 4,220 strains with morphology against expression’s 1,440. Then run the expression pairing on the 1,440 as the mechanism test it was designed to be.

Substituting tyrosine for the 19 amino acids, as `igb_bx_aa.yaml` proposes, is now predicted to be the *null* arm rather than the live one, since tyrosine alone predicts betaxanthin at 0.064 against the profile’s 0.298. Keep it as a negative control, not as the headline contrast.

8.4 Structural options, further out ✗

Constraint-based coupling is the mechanism Figure 6 needs. The 023 result is that the coupling between amino-acid pools and betaxanthin is present in the data at $r = 0.298$ and that a plain auxiliary head does not use it, costing -0.0279 ± 0.0169 (table 22). That is a structural gap, and an unstructured second head is the wrong instrument to close it: it asks the encoder to discover a stoichiometric relationship from 19 noisy concentrations and one pigment readout, with nothing in the architecture saying the two are connected through a pathway.

A constraint-based layer says it. Penalizing predictions that violate a stoichiometric or flux balance over yeast-GEM makes the tyrosine-to-betalamic-acid route a property of the model rather than something it must infer, and it turns the metabolome head from an auxiliary target into a set of intermediates the constraint actually references. **This is the Figure 6 mechanism, and it is what makes the amino-acid panel a step in an argument instead of a negative result.**

Two facts size it rather than block it. yeast-GEM covers 19.4 % of the deletions in the screen and 27 % of its high producers (table 18), so the constraint is silent on most of the genome and must be a term that switches off outside the model rather than a hard requirement. And betaxanthin is already the strand closest to its ceiling, so the headroom the constraint is competing for is the smallest of any strand: 0.430 against 0.914. Expect it to earn its place by making the mechanism legible and by extending to targets with no screen at all, rather than by moving betaxanthin's number a lot.

The wider reason a metabolic layer is worth more than its betaxanthin delta. Once metabolism is represented explicitly, most of the phenotypes this project cares about route through one object rather than through a separate head each: metabolite production is the constraint's own output; gene-state readouts such as expression and proteome attach to it as the state the fluxes act on; global phenotypes such as fitness read off a class token; interactions read off subsets of gene representations; and chemogenomic response becomes an interaction between a compound and the network. **That is an argument about convenience and coverage rather than about accuracy on any one strand**, and it is the strongest form of the case, since no single strand's headroom justifies the work.

Labeled as a design argument, not a measurement. Nothing here has been built or tested, and the one adjacent measurement (table 20) says only that a real coupling exists for such a layer to exploit.

Not an expression rescue. The same layer is attractive for expression on the argument that metabolism constrains transcription. table 9 is the reason to be cautious: expression's binding constraint is 1,484 perturbations each seen once, and a metabolic prior does not add perturbations. Gate it on the same evidence standard the amino-acid coupling met, a measurable relationship in the data first.

Reifying the gene node into mRNA and protein. Not yet. The measured case for it is thin: per-gene mRNA-to-protein correlation across strains has a median of 0.08, a ridge map between the two recovers 2 to 3.5 % of held-out variance, and the two datasets were measured in different media, so the decoupling that would justify separate nodes is partly a condition artifact. Splitting the node multiplies parameters on an axis the data does not yet resolve. The case changes when a same-media proteome and expression pair exists, or when perturbations become fine-grained enough that the central-dogma collapse is what breaks.

Masked-label imputation as its own deliverable. The oracle reaches cross-study 0.4838 at $m = 1000$ against the model's best run of 0.238, and 97.5 to 100.6 % of that gain survives removing a genotype predictor, so it is a genuinely separate capability rather than a better version of the genotype task. **Something the group could offer on its own terms:** measure a subset of a strain's transcripts and get the rest, which is useful to someone who has an assay budget and no interest in our genotype model.

Two things make it more attractive than it looked. The expensive-sounding $m = 1000$ is not the operating point: 100 randomly chosen genes already reach 77 % of the $m = 1000$ cross-study score, and 10 reach 46 % (table 8). And which genes are chosen matters at the cheap end, where a variance-selected panel beats a random one and transfers across studies better. Deciding whether to ship it is a decision, not an experiment; the experiment worth running first is the ease-of-measurement panel selection (section 9).

8.5 What to do about Figure 3 and Figure 6 ✗

Figure 3. The manuscript currently claims $r = 0.543$ for expression and 0.619 for morphology inside a paragraph marked [FILLER - R3.]. The measured maxima are 0.238 and 0.082, and expression's replicate mean is 0.196 ± 0.022 (section 10). Neither placeholder should survive into a draft anyone reads. The honest version of the panel is the ceiling-relative one: expression at 25 % of a measured 0.775 on average and 31 % at its best draw, morphology at 13 % of a measured 0.611 *on a quarter of its data*, with the fitness panel carrying the strong result. **A 0.24 per-gene Pearson is not a Figure 3 headline.**

Whatever the campaign finds, the placeholders come out of the manuscript now. A number nobody measured, sitting in a Results paragraph, is the failure mode this whole document exists to prevent, and removing it does not depend on having a better number to put there.

The cheapest route to a better panel is not an architecture ladder. Morphology at full scale is a query change and remains the largest single change available to the figure, though it is deferred behind expression. And expression's own number is not yet an estimate anyone can defend: it is the best of eight draws with spread 0.0222, five of which peaked

by epoch 4,109. The replicated arms and the resumed run in section 10 are what turn 0.238 into either a defensible ceiling-relative claim or a higher number.

If the campaign comes back inside the noise floor, the panel should be built around what is solid rather than around a genotype-to-expression correlation: the ceiling-relative statement, the imputation capability with its measured gene budget, and the fitness panel that already works.

Figure 6. The betaxanthin case is the strongest thing in this document and it does not depend on beating anyone on accuracy. The argument is coverage: yeast-GEM reaches 19% of the screen and misses 73% of its high producers, and CGT finds high producers in the part a flux method cannot represent at $4.4\times$ over base rate.

The last panel is where the figure either becomes an argument or stays a list, and getting its *structure* right is the open work. The sequence that carries it: setup diagram; betaxanthin performance against its 0.914 ceiling; the coverage gap; the capability gap; the Flux Cone Learning comparison, framed as a tie on the genes both can score; and then the structural panel. That last one is the constraint-based layer above: the amino-acid coupling is real in the data and unused by a plain auxiliary head, and a stoichiometric constraint over yeast-GEM is what would close it. Shown that way the amino-acid material is the *motivation* for the mechanism rather than a failed joint-training result, which is the only framing in which a negative belongs in a figure.

Keep the option text in the draw.io boxes. Several of those panels name alternatives that have not been run, and a composition that still reads when one of them does not arrive is worth more than a tidier one that does not.

The Figure 3 panel most worth adding. Does adding fitness as a second label improve morphology prediction at all? It is a yes-or-no question with an existing correct control, it runs on 4,220 strains rather than 1,440, and either answer is publishable: a gain is the multi-task claim the figure wants, and a null is the honest counterweight to the one auxiliary-head experiment run to completion here, which came out negative. Draw the panel per CalMorph feature as well as averaged, since a second label that helps only the well-measured features would vanish in a mean.

9 The expression campaign ✗

The next campaign is **expression**, for Figure 3, with morphology deferred until expression is understood (section 8). What follows is sized in GPU-days rather than in arms, because the measured cost per run is what makes most possible designs unaffordable. The morphology sizing is kept at the end for when it comes off the shelf.

9.1 The budget arithmetic, measured ✗

run	epochs	s/epoch	wall clock	peak at
v9 M_fine (masked)	9,999	32.9	91.4 h = 3.81 d	epoch 9,674 (97%)
v8 V_basis64	4,106	42.0	47.9 h = 2.00 d	epoch 3,921 (95%)

Table 31: One expression run is a multi-day object. Both runs peaked in their last few percent, so neither bracketed its own maximum.

At ≈ 35 s/epoch, one GPU delivers $\approx 2,500$ epochs per day. So with G GPUs for D days the campaign has

$$\text{arm-slots} = \frac{G \cdot D \cdot 2,500}{E}$$

runs, where E is the per-arm epoch budget. **Every design decision below is a consequence of that fraction**, so it should be evaluated with the real G and D before anything is launched.

9.2 Set the epoch budget at 4,000, and the reason is measurable ✗

The previous round’s failure was scoring arms at 300 epochs against a curve that had not turned. The opposite failure is affordable only once: 10,000 epochs is 3.8 days per arm, which at four GPUs for four days buys four runs and no replicates.

$E = 4,000$ is the defensible middle, and not by taste. The v8 run **peaked at epoch 3,921** of 4,106, so a pair-rank arm has been observed to reach its maximum inside that budget. It costs ≈ 1.6 days per run, so four GPUs for four days buys ≈ 10 slots.

What $E = 4,000$ gives up, stated plainly. The v9 masked run was still climbing at 9,674 and reached 0.2362; truncating it at 4,000 would have scored it near 0.21. So a 4,000-epoch comparison is fair between arms and **understates** the absolute number by roughly 0.02 to 0.03. That is the right trade for an arm comparison and the wrong one for a headline figure, which is why the winning arm gets one long run afterward.

But the first move is not a 4,000-epoch arm comparison, it is one resumed run. $E = 4,000$ is a budget chosen without knowing where the curve turns, and it is a guess until something has been trained past its own maximum. **Resume the best existing checkpoint and keep going until the validation curve turns.** That costs no fresh warm-up, it answers the question every arm budget depends on, and it is the difference between choosing E from evidence and choosing it from an arm that happened to peak at 3,921.

Superseded in part by the launch plan. This subsection predates two measurements that change its arithmetic: the wall clock for 9,900 epochs is 3.5 days on either IGB card, inside the 5-day partition cap, and the identical-config spread at that budget is 0.0222, five of eight runs having already peaked by epoch 4,109. So the launch (section 10) runs every arm at the incumbent’s full $E = 9,900$ rather than truncating to 4,000, buys its slots by packing two runs per GPU rather than by shortening runs, and reads the resumed run as one draw rather than as the answer.

Two practical notes. Resuming from a checkpoint is strictly cheaper than the equivalent fresh run, so this is the low-cost form of the 10,000-epoch experiment rather than an additional one. And the wall clock is the binding constraint at 3.8 days for a single-GPU 10,000-epoch run, so **multi-GPU DDP is what makes the answer arrive in useful time**; check what the IGB `mmli` partition has free before sizing the rest of the campaign around a single-GPU rate.

9.3 Seeds: the noise floor is budget-dependent, and the short-budget figure was the wrong one $\times$

Expression’s across-initialization spread was measured at ≈ 0.006 ($n = 8$, fixed split) in wave 5, an order of magnitude below the betaxanthin strand’s 0.030 (table 28), and an earlier version of this plan concluded two seeds per arm from it. **That figure does not transfer to long budgets.** The eight identical-config runs at 9,900 epochs spread 0.1663 to 0.2382, a standard deviation of **0.0222** (section 10), $3.7\times$ the short-budget floor, and they differ in nothing but nondeterminism, not even the seed label. At $\sigma = 0.0222$, two seeds resolve only a ≈ 0.05 gap; the replicate arithmetic that sizes the launch is in table 35.

9.4 Phase A, the objective, and why it goes first $\times$

section 2.3 found that the pinball loss reaches its minimum at epoch 463 and rises for the next 9,500 while Pearson climbs, and that the model as trained sits only level with the mean predictor in squared error, at `nmse` ≈ 1.00 to 1.04, against a calibrated floor of 0.94. A mechanism arm evaluated under an objective that is optimizing something else cannot separate “the mechanism does not help” from “the objective does not reward it”, so the objective is settled first.

The head comparison on record is confounded with budget, and that has to be said first. Six distributional heads are implemented (`point`, `crps`, `quantile`, `laplace_crps`, `nll`, `energy`), and the incumbent choice rests on a comparison that is not one.

dist head	all runs			matched budget, 100–200 epochs	
	runs	best	max epochs	best	median
<code>quantile</code>	344	0.2382	9,900	0.1751	0.1352
<code>crps</code>	158	0.1631	199	0.1631	0.1267
<code>energy</code>	50	0.1421	199	0.1421	0.0960
<code>laplace_crps</code>	56	0.1411	199	0.1411	0.0775
<code>point</code>	34	0.1117	163	0.1117	0.1054
<code>nll</code>	59	0.0983	199	0.0845	0.0680

Table 32: `quantile` is the only head ever trained past 200 epochs, so the left block compares one head at 9,900 epochs against five at roughly 150. That is the same error this document indicts everywhere else. The right block is the honest comparison, on the window where all six ran: `quantile` still leads, but by $+0.0085$ on the median against a short-budget seed floor of ≈ 0.006 and a long-budget replicate spread of 0.0222 (section 10), and its maximum is drawn from 84 runs against `crps`’s 25, so more of its lead is selection inflation. It is also an ordering measured at 150 epochs, which section 2 shows is a regime of transients.

What the incumbent actually optimizes, and what the metric actually reads. The head emits 19 `quantile` knots per feature, `pinball` scores all 19, and `pearson_per_feature` reads the median knot alone (section 1.1). So

18 of 19 outputs per feature are optimized against a quantity the metric never inspects. The calibration diagnostics say that spend is not being recovered: at epoch 10,000 the run reads `coverage_80` = 0.578 against a target of 0.80 and `coverage_50` = 0.326 against 0.50, so the predictive intervals are far too narrow, while its point estimates are simultaneously *over*-dispersed at $s/r = 1.92$. Two calibration failures in opposite directions.

Hypothesis, untested: mean pinball over an even τ grid approximates CRPS, which is dominated by its spread term, so most of the gradient goes into distribution shape that the metric discards, and the validation pinball rising after epoch 463 while Pearson climbs is the model overfitting that shape.

Two heads are ruled out before spending a GPU. `n11` is the designed negative control for σ -collapse and is already last on both columns. `energy` is more interesting and still out: its covariance $\Sigma = D + VV^\top$ is *global*, one matrix shared by every strain, so it changes the predictive distribution but does not enter μ and therefore **cannot move a point metric like `pearson_per_feature`**. Its rank-32 factor is tempting against the measured effective rank of 32.78 in the residual covariance, but that structure lives in the spread, not the location.

Correction to an earlier version of this plan. It proposed an `0_zmse` arm described as turning on `standardize_per_feature_target`. That lever is **already on**: it is set in `cgt_expr_008`, which the whole v9 chain inherits, and the incumbent run reports `target_norm` = `zscore`. So the arm is not “z-score the targets”, which is done; it is **switch the head to point**, one flag, putting every output parameter on the location the metric reads.

On `0_corr`, and the fair objection that nobody optimizes correlation. Correlation is almost always a reporting metric rather than a training loss in the published literature, and that is a reason for caution rather than a reason to skip the arm. Why it is rarely done: a batch-level Pearson is not a sum over examples, so it is sensitive to batch composition and batch size in a way squared error is not, and it is invariant to scale, so it constrains only the ordering and leaves magnitudes free, which is exactly the under-shrinkage this document measures. **So `0_corr` risks buying the calibration defect deliberately.**

That makes `0_zmse` the better-motivated arm of the two and the one to run first: it targets the same failure (“predict the mean” scoring zero rather than winning), it is already implemented as `standardize_per_feature_target`, and it keeps a per-example loss. Keep `0_corr` as the third arm, run it with the calibration diagnostics below, and read a Pearson gain from it as provisional until `nmse` is checked alongside.

Report `nmse` and the SD ratio on every arm, not just Pearson. The calibration identity in section 2.3 means an arm can raise Pearson while leaving `nmse` above 1, and that is a different result from one that fixes both. Both numbers are already logged, and table 5 shows why reading them together matters: which side of 1 the current model lands on depends on the smoothing rule.

The post-hoc rescale is not part of this campaign. Applying r/s to the best checkpoint changes no correlation, so it cannot move the strand’s headline number and it settles nothing the campaign is asking. Do it when a figure is drawn, at which point it removes the “loses to the mean” objection for free.

9.5 Phase B, the pair term, under the winning objective ✗

The reference operator provably cannot express any dependence on the pair (deleted gene, reporter gene), and masked unmasking does not supply one at $k = 0$. The ladder is the mechanism axis.

arm	pair rank	note
<code>P_ref</code>	0	additive reference
<code>P_basis64</code>	64	the current best-per-compute: 0.2274 in 2.0 days
<code>P_film</code>	90	diagonal multiplicative, identity at init
<code>P_asym</code>	–	the two directed relations no longer symmetrized in the mask builder, plus unconstrained heads added beside the masked ones

Table 33: Phase B. Four arms, two seeds: eight slots, ≈ 13 GPU-days. The originally planned `P_graph` arm, which would have made the pair term a function of network distance, is dropped: the probe found the network does not carry that relationship on expression (section 7.7). `P_asym` replaces it, costs no new mechanism, and unlike freeing the masked heads it does not discard the graph channel on the strength of a single phenotype’s null.

The gate on `P_graph` is narrower than it looked, and the graph channel stays. The probe has run on all six strands (section 7.7). Graph proximity does not predict which individual *reporters* respond to a deletion, so **`P_graph` as**

specified is still not worth building: a pair term that is a function of network distance to the responding reporter rests on the one relationship measured to be absent. Drop it from Phase B.

But the graph channel itself carries information, and revision 2's proposal to free the nine masked heads is withdrawn. Deletions of adjacent genes give measurably similar phenotypes, up to AUC 0.6579 against a 0.4971 control, and which graph carries that differs by phenotype (table 29). The Phase B arm is therefore *additive*: keep the masked heads, **add unconstrained heads beside them**, and **stop symmetrizing the two directed relations** in `_build_attention_mask`, where $TF \rightarrow target$ reaches 0.5508 on `tflink` against a 0.5017 control and the builder currently averages that against its uninformative reverse.

The one graph a removal could be justified on today is `tflink`, which is at the noise floor on all six strands in the pair form and carries only the directional reporter-form signal that the symmetric mask currently discards. **Fix the symmetrization before concluding anything about that head**, since the fix is precisely what would make it useful.

9.6 Morphology, deferred, sized for when it starts ✗

Morphology is not a smaller copy of expression. Its target is 278-dimensional rather than 6,127, no morphology run has ever passed 121 epochs, and every one of the 397 runs across eight projects trained on 1,161 strains (table 11) out of a 4,718-strain screen.

Why it is deferred rather than run alongside. It is close to four times the strains at a comparable architecture, its per-epoch cost has never been measured, and the only figure on record is a tiny-model smoke at ≈ 47 s/epoch against expression's ≈ 15 in the same smoke. If that ordering survives at full size, morphology is the *more* expensive of the two despite the smaller target, and it would be bought with the hardware that is currently answering the better-posed question.

When it starts, in this order:

1. **The scalar warm-up first**, on A113 (`actin_n_ratio`, ceiling 0.873, robust CV 2.37) or C124_C. It calibrates the epoch budget for the 278-feature target at a fraction of the cost, and the budget is the one thing not knowable in advance. Treat it as a calibration, not a result.
2. **Measure the per-epoch cost on that first run**, so the slot arithmetic above applies to morphology at all.
3. **Then morphology at full scale**, either by dropping `require_modalities` and rebuilding on all 4,718 Ohya deletions, or better by masking the loss over absent outputs so no strain is discarded for lacking a second phenotype. This is what turns 0.0824 into a statement about morphology rather than about a quarter of it.

9.7 The joint question, which is the one Figure 3 is about ✗

Does expression help morphology. The control already exists and is correct: `delta_joint_expr_morph_000.yaml` pins all three conditions to the 1,440 genotypes carrying both phenotypes, so `expr_morph` minus `morph` isolates the auxiliary head from a data-quantity difference. It has never been run past 87 epochs.

Ask the fitness version of the question first, and ask it plainly. Does adding fitness as a second label improve morphology prediction at all? Paired with fitness rather than expression the joint arm keeps 4,220 of the 4,718 strains, three times the instance count, which is where the statistical power is. Report the answer per CalMorph feature as well as averaged over features: a second label that helps the 201 well-measured features and does nothing for the rest would be invisible in the mean, and that is a different finding from no effect.

The expression pairing on the 1,440 shared strains remains the mechanism test `delta_joint_expr_morph_000.yaml` was designed for, and it stays worth a slot. The two answer different questions; the fitness pairing is the one likely to produce a detectable effect. Both come after single-phenotype morphology, since a joint result is uninterpretable without a morphology-only arm run to the same budget.

The prior from the metabolite strands is not encouraging, and should be said out loud. The one auxiliary-head experiment run to completion anywhere in this project is the metabolome head on betaxanthin, and it reads -0.0279 ± 0.0169 (table 22). A second phenotype sharing an encoder has not yet been shown to help any first phenotype here. The expression-morphology pairing has a better prior than that one, because the two phenotypes share strains rather than merely genes, but the campaign should be designed to **measure** the effect rather than to confirm it, which means the control arm matters more than the joint arm.

9.8 What the campaign cannot fix ✗

Expression sits at 0.196 ± 0.022 , best run 0.238, against a ceiling of 0.775, and the perturbation axis has 1,484 points with each gene deleted exactly once (table 9). No objective and no pair term changes that. If Phase A and Phase B both come back inside the noise floor, the honest reading is that the binding constraint is the data rather than the model, and the figure’s expression panel should be built around the ceiling-relative statement and the imputation capability rather than around a genotype-to-expression number.

And if it does become an imputation figure, the question becomes how small the measured panel can be. That is a well-posed question with a partial answer already: table 8 shows 100 randomly chosen genes reach 77 % of what 1,000 reach across studies, and choosing genes by across-strain residual variance beats random at the cheap end. The version that would matter is selection by **ease of measurement**, since a panel that is cheap to assay by mass spectrometry and still spans the residual structure is a deployable object rather than a curiosity. That was not run, because no measurement-cost annotation exists in the repository and inventing one would have been guessing. Building that annotation is the prerequisite, and it is a data task rather than a modeling one.

10 The launch, 2026-08-31: what runs next and why ✗

The near-term strategy is one round on IGB: settle the objective question of section 9 Phase A with replicated arms at the incumbent’s full budget, while the incumbent’s own checkpoint resumes past 9,900 epochs. It uses six GPUs, all four idle mmli A100s and two of cabbi’s five free RTX 6000 Ada, at two runs per GPU, under the administration’s 5-day-per-job limit, with no job chaining. Every design choice below is stated with the measurement that forces it, and the round is deliberately smaller than the hardware allows, because table 35 shows the extra runs would buy resolution the question cannot use.

10.1 Evidence 1: the replicate spread at 9,900 epochs is 0.0222 ✗

Every expression run past 9,000 epochs on the leaderboard, eight of them, carries one identical configuration: **quantile** head trained by pinball loss (section 1.1: the head is what emits the 19 knots, pinball is what scores them), lr 3×10^{-4} , dropout 0.1, $L = 6$, hidden 90, mask prior, **s1_pool**, seed 0. The script asserts this before computing anything, so the spread below is run-to-run nondeterminism and nothing else.

statistic	value
scores, sorted	0.1663, 0.1804, 0.1824, 0.1887, 0.2010, 0.2057, 0.2091, 0.2382
mean $\pm$ sd	0.1965 ± 0.0222
expected maximum of 8 draws (Blom)	0.2282
observed maximum	0.2382
peak epochs, sorted	2199, 3398, 3503, 3715, 4109, 8859, 9691, 9697

Table 34: The eight identical-config expression runs at 9,900 epochs. The headline 0.2382 is the maximum of these draws; the config’s central estimate is 0.196 ± 0.022 . Five of eight peaked by epoch 4,109 and never exceeded that value in the following six thousand epochs, so budget raises the maximum draw rather than every draw.

10.2 Evidence 2: what that spread costs in replicates ✗

two fresh arms		one fresh arm vs the $n = 8$ baseline	
gap to detect	runs per arm	runs in the arm	smallest detectable gap
0.02	19.3	2	0.049
0.03	8.6	3	0.042
0.04	4.8	6	0.034
0.05	3.1	7	0.032

Table 35: Replicate arithmetic at $\sigma = 0.0222$, 80 % power, $\alpha = 0.05$. The left block is the symmetric two-arm design; the right block is the design used here, where each challenger head is compared against the eight quantile replicates that already exist. The matched-budget head gaps on record (+0.0085 on the median, table 32) sit below every row of this table, which is the arithmetic demonstration that the previous round’s one-run-per-arm scoring could not have resolved the question it was asking.

The consequence that shapes everything: **re-running quantile is wasted budget**, because its $n = 8$ baseline already exists at this exact budget and config, and every fresh slot spent on a challenger tightens the challenger side of an already-anchored comparison.

10.3 Evidence 3: wall clock, and the packing cost measured $\times$

From the same eight runs, the two IGB card classes separate cleanly on compute time and not at all on wall clock: RTX 6000 Ada 17.2 s/epoch compute against the A100’s 27.0, but 31.0 against 30.5 s/epoch wall, because validation and checkpointing overhead dominates. So cabbi and mmli deliver the same epochs per day, and the partition split can be chosen on availability alone. At those rates 9,900 epochs takes 3.5 days solo on either card, inside the 5-day cap.

Packing was then measured directly rather than assumed, on two idle GilaHyper 46 GB cards: the incumbent config resumed from the 1445 checkpoint with `train_eval_every=0`, once solo and then two concurrent runs on one card, ten epochs each at steady state. SRC: `experiments/019-simb-multimodal/scripts/gh\_packing\_benchmark.sh`

	solo	two packed on one GPU
median s/epoch, per run	17.0	26.0
per-run slowdown		1.53 $\times$
aggregate throughput		1.31 $\times$
VRAM, both runs together	19,379 MiB = 9.5 GiB per run	

Table 36: The packing benchmark. Memory is nowhere near binding, at 9.5 GiB per run on cards of 46 to 80 GiB; the whole cost of packing is the 1.53 $\times$ per-run slowdown. Extrapolated to the IGB wall-clock rates, which is an extrapolation from a different machine and an eval-free configuration and is labeled as such, a packed run covers $\approx 9,100$ to 9,250 epochs in 5 days, 7 % short of the 9,900 target.

Two runs per GPU is the chosen operating point: it doubles the replicate count, which table 35 shows is the binding resource, at the price of arms reaching $\approx 9,100$ rather than 9,900 epochs before the wall. The scoring rule below absorbs that shortfall.

10.4 The allocation: 10 GPU slots and four CPU baselines $\times$

Sized against the partitions as measured on 2026-08-31: `compute-5-7` (mmli) is IDLE with all four A100s free, and `compute-3-3` (cabbi) reports `Gres=gpu:RTX6000:8` against `AllocTRES=gres/gpu=3`, so five of its eight cards are free. The other cabbi jobs queued at the time hold no GPU at all (16 CPUs, no `gres/gpu`), so they do not compete for cards.

partition	job	GPUs	runs	contents
mmli (A100)	2368333 array 0-3	4	8	<code>crps</code> $\times$ 3, <code>laplace_crps</code> $\times$ 3, <code>point</code> seeds 0-1
cabbi (Ada)	2368337 array 4	1	1	<code>point</code> seed 2
cabbi (Ada)	2368339	1	1	resume of the incumbent, <code>max_epochs</code> 19,000
CPU, no GPU	<code>expression_baselines.py</code>		4	B0-B3, ran first (table 39)

Table 37: The launch AS SUBMITTED, 2026-08-31, source `e2deb5dc` with a clean worktree. 10 runs on 6 GPUs at two per GPU, everything but the head pinned to the incumbent configuration, 5-day walltime each and no chaining. Nine fresh runs across three challenger heads at three replicates each, scored against the existing $n = 8$ **quantile** baseline, plus the resumed incumbent. **Three of cabbi’s five free cards are left alone.** Two deviations from the plan as written, both forced and neither affecting the comparison: the launcher packs runs seed-major, so `point` seed 2 lands on cabbi while its other two replicates are on mmli, the only arm split across partitions; and the resume runs on cabbi rather than mmli because the four A100s are fully committed to the arms. The resume on an Ada card is arguably the closer match anyway, since the checkpoint it continues was itself trained on an Ada GPU.

Why three replicates and not seven, which is a concession the power table forces. An earlier version of this plan spent every free GPU, at six to seven replicates per arm. table 35 does not support that. The gap actually on the table is the matched-budget head difference of +0.0085 (table 32), and at $\sigma = 0.0222$ resolving it needs on the order of fifty replicates per arm. **No affordable design resolves the real gap, so the round can only detect a large effect**, and for that three replicates against the $n = 8$ baseline gives 0.042 where seven gives 0.032. That last 0.010 is precision the question cannot use, and it costs eight GPUs. The honest statement of what this round can return is therefore: a challenger head is either dramatically different from **quantile**, or the round reports that the heads are indistinguishable at the budget anyone can afford, which is itself the answer to whether the incumbent choice was ever load-bearing.

Everything in flight, on one page. Two rounds are running, plus the baselines that gated them. Each row states what varies and what a null result would mean, because an arm whose null is uninterpretable should not have been launched.

round	arms	what varies	what a null means
baselines	B0–B3	no model: mean, no-change, bilinear, neighbor	<i>ran first, cleared:</i> B2 0.1040 vs 0.1965
objective	crps, laplace_crps, point	the distributional HEAD, 3 replicates each, everything else pinned	the head choice was never load-bearing, and quantile can be kept without argument
mechanism	R_ref, R_basis64, R_pergene, R_pergene_basis64	the pair term and the READOUT, 2 replicates each, in-round reference	the additive collapse is not what binds, and the ceiling gap is data rather than architecture

Table 38: The three rounds. **Only the objective round can settle its question;** the mechanism round is a screen, resolving ≈ 0.062 at two replicates against its own reference, so it can find a large readout effect and nothing subtler. Both are scored the same way and against the same $n = 8$ **quantile** baseline, so a positive result in either is directly comparable to the incumbent’s 0.1965 ± 0.0222 .

The baselines run before the arms, and one of them is a stop condition. After section 10.8, a deep-learning number reported without these is not interpretable. They are cheap, they need no GPU, and **they are a prerequisite rather than a companion:** if B2 matches or beats the incumbent’s 0.196 ± 0.022 , the finding of this round is that a bilinear model suffices and the head comparison is the wrong question. That is exactly what Ahlmann-Eltze et al. found for GEARS, so it is a live possibility rather than a formality.

	prediction	what it isolates	pf	nmse
B0	each gene’s training mean	the definitional floor	0.0000	1.0000
B1	no change, $\log_2$ ratio = 0	“nothing happened”	0.0000	1.3784
B2	$\mathbf{GWP}^\top + \mathbf{b}$, ridge, rank-swept	a pair term without deep learning	0.1040	–
B3	mean of the k nearest deleted genes in embedding space	genotype-side signal from proximity alone	0.0908	–
<i>incumbent, $n = 8$ identical-config runs</i>			0.1965	

Table 39: The four baselines, MEASURED 2026-08-31, **pf** = **pearson_per_feature**. B2 and B3 are best over four gene embeddings, both won by **prot_T5_all**. **B0 and B1 are definitionally zero**, because a prediction constant across strains has no variance to correlate, and B0’s **nmse** of exactly 1.0000 is the quantity **nmse** divides by: the script raises unless both land there, so they are the self-check that the data path is right rather than competitors. **The gate clears.** The incumbent’s 0.1965 ± 0.0222 beats B2 by 0.093, about 4.2σ , so a bilinear model does not suffice here and the head comparison is a real question. That is the opposite of what Ahlmann-Eltze et al. found for GEARS, and it is the strongest single argument that this round is worth its GPUs. SRC: `experiments/019-simb-multimodal/scripts/expression\_baselines.py`

Every deleted gene appears exactly once, which forced the baseline design and is worth stating on its own. The loader returns 1,482 single-deletion strains carrying **1,482 distinct deleted genes**, so no gene is in both the fit and the validation split and **every validation strain is an unseen perturbation** (measured: an in-data perturbation representation covers 0 of 155). B2 and B3 therefore cannot represent a perturbation by its own observed response and must use an external gene embedding, which is exactly the regime Ahlmann-Eltze et al. benchmark. One qualification: B3 here reads 0.0908 where section 2 quotes 0.117 from `knn_embedding_probe.py`; the constructions differ (residual space, cosine on standardized embeddings), so this is a companion measurement rather than a reproduction of that number.

Justification, arm by arm.

Why these three heads and no others **nll** is the designed σ -collapse negative control and is already last at every budget; **energy**’s covariance $\Sigma = D + VV^\top$ is global and does not enter μ , so it cannot move a point metric (section 9); **quantile** needs no fresh runs because its baseline is the eight replicates of table 34. That leaves exactly the three heads whose point estimates could differ: **crps** (Gaussian μ), **laplace_crps** (Laplace median, structurally the closest analog to the quantile head’s median knot), and **point** (all parameters on the location on the metric reads, which is the corrected **0_zmse** arm of section 9, since per-feature target z-scoring is already on).

Why every arm gets three, with none favored **crps** is the strongest challenger on the matched-budget record (0.1631 best, 0.1267 median against **quantile**’s 0.1751 and 0.1352), which is an argument for resolving it more tightly. The power table refuses that argument: the extra replicates would move the detectable gap from 0.042

to 0.032 while the gap being chased is 0.0085. Three each, equally, is the design that admits what the noise floor permits.

Why full budget rather than 4,000 epochs The comparison target is the incumbent at its own budget. Truncated arms would re-introduce the exact confound table 32 documents, and the measured wall clock makes full budget affordable, so nothing is bought by truncating.

Why the continuation, and why only one The two best replicates peaked in their last 12%, so whether the top draw keeps climbing past 9,900 is open, and resuming the existing checkpoint is the only way to see past it without paying for the first 9,900 again. It is one draw with $\sigma = 0.0222$, so it cannot establish saturation by itself; it can only show whether *this* draw’s curve turns. Budget: packed on an A100 at the extrapolated 46.7 s/epoch, five days adds $\approx 9,200$ epochs, so `max_epochs` = 19,000. It runs on mmli, and it is a resume of the checkpoint rather than a chained job.

Why seeds vary when the baseline’s did not The eight baseline runs spread 0.0222 at a *fixed* seed label, so run-to-run nondeterminism alone produces the measured floor. Varying the seed across replicates adds initialization variance on top, making each arm’s spread an honest upper-bound match to what any future user of the winning head would see.

The scoring rule, declared before launch Each arm and the baseline are scored as `roll_max` over epochs $\leq 9,000$, the largest budget every packed run reaches before the 5-day wall, with the full-history `roll_max` reported alongside. `nmse`, the SD ratio s , and the coverage diagnostics are reported per arm as section 9 requires, so a head that moves Pearson while leaving calibration broken is visible as such. No arm is compared on a raw per-epoch maximum. **Every arm is additionally reported against B0–B3 (table 39) and in `pearson_per_instance` as well as `pearson_per_feature`**, the first so the round has a floor, the second so it has a number commensurable with the published models (section 10.8).

Operational constraints honored 5 days per job per the administrator email, with a continuation counted as a fresh 5-day job rather than a chain; two of cabbi’s five free GPUs, leaving three; all compute on partition nodes with the login node used only for `sbatch`, `rsync`, and `wandb sync`.

10.5 The model being launched: 1,194,509 parameters ✗

The incumbent checkpoint’s state dict reconciles exactly to the logged `total_param_count` once two artifacts are removed: 98,370 parameters that appear twice because `EquivariantPerturbationTransform` aliases layer 0 of its module lists (`self.cross_attn=self.cross_attn_layers[0]` and siblings), and 12,273 non-trainable buffers (the two per-gene normalization vectors and the 19 quantile knot positions).

module	parameters	role
<code>transformer_layers</code>	590,220	6 graph-masked encoder layers
<code>embedding_preprocessor</code>	369,639	input projection to $d = 90$
<code>post_perturbation_mixing</code>	101,430	Perceiver mixing after the perturbation
<code>perturbation_transform</code>	98,370	the cross-attention perturbation operator
<code>perturbation_head</code>	16,381	graph-level scalar head
<code>per_gene_head</code>	9,919	per-gene readout, $d \rightarrow 19$ quantile knots
<code>observed_label_encoder</code>	8,460	masked teacher-forcing encoder
<code>cls_token</code>	90	
total trainable	1,194,509	matches the logged <code>total_param_count</code>

Table 40: Parameter budget of the incumbent model. The striking row is the readout: 6,127 reporter genes are predicted by 9,919 parameters, because `PerGeneHead` is one MLP shared across every gene token, emitting that gene’s 19 quantile knots. The equivariant weight sharing is what makes the output dimension nearly free, and it is also why the perturbation operator, at 8% of the parameters, carries the entire burden of making predictions strain-specific: the encoder runs once on the wild-type graph, so everything downstream of `perturbation_transform` is the only place strain identity enters.

10.6 Graph attention, as currently built ✗

For reading the arms above against the architecture, the current facts, all verified in code or measured in section 7:

- The nine interaction graphs enter as a **hard additive attention mask** on the encoder’s self-attention scores (`masked_fill` to $-\infty$ per head), not as edge features; composing $L = 6$ layers makes influence approximately 6-step random-walk reachability (section 7.7).

- `_build_attention_mask` still **symmetrizes the two directed relations**, discarding the measured $\text{TF} \rightarrow \text{target}$ signal (0.5508 against 0.5017 rewired). The fix is queued in Phase B, not in this launch, which holds the architecture fixed by design.
- The pair-form probe found the graph channel carries real signal on expression, up to AUC 0.6579 on STRING database (table 29), so the masks stay.
- The perturbation enters *after* the encoder, through a cross-attention whose keys are the perturbed genes, and at a single deletion that softmax runs over one key and degenerates to $g(h_i + c_b)$: no term depends on the pair (deleted gene, reporter gene) (section 2).

10.7 Perturbation encodings that supply one-to-many, in this attention style ✗

The question the degeneracy raises: are there perturbation encodings, compatible with a graph-masked transformer over gene tokens, in which *one* perturbed gene produces *gene-specific* effects across all 6,127 reporters. Yes, on two tiers.

Already implemented in `equivariant_cell_graph_transformer.py`, each gated to be the exact identity at initialization. Ordered by the rank of the pair term they add:

mechanism	pair rank	how one deletion reaches many genes
additive reference	0	it does not: one c_b added to every gene
null sink	9	sigmoid gate per head, α_i depends on the querying gene
graph propagation	data	random-walk mass from the deleted gene over the nine graphs, plus a hop-0 self indicator
low-rank bilinear	r	$\sum_j b_{ij}(h_i) a_j(c_b)$
response basis (<code>P_basis64</code>)	64	measured at 0.2274 in 2.0 days
Hadamard / FiLM	90	$h_i \odot (1 + \gamma(c_b))$, gene and strain multiply

Table 41: In-repo one-to-many perturbation mechanisms, all resident in the model file and selectable by config. These are the Phase B arms of table 33; none is in this launch, which settles the objective first so mechanism deltas are measured under a settled loss.

The published designs, mapped onto this architecture. These are literature descriptions, not measurements made here.

- **GEARS** (Roohani et al., 2024) propagates a learned perturbation embedding over a relation graph to produce per-gene deltas. `PerturbationGraphPropagation` is this mechanism with fixed random-walk features in place of learned message passing, so the cheap version is already an arm; the learned version is the upgrade if the cheap one moves anything.
- **Token-edit conditioning** (Geneformer’s in-silico deletion, Theodoris et al. 2023; scGPT’s perturbation flag, Cui et al. 2024; STATE-style perturbation tokens) puts the perturbation *into the encoder’s input*, as removal of the gene’s token or a condition embedding added to it, so the encoder’s own self-attention, here graph-masked, carries the deletion outward and the one-key degeneracy never arises: every gene attends to the perturbed gene among 6,607 tokens, not to a key set of size one. This is the design that most directly “fits into our attention style”. Its cost is structural: the current model runs the encoder **once** on the wild-type graph and reuses it for every strain, which is precisely why the perturbation had to enter post hoc; a token edit forces one encoder pass per strain, multiplying epoch cost by roughly the batch size. **Hypothesis, untested:** at 39 batches per epoch and 17 to 31 s/epoch, that overhead is a wall-clock multiplier this campaign cannot absorb, but a 2-layer strain-specific encoder atop frozen wild-type layer outputs could cap it.
- **CPA-style additive latent composition** (Lotfollahi et al., 2023) is the same rank-0 additive structure as the reference operator, with gene specificity living entirely in the decoder. It is what the model already is, and its limitation on the pair axis is the one section 2 proves.

The ordering stands as in section 9: objective first, with this launch; the mechanism tier second, under the winning objective, with the in-repo arms before any encoder restructuring.

10.8 What the perturbation-response literature says, and what it changes here ✗

Three papers were read in full against this plan, all mirrored and `sha256`-pinned. They are named inline with their DOI, following this document’s convention. The reason to read them together is that they converge on one uncomfortable point.

Every published operator in this family is additive, including both of Roohani’s. GEARs (Roohani et al. 2023, doi:10.1038/s41587-023-01905-6) combines a perturbation with a gene, and State (Adduri et al. 2025, doi:10.1101/2025.06.26.661135), by the same senior author two years later, forms its transformer input, as

$$h_u^{\text{post-pert}} = \text{MLP}(h_u^{\text{gene}} + h^{\mathcal{P}}), \quad \mathbf{H} = \mathbf{H}_{\text{cell}} + \mathbf{H}_{\text{pert}} + \mathbf{H}_{\text{batch}}.$$

GEARs composes multiple perturbations by summing their embeddings before an MLP. Neither paper ablates addition against a multiplicative or cross-attentional alternative. **So the additive operator this document indicts is not a torchcell defect; it is the field’s default.** That reframes section 2’s structural finding from a bug report into a statement about the whole model class, which is a stronger result and should be written as one.

Where they differ from us is the readout, and that is the actionable gap. Both put gene identity in *per-gene output parameters* applied to a shared nonlinear state. GEARs gives every gene its own linear layer, “for every gene u , we apply a gene-specific linear layer parameterized by $w_u \in \mathbb{R}^d, b_u \in \mathbb{R}$ ”; State’s readout is a single $\mathbf{W}_{\text{recon}} \in \mathbb{R}^{d_h \times G}$, one column per gene.

	tokens	perturbation enters	per-gene readout
GEARs	genes	$\text{MLP}(h_u + h^{\mathcal{P}})$	yes , $w_u \in \mathbb{R}^d$
State	cells	$\mathbf{H}_{\text{cell}} + \mathbf{H}_{\text{pert}}$	yes , $\mathbf{W}_{\text{recon}} \in \mathbb{R}^{d_h \times G}$
torchcell v9	genes	$g(h_i + c_b)$	no , one shared MLP

Table 42: Where gene specificity lives. Ours composes to $F(h_i + c_b)$ with a single shared F , so reporter identity enters only through h_i , inside the same function the perturbation enters. A GEARs-style per-gene output weight would cost $6,127 \times 91 = 557,557$ parameters here, +47% on the current model, and is a mechanism rather than capacity. It is the best-supported new arm for the mechanism tier.

The benchmark that should govern how this campaign is read. Ahlmann-Eltze, Huber and Anders 2025 (doi:10.1038/s41592-025-02772-6) benchmarked five foundation models and two purpose-built ones, GEARs and CPA, against deliberately simple baselines: “None outperformed the baselines.” On unseen single perturbations, across the Replogle and Adamson data, “None of the deep learning models was able to consistently outperform the mean prediction or the linear model.” On double perturbations every model was substantially worse than an additive baseline that simply sums the two single-perturbation log fold changes, and on genetic-interaction calls “None of the models was better than the ‘no change’ baseline.” Their linear model is

$$\underset{\mathbf{W}}{\text{argmin}} \quad \|\mathbf{Y}_{\text{train}} - (\mathbf{GWP}^{\top} + \mathbf{b})\|_2^2,$$

a rank-limited bilinear pair term plus a per-gene intercept, and it beat GEARs while *using GEARs’s own perturbation embeddings*. Several concurrent benchmarks are reported to agree. ●[via Ahlmann-Eltze et al.’s reference list, not primary] Kernfeld et al. (doi:10.1101/2023.07.28.551039) and Csentesi et al. 2025 (*BMC Genomics* 26, 393) are named there as reaching the same conclusion; neither is in our mirror and neither was read.

State reports the same thing about itself where its perturbations are genetic rather than chemical: “On genetic perturbation datasets, State matched the performance of the perturbation mean baseline”, with a context-mean baseline beating it on fold-change prediction by 10%. That is a 600M-parameter encoder trained on 167 million cells, achieving parity with predicting a mean.

This is direct external support for two of this document’s conclusions, and it should be said plainly. First, **scale is not the lever.** section 10 argues that from our own matched-budget evidence, where 145 runs spanning 0.12M to 4.7M parameters show `pearson(log10params, score) = -0.129`; the literature reaches the same place from the opposite direction, at 500 times the scale and in a different organism. Second, **the mean-baseline failure mode is the field’s central problem, not our local one.** Our `nmse` ≈ 1.00 is precisely the “no better than the mean” result these benchmarks report, and it is now clear that reporting it was not a confession but the standard finding stated honestly.

One asymmetry in our favor, which the metric choice earns. A baseline predicting each gene’s mean across strains scores *zero* on `pearson_per_feature`, because a constant prediction has no variance across strains to correlate.

The Pearson these papers report is computed the other way, across genes within one perturbation, where the enormous gene-to-gene variation is shared by every prediction and a mean baseline therefore scores high. **So our 0.196 and their 0.556 are not comparable, and ours is the harder statistic.** Any comparison drawn against GEARS or State must use `pearson_per_instance`, or it is not a comparison. This also resolves the apparent tension with `nmse` ≈ 1 : the model orders strains within a gene better than chance while its magnitudes are no better than the mean in squared error, which is the under-shrinkage of section 2.3 and not a contradiction.

What is not portable, checked rather than assumed. State’s actual contribution is not its operator but its set machinery: bidirectional self-attention across sets of 256 cells, trained with an MMD under the energy kernel $k(u, v) = -\|u - v\|_2$. Both require a *population* of measurements per condition. We have one bulk profile per deletion strain, so at $S = 1$ that MMD reduces to $2\|\hat{x} - x\|_2$, an ordinary per-sample regression loss with no distributional content, and cross-cell attention has nothing to attend to. The expression ceiling reinforces this: it rests on 82 cross-study shared deletions, not on per-strain replicates. **This rules out the single most effective idea in that paper for our data, which is worth knowing before anyone proposes it.** It is also a concrete argument for replicate measurement in any future screen we design.

Consequences for the plan, in priority order.

1. **Add the baselines to the launch scoring, not to a later round.** *Done:* B0–B3 are in table 39 and run before the arms, with B2 carrying a stop condition on the round.
2. **Report `pearson_per_instance` beside `pearson_per_feature` on every arm,** so the strand has a number that is commensurable with the literature. *Done:* folded into the declared scoring rule above.
3. **Promote the per-gene readout to the first mechanism arm,** ahead of the rank-64 basis term. It is what both published models have and we lack, and table 42 gives its cost.
4. **Reframe the structural finding as a claim about the model class,** since the additive collapse is shared by every published operator in this family.

2026-09-01: the round was relaunched after a dataloader defect, and the job identifiers above no longer resolve. table 37 records the launch as submitted and is left unedited. Every job in it is gone. Two cabbi runs died roughly a day into their five-day budgets, from one defect wearing two disguises:

```
2368337, epoch 2,580: RuntimeError: received 0 items of ancdata
2368339, epoch 12,230: DataLoader timed out after 10800 seconds
```

The first was raised from inside the pin-memory loop; the second is the same exhaustion reaching the main thread as a stall.

Cause.. Torch’s default sharing strategy on Linux passes every shared tensor storage between a worker and its parent as a separate open file descriptor. A `HeteroData` batch is dozens of tensors and several batches are in flight at once, so the parent eventually cannot receive another handle. GilaHyper never saw this because its `RLIMIT_NOFILE` is 524,288, and no canary can see it because it takes a day of training to appear. Commit `f1bfa95b` switches to the `file_system` strategy in both the parent and a worker hook, and lifts the soft limit to the hard limit. The change moves batches between processes differently and touches no arithmetic.

A raise in a non-main thread does not end the process.. 2368337 stayed `RUNNING` for three hours after its pin-memory thread died, holding a card and writing nothing. A job that is `RUNNING` with a stale log is dead, not slow.

Relaunch.. The eight `mmli` runs sat at epoch $\approx 2,030$, inside the 2,229–2,580 band where both cabbi runs died, so they were restarted from checkpoints written within ninety seconds of the cancellation rather than left to fail unattended. The mechanism round was at epoch 32 and was resubmitted from scratch, which puts all eight of its runs on one source version.

Two consequences for reading the results.. Each restarted run’s history is split across two W&B runs, the first ending near epoch 2,030 and the second continuing from it, and the leaderboard puller reads one run at a time. The declared scoring window of epochs $\leq 9,000$ falls entirely inside the second, so the split changes plotting rather than scoring. Separately, all three point replicates report `pred_sd_ratio` = 0.000 and `pearson_per_feature` = 0.000 at epoch $\approx 2,040$, meaning that head emits one constant for every strain. Whether it escapes that regime by epoch 9,900 is not yet measured.

2026-09-02: the mechanism round is split by seed across two GPU types, on purpose. Seed 1 was stuck. Another user held twelve of `compute-3-3`’s CPUs against a five-day wall, our three running tasks held the rest, and

partition	job	GPUs	runs	contents
mmli (A100)	2369693 0-3	4	8	objective round, resumed from epoch $\approx 2,030$
gpu (A40)	2369691	1	1	point seed 2, from epoch 2,580
gpu (A40)	2369692	1	1	incumbent, from epoch $\approx 12,230$
cabbi (Ada)	2369697 0-1	2	4	mechanism round, seed 0, from scratch
gpu (A40)	2371531 2-3	2	4	mechanism round, seed 1, from scratch

Table 43: The round as relaunched, 2026-09-01, source `bebc5df3` with a clean worktree. The two casualties moved to the stock `gpu` partition, whose six A40s were idle, because cabbi was oversubscribed on CPUs and they would otherwise have waited four days behind the mechanism round. Scores are indexed by epoch rather than by wall clock, so the card change costs time and not comparability. Resuming a packed round needs a checkpoint per run rather than per stage, which `resume_many` reads from a map file that is itself the record of which run resumed from where.

the pending task was the per-gene readout pair, which is the arm the round exists to test. It would have started three days after everything else. Two A40s were idle on `compute-0-2` with all 32 of its CPUs free, which is exactly two tasks at one card and 16 CPUs each.

Placing only the per-gene pair there would have broken the contrast.. The comparison is `R_pergene` against `R_ref` within a seed, so putting one arm on an A40 and its reference on an Ada card puts hardware and arm in the same column. The whole seed-1 half moved instead, and it was restarted from scratch rather than resumed from its epoch-1,529 checkpoints, because a resumed reference would have carried 18% of its history on the other card while its per-gene counterpart carried none. That costs 22 hours on two runs and makes GPU type a clean seed-level block: seed 0 entirely on Ada, seed 1 entirely on one A40 node, and no pair straddling the two.

The epoch budget does not fit the per-gene arms, on either card.. Measured throughput at two runs per GPU is 84 epochs per hour for the reference arms and 64 for the per-gene arms, which carry the extra output parameters. Against a five-day wall the reference arms reach the full 8,500 while the per-gene arms reach roughly 7,600, below the declared window of epochs $\leq 8,000$. That applies to seed 0 as much as to seed 1 and is not a consequence of the move. Either the window comes down to 7,500, which every arm clears, or the per-gene arms need a checkpoint resume for the tail.

First indication, one replicate per arm, matched epochs.. Adding the per-gene readout to the rank-64 basis arm leads at every sampled epoch from 400 onward, by 0.03 to 0.05. On the plain arm it led from epoch 400 to 1,200 and peaked at 0.194, then fell back to 0.174 where the reference also sits, so that pair is level at the last matched point and the peak is a biased order statistic rather than a result. The sharper difference is in the dynamics: the predicted-to-observed spread ratio reaches ≈ 0.50 by epoch 600 with the per-gene readout against $\approx 1,200$ without it, which is what per-gene output parameters would do if the shared projection had been holding every gene to one scale. None of this is significant at one replicate against a screen that resolves ≈ 0.062 .

11 The readouts, 2026-09-08: three rounds, one large effect ✗

Three rounds returned within two days of each other: the 2^4 factorial on Delta (section 9), the mechanism round on IGB (table 38), and the first two days of the metric-aligned objective round, which trains on $1 - r$ directly. Every score below is `roll_max`, the maximum of a centered 5-epoch rolling mean of `val/expression/pearson_per_feature`, read from the full per-epoch history, and every contrast is at a MATCHED budget computed as the smallest final epoch across the runs being compared. The scoring rule is an upward-biased order statistic; it is the same rule on every side of every contrast, so the bias cancels in a difference and not in an absolute number.

11.1 The factorial: embedding content is the only factor that moves the score ✗

Job 21830323 ran 16 cells at two seeds, one run per GPU, four per node. Five of the eight array tasks completed 1,400 epochs; three hit the two-day wall between epochs 990 and 1,198, so the matched budget is epochs ≤ 990 .

Embedding content is worth 0.07, three times the replicate spread.. The ProtT5 cells average 0.129 against 0.061 for random vectors of the same width, and no cell of one level overlaps any cell of the other (fig. 15a). The

factor	contrast (level 1 – level 0)	effect	t	effect, 31 runs	t
embedding	prot_T5_all – random_1024	+0.068	+7.8	+0.076	+17.3
trunk	$L=2, h=45$ – $L=6, h=90$	–0.012	–1.4	–0.018	–4.0
readout	linear – two-layer	+0.007	+0.8	+0.002	+0.5
weight decay	10^{-4} – 10^{-8}	+0.005	+0.5	–0.000	–0.1
embedding $\times$ trunk		+0.017	+2.0	+0.013	+2.9

Table 44: Main effects of the v10 grid at epochs ≤ 990 , 16 runs a side. The error is the pooled within-cell replicate standard deviation, 0.0246 on 16 degrees of freedom over all 32 runs (standard error per effect 0.0087), and 0.0122 on 15 degrees of freedom once the one run that never left the chance band is dropped (standard error 0.0044). The other five two-way interactions have $|t| < 1.3$ in both readings.

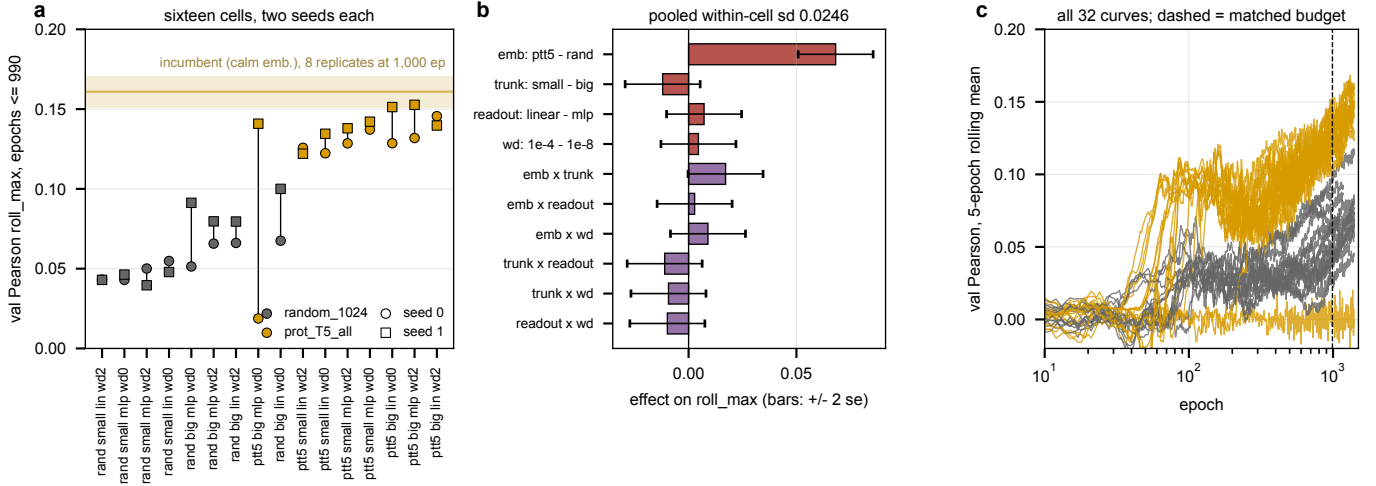


Figure 15: The v10 grid at a matched budget. **a**, every cell's two seeds at epochs ≤ 990 , sorted by cell mean and colored by embedding, against the eight identical-config incumbent runs at 1,000 epochs (0.161 ± 0.010 , calm embeddings, band is one standard deviation). **b**, main effects and two-way interactions with ± 2 standard errors from the pooled within-cell spread. **c**, all 32 validation curves; the dashed line is the matched budget. Generated by `v10_grid_factorial.py`.

contrast was built as a content test at matched width 1,024, so it says the content matters and says nothing about `calm`, the incumbent's embedding, which is not a level of the grid. The one healthy draw of the incumbent-with-ProtT5 cell scores 0.141 against the incumbent's 0.161 ± 0.010 at the same budget, one draw against eight, and that is not a resolved gap in either direction.

Readout width and weight decay are measured nulls.. Both effects are under 0.008 against a resolution of about 0.02 at two standard errors. The trunk is a small effect in favor of the deeper one, -0.012 over all runs and -0.018 once the stuck run is dropped, and it is sharper under ProtT5 (the interaction). Of the four factors chosen to act on the train-validation gap, one acts.

One run in 32 never learned.. Cell 1 seed 0 (te8272kk: ProtT5, $L=6$, two-layer readout, weight decay 10^{-8}) sits at 0.019 at its best and -0.007 at epoch 1,399 with nmse 1.010, while its seed-1 twin is the grid's best single run at 0.169. That pair alone carries most of the pooled variance, which is why both readings are given. Its cause is not measured.

11.2 The mechanism round: per-gene readout $+0.03$, under the round's resolution $\times$

All four array tasks of the round ended in the cgroup out-of-memory handler at host RSS 61 GB against a 60 GB request, after 3.5 to 4.6 days. In each task one of the two packed runs was killed and the other finished its 8,500 epochs, so the matched budget is set by the shortest survivor, epochs $\leq 4,079$. The two seed-1 runs abandoned on cabbi at epoch $\approx 1,595$ when that half moved to the A40 node are superseded, not replicates, and are dropped.

arm	seed 0	seed 1	paired difference vs R_ref (s0 / s1)	mean
R_ref	0.188	0.169		
R_basis64	0.175	0.200	-0.014 / $+0.030$	$+0.008$
R_pergene	0.189	0.224	$+0.001$ / $+0.054$	$+0.028$
R_pergene_basis64	0.214	0.199	$+0.026$ / $+0.029$	$+0.027$

Table 45: The mechanism round at epochs $\leq 4,079$, paired within seed against the in-round reference. Seed 0 ran on cabbi Ada cards, seed 1 on one A40 node, so GPU type is a seed-level block. The incumbent's eight replicates score 0.188 ± 0.017 at 4,000 epochs and both **R_ref** draws fall inside that band.

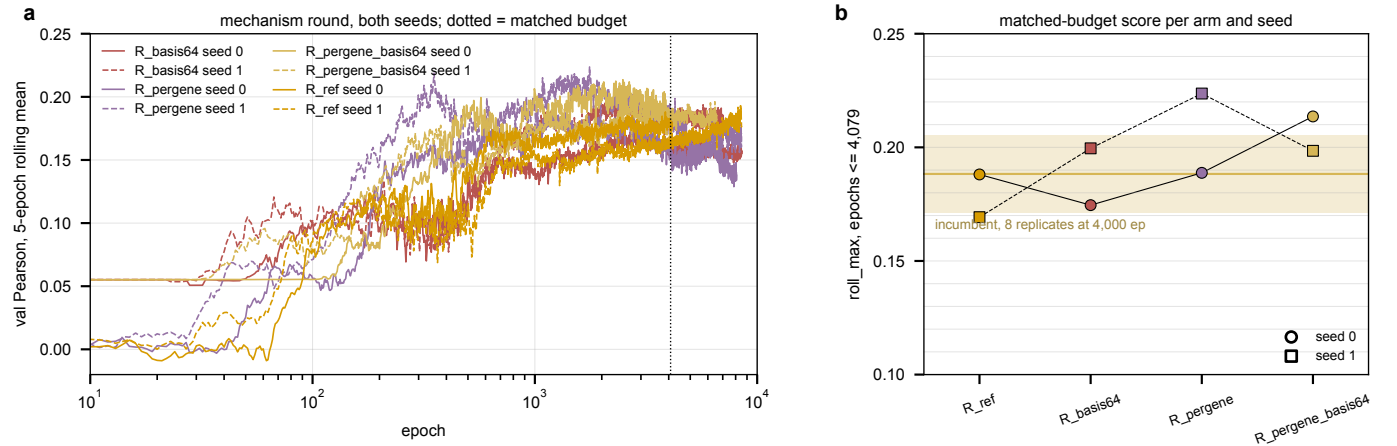


Figure 16: The mechanism round. **a**, validation Pearson of all eight runs (solid seed 0, dashed seed 1); the dotted line is the matched budget. **b**, the matched-budget score per arm and seed against the incumbent band at 4,000 epochs. Generated by `mech_round_readout.py`.

The pair term alone is a null.. Its two paired differences disagree in sign and average $+0.008$. The per-gene readout arms average $+0.027$, and the combined arm is positive at both seeds, but the round was sized to resolve about 0.06 at two pairs (table 38), so this is a direction and not a result. What every curve does show is the dynamics: the per-gene arms reach their peak between epochs 1,200 and 2,600 and give part of it back (**R_pergene** ends at 0.137 and 0.170 against peaks of 0.189 and 0.224), while the reference and basis arms are flat or still rising at 8,500. Whether the give-back is overfitting of the extra per-gene parameters is a hypothesis and is not measured.

A tagging defect, found in the readout.. The arm script carried a wave-4b **R_ref** branch ahead of the wave-5 one; a shell `case` takes its first match, so every reference run of this round is tagged `stage-wave4b`. The readout selects by arm and config tag, and the shadowing branch is removed.

11.3 Packed five-day runs die of host memory ✗

Four of four packed tasks that ran past three and a half days ended `OUT\OF_MEMORY` (2369697 0–1, 2371531 2–3), at `MaxRSS` 61.4 and 61.1 GB. The Pearson-round packed tasks read 21.3 GB at 1.9 days and the solo batch-64 tasks 17.2 GB. Hypothesis (untested): resident memory grows roughly linearly through a run, in which case the packed Pearson tasks reach the 60 GB line near day five, at the edge of their wall. What grows is not identified; the `file_system` sharing strategy introduced in `f1bfa95b` to stop the file-descriptor exhaustion is the obvious suspect and is unverified. The memory of a running job cannot be raised, and the two rounds that finished lost one run per task to it rather than the whole task.